



FACULTY OF
COMPUTING
& INFORMATICS

FINAL YEAR PROJECT REPORT

FYP02-SE-T2510-0032

LLM-Powered Dashboard using Multi-Agent Framework

1211101583

LUQMAN HAKIM BIN NOORAZMI

BACHELOR OF COMPUTER SCIENCE (HONS.)

JUNE 2025

FYP02-SE-T2510-0032 LLM-Powered Dashboard using Multi-Agent Framework

BY

1211101583
LUQMAN HAKIM BIN NOORAZMI

FINAL YEAR PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT OF
THE

REQUIREMENT FOR THE DEGREE OF

COMPUTER SCIENCE

in the

Faculty of Computing and Informatics

MULTIMEDIA UNIVERSITY
MALAYSIA

June 2025

Copyright

© 2025 Universiti Telekom Sdn. Bhd. ALL RIGHTS RESERVED.

Copyright of this report belongs to Universiti Telekom Sdn. Bhd. as qualified by Regulation 7.2 (c) of the Multimedia University Intellectual Property and Commercialisation Policy. No part of this publication may be reproduced, stored in or introduced into a retrieval system, or transmitted in any form or by any means (electronic, mechanical, photocopying, recording, or otherwise), or for any purpose, without the express written permission of Universiti Telekom Sdn. Bhd. Due acknowledgement shall always be made of the use of any material contained in, or derived from, this report.

Declaration

I hereby declare that the work has been done by myself and no portion of the work contained in this report has been submitted in support of any application for any other degree or qualification of this or any other university or institution of learning.



Name of candidate: LUQMAN HAKIM BIN NOORAZMI

Faculty of Computing & Informatics

Multimedia University

Date: 21: 06: 2025

Acknowledgements

I extend my deepest gratitude to my supervisor, Dr. Zarina binti Che Embi, for her unwavering support and insightful critiques throughout my project journey. Her deep commitment to academic excellence and meticulous attention to detail have significantly shaped this dissertation. I am equally thankful to the members of the FYP committee for managing the students with usable guidelines and a plausible timeline of the project.

I am also deeply thankful for my family whose intellectual curiosity at home sparked my interest in computer science from a young age. Their continuous support and belief in my academic pursuits have been instrumental in my success.

This dissertation reflects not only my work but also the collective support of everyone who has touched my life academically and personally.

Abstract

Effective data analysis and interpretation are made possible by data visualization, which is essential for breaking down complicated datasets. There are many data visualization tool available for deriving analysis from raw data into insightful visual. However, a lot of the data visualization tools available today have a high learning curve, which limits their use for non-technical user. This limitation hinders the usability for non-technical user as more complex tasks require more time and resources to achieve the desired visual. There has been attempts by these existing data visualization tools to improve their platform's usability. There has been attempts to incorporate LLM into data visualization process, however none has directly utilized existing chart and graph libraries. This paper proposes a Large Language Model (LLM)-oriented solution to the usability limitation from existing data visualization tools by integrating LLM with the steps taken to produce data visualization. This solution is aimed to reduce the number of steps and actions taken by the user to generate data. By incorporating LLM agents to automate data interpretation and visualization, this paper seeks to implement these solutions into a data visualization tool and assess the effectiveness of the LLM in producing accurate data visualization with minimal number of steps. The project will assess features from existing data visualization tools to gather the required features that makes up a data visualization tool and integrates it with LLM. The proposed data visualization tool will be able to receive prompt from user where a Structured Language Query (SQL) query will be generated by the LLM based on user prompt and data structure. With the query generated, it will be executed to retrieve the relevant data. LLM will convert the data into a structured format before making the best visualization choice. The LLM will also be capable of generating in-depth analysis of the queried data. In the end, user would only have to enter a prompt and the data visualization tool will display the visual of the data.

Table of Contents

Copyright.....	iii
Declaration.....	iv
Acknowledgements	v
Abstract.....	vi
Table of Contents.....	vii
List of Tables	xi
List of Figures	xi
List of Abbreviations/Symbols	xvi
List of Appendices	xviii
Chapter 1: Introduction.....	xix
1.1. Overview	xix
1.2. Problem Statement.....	xix
1.3. Project Objectives.....	xxi
1.4. Project Scope.....	xxi
1.5. Project Limitation	xxii
1.6. Methodology	xxiii
1.6.1. Milestone	xxv
1.6.2. Gantt Chart.....	xxvii
1.7. Target Audience	xxviii
1.8. Summary.....	xxviii
Chapter 2: Background Study	xxix
2.1. Overview	xxix
2.2. Data Visualization	xxx
2.3. Large Language Model.....	xxxiii
2.4. Agent.....	xxxv
2.4.1. Multi-agent Framework	xxxvi
2.5. Natural Language Processing	xxxvii
2.6. Text-to-SQL	xxxix
2.7. LLM and Agents in Data Visualization.....	xl

2.8. Summary.....	xli
Chapter 3: Requirement Analysis.....	xl ii
3.1. Overview	xl ii
3.2. Fact-Finding Techniques	xl ii
3.2.1. Justification.....	xl ii
3.2.2. Observation	xl iii
3.2.3. Questionnaire	xl viii
3.2.4. Requirement Analysis Summary.....	lx ii
3.3. Functional Requirements.....	lx ii
3.4. Quality Requirements	lxx i
3.5. System Context Diagram.....	lxxv
3.6. Data Analysis Plan.....	lxxvi
Chapter 4: System Design	lxxix
4.1. Use Case Diagram	lxxix
4.2. Sequence Diagram.....	lxxx
4.2.1. Submit Prompt.....	lxxxii
4.2.2. Connect to Data Source	lxxxiii
4.2.3. Select Visuals	lxxxiv
4.2.4. Save Visual and Analysis	lxxxiv
4.2.5. Generate SQL	lxxxv
4.2.6. Generate Data Dictionary.....	lxxxv
4.2.7. Generate Visual.....	lxxxvi
4.2.8. Generate Analysis	lxxxvi
4.3. System Interface	lxxxviii
Chapter 5: Implementation	xcii
5.1. Software Development Model	xcii
5.2. Deployment.....	xcv
5.2.1. Implementation Scope	xcv
5.2.2. Software Modules.....	xcvii
5.3. Development Environment	xcix
5.3.1. Programming Languages Used	xcix
5.3.2. Framework and Libraries	c

5.3.3.	IDEs and Tools	cv
5.3.4.	Version Control System.....	cvi
5.3.5.	Operating System.....	cvii
5.4.	System Configuration and Setup	cvii
5.4.1.	Backend Setup	cvii
5.4.2.	Frontend Setup.....	cviii
5.4.3.	Build Tools and Package Managers	cviii
5.5.	Database Implementation.....	cix
5.5.1.	Database Schema Design	cx
5.5.2.	SQL Database Tables.....	cx
5.5.3.	Stored Procedures or Triggers.....	cx
5.5.4.	Tools Used for Database Management	cx
5.6.	Key Modules and Features Developed.....	cxii
5.6.1.	User authentication	cxii
5.6.2.	Database connection.....	cxix
5.6.3.	Prompt handling	cxxiv
5.6.4.	LLM agent.....	cxxvi
5.6.5.	Visualization tools.....	cxliii
5.6.6.	Frontend UI.....	cxlix
5.7.	APIs and Integration	clxxxii
5.7.1.	Description of Internal APIs or Third-Party APIs Used	clxxxii
5.7.2.	API Endpoints Implemented	clxxxiii
5.7.3.	JSON Payload Structure	clxxxvi
5.7.4.	Authentication Mechanisms	clxxxix
5.8.	Security Measures	clxxxix
5.8.1.	Input Validation, Encryption, HTTPS Configuration	cxc
5.8.2.	Role-Based Access Control (RBAC).....	cxc
5.8.3.	Error Handling and Logging	cxc
5.9.	User Interface	cxcii
5.9.1.	Login Page.....	cxciii
5.9.2.	Register Page.....	cxciv
5.9.3.	Main Menu	cxcv

5.9.4.	Profile Page	cxcvi
5.9.5.	Data Source Credentials Page	cxcvii
5.9.6.	Data Preview Page.....	cxcix
5.9.7.	Prompt Field Page.....	cci
5.9.8.	Generated Visual Page	ccv
5.9.9.	Agents Output Page	ccvi
5.9.10.	Saved Visual	ccvii
5.10.	Challenges Encountered and Solutions	ccix
5.11.	Summary	ccx
Chapter 6:	Testing.....	ccxi
6.1.	Unit Testing	ccxi
6.1.1.	User authentication	ccxi
6.1.2.	Database connection.....	ccxiii
6.1.3.	Prompt handling	ccxiv
6.1.4.	LLM agent.....	ccxv
6.1.5.	Visualization tools.....	ccxvi
6.1.6.	Frontend UI.....	ccxvii
6.2.	Integration Testing.....	ccxix
6.3.	Usability Testing	ccxxi
6.4.	Acceptance Testing.....	ccxli
Chapter 7:	Conclusion	ccxliii
References.		ccxliv

List of Tables

Table 1.1: FYP1 tasks description.....	xxiv
Table 1.2: FYP2 task description	xxiv
Table 1.3: FYP1 tasks milestone.....	xxvi
Table 1.4: FYP2 tasks milestone.....	xxvi
Table 2.1: Data visualization tools limitations	xxxii
Table 2.2: Agent's usage within different domains	xxxv
Table 2.3: NLP tasks description.....	xxxviii
Table 3.1: Existing data visualization tools feature description	xlvi
Table 3.2: Data visualization tools features	xlvi
Table 3.3: Features' existence across data visualization tools	xlvi
Table 3.4: Features respondents wish to see in data visualization tools	lviii
Table 3.5: Limitations on existing data visualization tools	lix
Table 3.6: System's functional requirements	lxiii
Table 3.7: Register use case description	lxiv
Table 3.8: Login use case description.....	lxiv
Table 3.9: Connect to data source use case description	lxv
Table 3.10: Submit prompt use case description.....	lxvi
Table 3.11: Select visuals use case description	lxvi
Table 3.12: Generate SQL query use case description.....	lxvii
Table 3.13: Generate data dictionary use case description	lxviii
Table 3.14: Generate visual use case description.....	lxix
Table 3.15: Generate analysis use case description	lxix
Table 3.16: Save visual and analysis use case description	lxx
Table 3.17: View saved visual and analysis use case description	lxx
Table 3.18: View agents output use case description	lxxi
Table 3.19: Visualization per minute quality description.....	lxxii
Table 3.20: SQL Generation response time quality description	lxxii
Table 3.21: Amount of time taken to generate data dictionary quality description	lxxiii
Table 3.22: Amount of time taken to generate visual quality description	lxxiii
Table 3.23: Amount of time taken to generate analysis quality description	lxxiii
Table 3.24: Time for user to familiarize with the system quality description ...	lxxiv
Table 3.25: Percentage portable code quality description	lxxiv
Table 3.26: Number of operating systems supported quality description	lxxiv
Table 3.27: System's data dictionary	lxxvii
Table 5.1: Software modules for development	xcvii
Table 5.2: Register submodule.....	cxii
Table 5.3: Login submodule	cxv
Table 5.4: User session submodule	cxvii
Table 5.5: Logout submodule	cxix

Table 5.6: MySQL database connection submodule	cxx
Table 5.7: Data preview submodule.....	cxxii
Table 5.8: Process prompt submodule	cxxv
Table 5.9: Prompt tags and purposes	cxxvii
Table 5.10: Agents purpose, input, and output type	cxxvii
Table 5.11: Example of expected agents output.....	cxxix
Table 5.12: State object attributes.....	cxxxiv
Table 5.13: Agents submodule	cxxxvi
Table 5.14: Bar chart function annotation and body	cxliii
Table 5.15: Various frontend UIs submodule	cxlix
Table 5.16: Internal API description	clxxxiii
Table 5.17: Third-Party APIs description.....	clxxxiii
Table 5.18: API endpoint method and description	clxxxiv
Table 5.19: JSON payload structure	clxxxvii
Table 6.1: User authentication test plan.....	ccxi
Table 6.2: User authentication test data	ccxi
Table 6.3: Create a new account test case.....	ccxii
Table 6.4: Login with valid credentials test case	ccxii
Table 6.5: Check active user session test case.....	ccxiii
Table 6.6: Logout user and invalidate session test case.....	ccxiii
Table 6.7: Database connection test plan.....	ccxiii
Table 6.8: Connect to database test data	ccxiv
Table 6.9: Connect to database test case	ccxiv
Table 6.10: Prompt handling test plan	ccxiv
Table 6.11: Submit prompt test data	ccxv
Table 6.12: Submit prompt test case	ccxv
Table 6.13: LLM agent test plan.....	ccxv
Table 6.14: LLM agent test data.....	ccxv
Table 6.15: LLM agent test case	ccxvi
Table 6.16: Visualization tools test plan.....	ccxvi
Table 6.17: Choose tools test data.....	ccxvi
Table 6.18: Choose tools test case	ccxvi
Table 6.19: Frontend UI test plan.....	ccxvii
Table 6.20: Full workflow test data.....	ccxvii
Table 6.21: Full workflow test case	ccxviii
Table 6.22: Integration testing results	ccxix
Table 6.23: Tasks and steps to be executed for usability testing	ccxxi
Table 6.24: Usability test 1	ccxxiv
Table 6.25: Usability test 2	ccxxix
Table 6.26: Usability test 3	ccxxxv
Table 6.27: Acceptance test 1	ccxli
Table 6.28: Acceptance test 2	ccxli

Table 6.29: Acceptance test 3	ccxlii
-------------------------------------	--------

List of Figures

Figure 1.1: FYP 1 Gantt chart	xxvii
Figure 1.2: FYP 2 Gantt chart	xxvii
Figure 3.1: Respondents professional status	I
Figure 3.2: Respondents data visualization tools usage.....	li
Figure 3.3: Data visualization tools used by respondents.....	li
Figure 3.4: Respondents proficiency with data visualization tools	lii
Figure 3.5: Data modelling feature rating	liii
Figure 3.6: Advanced analysis feature rating	liv
Figure 3.7: Automated visualization feature rating	lv
Figure 3.8: Data storytelling feature rating	lvi
Figure 3.9: Collaboration feature rating	lvii
Figure 3.10: System's context diagram (Anisya et al., 2021).....	lxxv
Figure 3.11: System's entity relationship diagram (Rashkovits & Lavy, 2021)	lxxvii
Figure 4.1: System's use case diagram (Sudiarta et al., 2021).....	lxxix
Figure 4.2: Submit prompt sequence diagram (Al-Fedaghi, 2021).....	lxxxii
Figure 4.3: Connect to data source sequence diagram (Al-Fedaghi, 2021).	lxxxiii
Figure 4.4: Select visuals sequence diagram (Al-Fedaghi, 2021).	lxxxiv
Figure 4.5: Save visual and analysis (Al-Fedaghi, 2021).....	lxxxiv
Figure 4.6: Generate SQL sequence diagram (Al-Fedaghi, 2021)	lxxxv
Figure 4.7: Generate data dictionary sequence diagram (Al-Fedaghi, 2021).....	lxxxv
Figure 4.8: Generate visual sequence diagram (Al-Fedaghi, 2021).....	lxxxvi
Figure 4.9: Generate analysis sequence diagram (Al-Fedaghi, 2021).....	lxxxvi
Figure 4.10: View saved visual and analysis sequence diagram (Al-Fedaghi, 2021).....	lxxxvii
Figure 4.11: View agents output sequence diagram (Al-Fedaghi, 2021).....	lxxxvii
Figure 4.12: Welcome screen	lxxxviii
Figure 4.13: Login screen.....	lxxxix
Figure 4.14: Registration screen.....	lxxxix
Figure 4.15: Prompt field screen.....	xc
Figure 4.16: Prompt field screen (with sidebar open)	xc
Figure 4.17: Prompt field screen (with visual and analysis).....	xc
Figure 5.1: Waterfall model diagram (Pargaonkar, S., 2023).....	xciii
Figure 5.2: Full agents workflow	cxxxii
Figure 5.3: Login page	cxciii
Figure 5.4: Registration page.....	cxciv
Figure 5.5: Main menu page	cxcv
Figure 5.6: Profile page	cxcvi
Figure 5.7: Data source credentials page.....	cxcvii
Figure 5.8: Data source credentials page after successful connection	cxcviii

Figure 5.9: Database preview page	cxcix
Figure 5.10: Database preview page after selecting table	cc
Figure 5.11: Prompt field page	cci
Figure 5.12: Prompt field page with tool options visible	ccii
Figure 5.13: Prompt field page with database connection status visible	cciii
Figure 5.14: Prompt field page with model name visible	cciv
Figure 5.15: Generated visual page.....	ccv
Figure 5.16: Agent output page.....	ccvi
Figure 5.17: Saved visual list page	ccvii
Figure 5.18: Detailed saved visual page.....	ccviii

List of Abbreviations/Symbols

Word/Phrase	Abbreviation
Atomicity, Consistency, Isolation, and Durability	ACID
Artificial Intelligence	AI
AI Based Operating System	AIOS
Bidirectional Encoder Representations from Transformers	BERT
Dictionary	dict
Etcetera	etc.
Final Year Report	FYP
Generative Pre-Training Transformer	GPT
Identification	ID
Integrated development environment	IDE
Integer	int
JavaScript Object Notation	JSON
JWT	JSON Web Token
Large Language Model	LLM
Language Processing Unit	LPU
Model-view-controller	MVC
My Structured Query Language	MySQL
No Date	n. d.
Natural Language Generation	NLG
Neutral Language Model	NLM
Natural Language Processing	NLP
Natural Language Understanding	NLU
Node Package Manager	npm
pip Install Packages	pip

Pre-trained Language Model	PLM
Role-Based Access Control	RBAC
Requests Per Minute	RPM
Rate per Minute	RPM
Software Development Life Cycle	SDLC
Small Language Model	SLM
Structured Query Language	SQL
String	str
Test Case	TC
Task Planning and Tool Usage	TPTU
User Acceptance Test	UAT
User Interface	UI
Uniform Resource Locator	URL

List of Appendices

Appendix	Content
A	FYP I & II Meeting Logs
B	Turnitin Similarity Index
C	Agents Prompt

Chapter 1: Introduction

1.1. Overview

Data visualization tools are essential in data analysis as they transform large and complex datasets into clear and interpretable visual representations. Over time these tools have evolved from basic charts and maps to advanced interactive displays that makes it easier to understand and analyze information. Today, they play a crucial role in data-driven decision-making with a variety of options available that offer unique strengths and limitations (Lavanya et al., 2023).

With the rapid progress of artificial intelligence (AI) Large Language Models (LLM) have become a key part of modern technology. Models such as ChatGPT and GPT-4 from OpenAI are known for their natural language understanding and reasoning abilities. They can handle a wide range of tasks and significantly improve human-computer interaction (Shen, 2024).

1.2. Problem Statement

Many existing data visualization tools have different limitations. Some are hard to use, others lack useful features, and some do not support automation. While certain tools try to improve usability by adding user-friendly options or basic automation, these improvements are often limited to specific functions (Lavanya et al., 2023; Skender & Manevska, 2022).

Some systems try to make data easier to understand by using plain language or simple interfaces (Lavanya et al., 2023; Skender & Manevska, 2022). Others include features that help users explore data without needing technical skills

(Skender & Manevska, 2022). However, these changes only improve certain areas of data visualization process.

Large language models (LLMs) can help with many tasks, but they are not always accurate. They may give wrong answers or struggle with specific situations. This is because they rely on patterns from past data and do not always adapt well in real time (Shen, 2024).

Some efforts use language models to suggest how to visualize data, but they are limited in scope as they go as far as suggesting the best visual (Wang, Zhang, Wang, Lim, & Wang, 2023; Shi, Ren, & Hu, 2025). There is still a gap in creating systems that can fully explore advanced tools and choose the best way to show information automatically.

1.3. Project Objectives

Based on the usability limitation for non-technical user in using data visualization tools and the underutilization of LLM in data visualization process, this paper aims:

1. To overcome the usability for technical user in using existing data visualization tool by implementing LLM agents into a data visualization tool as part of improving the usability for non-technical user.
2. To implement multiple agents to assist with features from existing data visualization tool.
3. To allow user to have minimal interactions with the data visualization tool to generate data visualization.
4. To let LLM decide the best suited visualization for a set of data and using built-in functions from data visualization library to execute the function and display it to the user.

1.4. Project Scope

The size of the project is considered as small as it only aims to reuse concepts from existing systems and improving it using the proposed solution of LLM. The technology required mainly revolves around existing open source LLM and a framework that supports the language model. In addition, LLM agents will be used as part of leveraging LLM into data visualization tool. The functions to be executed by the agent will be retrieved from Plotly libraries. The type of visualization for the agent to consider are Scatter Plot, Bar Chart, Choropleth, Pie Chart, and Table.

The data visualization tool will be developed in a React.js and Flask environment, where React.js is used to manipulate the interface and display the correct information to the user, while Flask is used to implement the agents and the system's logic.

1.5. Project Limitation

Based on the scope defined for the project, the limitations include time constraint. In a 14-week time frame mixed with other commitment, the depth of the analysis and implementation might be restricted. Some features that require more than implementing existing resources may not be feasible during this time frame.

Open-source LLM may have its limitations compared to proprietary model, hence the result and response are reliance on its pre-trained knowledge. With the limited token of 100,000 combined input and output token per day, the amount of prompt for each agent must be limited to prevent overuse of token. The prompt that gives the best result using minimal token should be passed to each agent for providing instructions.

The visual for the agent to choose from is also limited to five (5) graph or chart due to the complexity of these visuals are considered low, hence catering for the limited token usage for the agent, compared to high complexity graph or chart that may require extra prompt for the agent to accurately pass the required parameter for the function.

1.6. Methodology

The traditional waterfall model of SDLC includes several phases of System Planning, Systems Analysis, System Design, System Implementation, and System Maintenance (Nurcahya et al., 2022). For this project, the phases include planning, analysis, design, and implementation. These phases will be spread out for two trimesters with FYP1 being the first part of the project and FYP2 being the second part of the project.

In this project, analysis on existing data visualization tools will be done to assess features that makes up a data visualization tool. In conjunction to the analysis, a questionnaire will be designed to understand which of the features are more likely to be implemented and which will be discarded. From the analysis, the design phase is to generate the logical design of the system with the help of Unified Modeling Language (UML) diagrams. With the baseline of the data visualization tool identified, a prototype of the data visualization tool will be implemented where the solution is to be integrated within the prototype.

The solution of integrating LLM with data visualization tool is proposed and will be planned, analyzed, designed, developed, and tested within a 28-week time frame. Throughout the 28 weeks, a meeting will be held every two weeks to update the project's progress and plan the goal until the next meeting. Table 1.1 and table 1.2 displays the task planned during the FYP1 and FYP 2 project execution and description of the outcome of from each phase.

Table 1.1: FYP1 tasks description

Week Number	Task	Description
1	Problem Formulation and Project Planning & Background Study	The phase to define the problem statement, project objectives, and scope. Background research is conducted to review relevant literature, existing systems, and technologies. A timeline is created to guide project execution.
2		
3		
4		
5	Requirement Analysis	This phase focuses on gathering, analyzing, and documenting functional and non-functional requirements for the project.
6		
7		
8		
9	Design	In this phase, the system architecture, use case diagram, and sequence diagram are designed based on the requirements.
10		
11		
12	Prototype Development	Develop the prototype of the system based on the designed architecture and action flow.
13		
14	Submission	Project submission.

Table 1.2: FYP2 task description

Week Number	Task	Description
1	Implementation	The implementation of the planned system is executed. The software is split between modules and implemented module by module until completion.
2		
3		
4		
5		
6		
7		
8	Testing	This phase is focused on applying testing techniques to the
9		
10		

11		completed modules and test the effectiveness of these modules when integrated with other modules.
12	Draft Report Completion	This phase marks the completion of the implementation and testing phase, where the implementation and testing result is described and included within the draft report. The report is also updated to cater for any changes done during phase 2 of the project.
13	Final Report Completion	The report draft is refined to its final form.
14	Submission	This phase marks the end of the project with the final report submission.

1.6.1. Milestone

Throughout these two parts of the projects and as part of the System Planning phase, a set of deadlines for the project's milestone is provided. This is to ensure the project will be ongoing and stay within its track until the end. Table 1.3 and table 1.4 displays the FYP1 and FYP 2 tasks with its proposed due date.

Table 1.3: FYP1 tasks milestone

Week	Task	Due Date
3	Project Planning	22/11/2024
5	Background Study	06/12/2024
7	Requirements Determination	20/12/2024
9	Requirements Structuring	03/01/2025
11	Design Specification Finalization	17/01/2025
12	Report Draft Completion	24/01/2025
13	Prototype Completion	31/01/2025
14	Report Completion & Submission	09/02/2025

Table 1.4: FYP2 tasks milestone

Week	Task	Due Date
1	FYP1 Review	28/03/2025
7	Software Module Completion	09/05/2025
11	Testing & Analysis of Result	06/06/2025
12	Draft Report Completion	13/06/2025
13	Final Report Completion	20/06/2025
14	Submission	29/06/2025

Each milestone will be verified through regular meetings throughout the project's duration. The project planning phase is part of the SDLC's planning phase. The analysis phase includes activities of background study, requirements determination, and requirements structuring. Then, the design phase is executed to finalize the design specification. By this point of the project, the report draft is to be completed before finishing with the prototype. The report is then submitted before moving on to FYP2.

The FYP2 continues with the implementation phase that include the modules completion and testing. After the completion of software development,

the testing result and analysis is included within the report before being finalized. The project ends with report submission by the end of the trimester.

1.6.2. Gantt Chart

To visualize the progress for FYP1 and FYP2, the Gantt chart that depicts the project's progresses and phases throughout the trimester can be seen in figure 1.1 and figure 1.2.

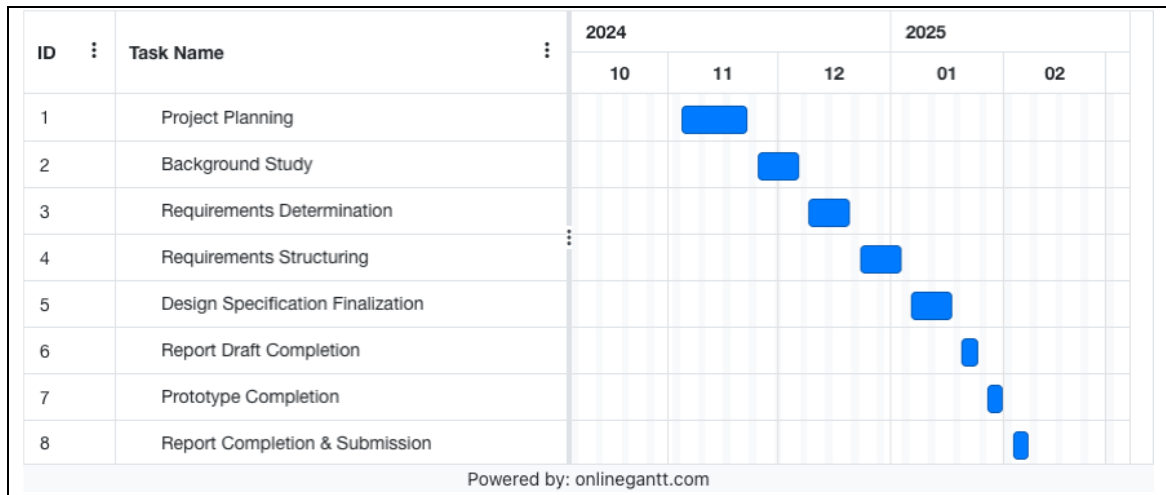


Figure 1.1: FYP 1 Gantt chart

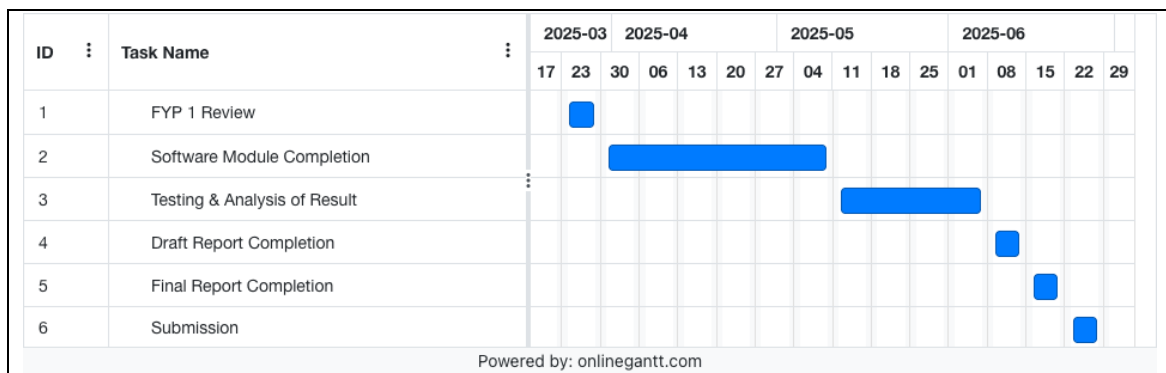


Figure 1.2: FYP 2 Gantt chart

1.7. Target Audience

The target audiences are mainly university students, specifically computer science students, learning and research purposes. Data visualization tools user or data scientists could benefit from this project's approach of visualizing data as the proposed tool is similar to other existing data visualization tools. LLM practitioner could benefit from the integration of LLM by assessing the implementation and outcome from the project.

1.8. Summary

In summary, this project proposes an end-to-end solution to the usability problem in data visualization tools and the underutilization of LLM in data visualization by integrating LLM agents into data visualization tool. The goal is to reduce the complexity of data visualization process using LLM by making the user interaction with data visualization tool as minimal as possible. Using Plotly visualization library, the agent will assist in executing the chosen visualization and the generated visual will be displayed to the user.

The methodology of this project involves assessing existing visualization tools, conducting user research, designing a logical system architecture, and developing a prototype, and apply testing techniques to the developed prototype.

Chapter 2: Background Study

2.1. Overview

This section explores key technologies and concepts related to data visualization and its tools, text-to-SQL, LLM agent, and the use of LLM in data visualization. Data visualization aids in simplifying complex information by representing data visually through existing data visualization tools, while LLM is used to process and generate response in a human-like text. Autonomous agents enhance LLM's ability to operate independently in explicit environments. NLP bridges the gap between human language and computers, and text-to-SQL systems further improve accessibility by allowing users to query structured databases using natural language. These advancements contribute to making AI-driven tools more intuitive and efficient across various industries. LLMs have also been used to suggest chart types.

2.2. Data Visualization

Data visualization is a method of displaying data in its visual representation. Data visualization tools aids data analysts in interpreting data and draw conclusions from it before making the final decision. Many data visualization tools have been developed with the goal of better observation to the data, which essentially lead to quicker and better decisions to be made. To achieve this goal, the visualization aspect becomes the main focus, as innovative techniques that affects the representativeness of the data is important, and this include primitives such as colors, elements, and dimensions, and analyses (Skender & Manevska, 2022).

There are many forms of visualizations data can be disguised as depending on the type of information being unveiled. Some form of visualization includes bar charts that are effective for comparing different categories or illustrating data distribution. Horizontal bar charts are beneficial for lengthy category label. Line charts are used to visualize trends over time, frequently employed in financial, scientific, and time-series data analyses by connecting data points with lines. Pie charts rely its effectiveness on limited number of categories for representing parts of a whole or composition of a dataset. Scatter plots make use the x-axis and y-axis as variables and plotting them against the data points as dots on the graph, making it valuable for spotting outliers, identifying clusters, and observing relationships between variables (Choudhary et al., 2024).

Data visualization tools of today exist with its own strengths and weaknesses. Some of the tools include Microsoft Power BI, which is best for business intelligence. Tableau is known for creating interactive charts while Qlik Sense excels in artificial intelligence-driven analytics. Choosing the right tool depends on specific needs as the field of data visualization continues to evolve. Data visualization tools have a long history, progressing from simple maps and charts

to advanced interactive displays. Today they are essential for understanding complex information and making data-driven decisions. While there are many options for those entering the field, these tools also come with limitations (Lavanya et al., 2023). Table 2.1 displays the limitations of some existing data visualization tools and its limitations.

Table 2.1: Data visualization tools limitations

Data Visualization Tool	Limitation(s)	Cited Sources
Power BI	Difficult learning curve	Skender & Manevska (2022), Lavanya et al. (2023)
Tableau	No option to refresh report	Skender & Manevska (2022), Lavanya et al. (2023)
	No auto-scheduling when manually updating data	
Oracle Analytics Cloud	Limited graphics	Skender & Manevska (2022)
Google Data Studio	Difficult learning curve	Lavanya et al. (2023)
	Limited functionalities	
	Data processing limitation	
D3.js	Difficult learning curve	Lavanya et al. (2023)
	Complex set up	
	Tough for beginners	
Matplotlib	Difficult learning curve	Lavanya et al., (2023)

Plotly	No automation process	Lavanya et al. (2023)
DOMO	Not intuitive interface	Skender & Manevska (2022)

Based on table 2.1, various data visualization tools are proved to have different limitations. Power BI has a steep learning curve. Tableau does not have an option to refresh reports and does not support auto-scheduling for manual updates. Oracle Analytics Cloud has limited graphic options. Google Data Studio is difficult to learn and has limited functionalities. It also faces restrictions in data processing. D3.js requires a complex setup and is hard for beginners to use. Matplotlib is also difficult to learn. Plotly does not support automation and DOMO has an interface that is not intuitive.

Tableau promotes usability through some interactive and easy to use features by providing drag-and-drop feature, implemented a data viewer through Tableau Reader, allows data size manipulation, introduced interactive table, provide highlight and filter for data, and automatically display new features to name a few. These features are useful to user when generating data visualization.

Attempt to bridge the technical gap between non-technical user and data visualization tool can be further assessed through Oracle Analytics Cloud. Oracle Analytics Cloud is a data visualization tool for reporting and analytics in organizations of any size. It uses machine learning to detect important information through clear visualizations. One of its main benefits is processing data in natural language. Users do not need coding skills to explore and analyze data. The tool also supports multidimensional data analysis and scenario simulations. These features help provide a detailed view of business data even for non-technical user (Skender & Manevska, 2022).

M Cognos Analytics includes an AI Assistant that responds to inquiries in plain language. It suggests new data visualizations and joins to help users discover hidden relationships in data. The AI capabilities also assist enterprises in identifying patterns in customer behavior, market trends, and business performance. These insights help businesses make informed decisions with ease. The AI-powered features enhance self-service analytics, making data exploration more accessible to users (Lavanya et al., 2023).

From these examples of solutions to improve usability in data visualization tools, it can be summarized that tableau provides easy-to-use and interactive feature, Oracle Analytics Cloud leverages NLP for processing data, and M Cognos Analytics integrated AI Assistance to serve user in plain language by suggesting visualization, data relationship, and market trends for business decision. These attempts only enhance specific area of the data visualization tools, and the use of NLP are only targeted towards understanding the data.

2.3. Large Language Model

The development of language models has gone through four key stages. The first stage, Statistical Language Models (SLMs), emerged in the 1990s and used basic mathematical techniques to predict words based on past patterns. While helpful in early search engines and text-based applications, these models struggled to handle large amounts of data effectively. The second stage introduced Neural Language Models (NLMs) which used more advanced learning methods inspired by how the human brain processes language. These models could better understand relationships between words, making them more useful for applications like speech recognition and translation. The third stage saw the rise of Pre-trained Language Models (PLMs) which took a different approach by

first learning from massive amounts of text before being fine-tuned for specific tasks. Models like BERT and GPT-2 made significant improvements in text understanding and became the foundation for many modern AI applications. The final stage, Large Language Models, built on this idea by significantly increasing the amount of data and computing power used for training. Models like GPT-3 and ChatGPT demonstrated new abilities, such as answering questions, writing human-like text, and reasoning through complex problems. This evolution has transformed language models from simple text predictors into powerful AI systems capable of assisting with a wide range of tasks, from customer service to creative writing and beyond. LLM is becoming a crucial part of modern system interaction due to the ongoing advancement in the Artificial Intelligence field. Models such as ChatGPT and GPT-4 from OpenAI have remarkable reputation for understanding natural language, which allows them to accomplish a variety of tasks and greatly improve human-computer interaction (Zhao et al., 2023).

While large language models (LLMs) are highly versatile, they face challenges when applied to specialized fields or tasks that require high accuracy and real-time decision-making. Their responses are based on patterns learned from vast amounts of data but they do not guarantee precision especially for tasks involving complex calculations or the latest information. This can lead to errors or misleading answers which often referred to as hallucinations. The main reason for these limitations is that LLMs are trained on past data. They cannot account for every real-time situation which makes it difficult for them to consistently provide fully reliable results in specialized areas (Shen, 2024).

2.4. Agent

To overcome the difficulty in specialized areas, LLM agents are utilized. Autonomous agents have been recognized as intelligent entities capable of accomplishing specific tasks by perceiving the environment, planning, and executing actions. Research done on agents have come up with various type of agents varying in purpose from general use to agent-based operating system. Table 2.2 displays the different agents that have been introduced and the process involved within the agents.

Table 2.2: Agent’s usage within different domains

Agent	Method	Advantage	Disadvantage	Cited Sources
Large Language Model-based AI Agents	Task Planning and Tool Usage (TPTU)	Sequentially Mimics Human Problem-Solving	Misunderstanding Output Formats	Ruan et al. (2023)
		Richer Contextual Understanding	Struggling to Grasp Task Requirements	
		Flexibility in Task Management	Endless Extensions	
		Improved Learning From History	Lack of Summary Skills	

AIOS (LLM-based AI Agent Operating System)	Application Layer for integrating native/non-native agents and Kernel Layer for LLMs integration	Resource Isolation & Management	Resource Management Challenges	Mei et al. (2024)
		Improved Agent Performance	Memory and Storage Constraints	
		Faster Execution Time	Security and Privacy Risks	
		Higher System Throughput	Tool Integration and Execution Conflicts	
		Scalability	Scalability Concerns	

The examples of agents from table 2.2 depict agent’s range of specificity. This proves the usefulness of agents in automating complicated tasks in specific fields to overcome the limitation of LLM.

2.4.1. Multi-agent Framework

By facilitating cooperation between agents with specialized functions, multi-agent systems enhance the capabilities of single LLM agents (Talebirad & Nadiri, 2023). They collaborate to finish tasks that are too big for one agent to do alone. Every agent contributes to a common objective and possesses special skills. Tasks requiring in-depth thought and creativity benefit greatly from this teamwork, which includes exercises like discussion and introspection.

Recent research has examined their potential in domains including reasoning in a debate environment (Liang et al., 2023), and teacher-student role play (Jinxin et al., 2023), demonstrating that multi-agent systems are capable of managing challenging real-world problems.

2.5. Natural Language Processing

Since not everyone is fluent in programming or machine-specific languages, NLP helps bridge this gap by allowing users to interact with computers using human language. NLP is a branch of AI and Linguistics designed to help machines interpret and process human-written text. Its primary purpose is to simplify human-computer interactions by enabling communication in natural language. NLP is generally divided into two key areas: Natural Language Understanding (NLU), which focuses on interpreting text, and Natural Language Generation (NLG), which involves producing human-like text. Linguistics, the study of language, includes aspects like phonology (sound), morphology (word formation), syntax (sentence structure), semantics (meaning), and pragmatics (contextual understanding). Theoretical linguist Noam Chomsky, known for his syntactic theories, played a major role in revolutionizing the field of language structure and understanding. Meanwhile, Natural Language Generation (NLG) involves creating meaningful sentences, phrases, or entire paragraphs from structured data (Khurana et al., 2023).

Many NLP research has been driven by computer scientists, but professionals from linguistics, psychology, and philosophy have also contributed to its advancements. NLP not only enhances computer capabilities but also deepens our understanding of human language (Khurana et al., 2023).

NLP has been applied into various areas like Machine Translation, Email Spam detection, Information Extraction, Summarization, Question Answering (Khurana et al., 2023). To implement NLP into these areas, several tasks are executed by NLP, as shown in table 2.3.

Table 2.3: NLP tasks description

NLP Task	Description
Automatic Summarization	Generating concise summaries from long texts.
Co-Reference Resolution	Identifying words that refer to the same entity in a sentence or document
Discourse Analysis	Studying how language is used in different social and conversational contexts
Machine Translation	Automatically translating text from one language to another
Morphological Segmentation	Breaking words into smaller meaning-based units
Named Entity Recognition (NER)	Identifying and classifying names, places, and organizations in text
Optical Character Recognition (OCR)	Converting handwritten or printed text into a machine-readable format
Part-of-Speech Tagging	Labelling words in a sentence based on their grammatical role

Nowadays, with technological advancements, LLM have enabled a wide range of NLP tasks to be performed within a single generative framework. Tasks

such as mathematical reasoning, text summarization, machine translation, information extraction, and sentiment analysis can now be handled more efficiently, achieving impressive results. Furthermore, some LLMs can perform NLP tasks without requiring additional training data, and in some cases, they even outperform traditional models that have been fine-tuned using supervised learning (Qin et al., 2024).

2.6. Text-to-SQL

Text-to-SQL systems function as a bridge between human language and structured databases by converting natural language queries into executable SQL commands. This technology makes databases more accessible to non-experts, enabling them to retrieve relevant information without learning SQL syntax. As databases become more complex, manually searching through data becomes inefficient, making text-to-SQL a valuable tool for simplifying data exploration.

While text-to-SQL systems are useful, building reliable and accurate models is challenging. Linguistic Complexity and Ambiguity causes natural language to be vague or contain complex sentence structures which makes it difficult for the system to interpret correctly. To generate precise SQL queries, the system must understand database structures including tables, columns, and relationships. Since database schemas differ across industries, correctly mapping natural language to a database is challenging. Some queries require advanced SQL functions such as nested subqueries, multi-table joins, or window functions, which can be difficult for models to generate accurately, especially if they have not encountered such operations during training. And lastly, models that are

trained on specific databases may struggle to perform well in entirely different fields due to differences in terminology, query styles, and database structures.

As organizations increasingly depend on relational databases to handle vast amounts of data, the need for efficient and user-friendly querying systems is more important than ever. According to the 2023 Stack Overflow Developer Survey, 51.52% of professional developers use SQL, making it one of the most commonly used programming languages. However, for non-technical users, SQL can be difficult to learn and use. Text-to-SQL helps eliminate this barrier by enabling natural language interactions with databases, allowing businesses to make faster and more data-driven decisions (Mohammadjafari et al., 2024).

2.7. LLM and Agents in Data Visualization

One major strength of LLM is the ability to reason. Due to this, LLM has been able to perform complex tasks such as mathematical reasoning (Wei et al., 2022), visual question answering (Yang et al., 2022), and tabular classification (Hegselmann et al., 2023) which indicates the extent of LLM's ability in making decision.

LLM4Vis is a ChatGPT-based visualization recommendation that explains the reason behind a visualization choice using a human-like explanation (Wang et al., 2023).

PlotGen uses multiple agents to generate data visualization based on user prompt. The agents are responsible to plan out the query, generate pseudocode and converts it into a Python code, and give numeric, lexical, and visual feedbacks to the generated code (Goswami et al., 2025).

2.8. Summary

To conclude the key findings from this section, data visualization is responsible for transforming raw information into visual formats such as charts and graphs, but the tools available suffers from high-level usability which limits the tools usage to only those with the technical skills. Existing data visualization tools have attempted to tackle this problem by utilizing NLP for non-technical user to easily interact with the tool but the solution is only applied on specific part of the tool rather than making a full-fledged NLP-oriented solution.

With LLM that have progressed from statistical models to large-scale AI systems like GPT-3 and ChatGPT it improves human-computer interaction but does not produce satisfying result in specific environment. AI agents, a solution to the LLM's limit in specific settings, assists in automating specific tasks with systems like AIOS optimizing resource management and task execution.

NLP enables machines to process and understand human language, supporting applications like machine translation, sentiment analysis, and speech recognition. Text-to-SQL systems address the challenge of database querying by converting natural language into SQL commands that makes structured data more accessible to non-technical users.

Despite the data visualization tools advancements and evolution of LLM, these technologies continue to face challenges that hinders its full potential. The introduction of agents which is the result of LLM's advancement and text-to-SQL which is the result of NLP's advancement in recent time proves that NLP and LLM have evolved to overcome their own limitations.

LLM and multi-Agent framework has been utilized to automate data visualization process, where LLM4Vis recommends visualization, while PlotGen automate the whole data visualization process using multi-agent framework.

These works prove the effectiveness of LLM in automating data visualization process.

In summary, these advancements from the domains mentioned in this section will be used to assist in implementing the solution to the difficult learning curve problem that exists in existing data visualization tools.

Chapter 3: Requirement Analysis

3.1. Overview

The system is aimed to solve the problems identified earlier using the proposed solutions of multi-agent framework. Before getting started with the solution, the system that qualify as a data visualization tool must be developed first for these solutions to be integrated inside the system. This section will present the process of gathering requirements for the system through the mean of observing existing system and designing questionnaire, analyzing the results from the observation and questionnaire, and finalizing the functional requirements and quality requirements of the system.

3.2. Fact-Finding Techniques

3.2.1. Justification

To better understand the features that makes up a cutting-edge data visualization tool, two elicitation techniques were used: observation and questionnaire. While both aim to gather requirements, these two techniques offer

different methods and steps to generate analysis that will be used to decide the final requirements of the system.

Through observation, analysis is derived from features that exist within the existing systems by understanding its usage amongst users (Gobov & Huchenko, 2021). Questionnaire on the other hand helps with understanding user's feeling towards the identified features (Kircher & Zipp, 2022). The features that were identified from the observation will be presented inside of the questionnaire to filter the features.

3.2.2. Observation

3.2.2.1. Preparation

Before starting the observation, some resources were prepared. First, the goal of the observation is established, which is to collect information regarding features from existing data visualization tools. The existing data visualization tools to be observed are amongst the four tools mentioned in the background study. The list of features available from these tools can be accessed through the respective tools' documentation.

Table 3.1 displays the description of the tools involved during the observation process.

Table 3.1: Existing data visualization tools feature description

Data Visualization Tool	Description	Cited Sources
Power BI	A data visualization tool that helps users transform data from different sources into interactive and reliable insights. It supports data from Excel files, cloud-based systems, and on-premises databases. Users can create business reports and dashboards to analyze data. The visualized information can be shared with others, making data exploration easier.	Biswal (2024)
		Microsoft Corporation, Inc. (2024)
Tableau	A comprehensive and adaptable data visualization tools that incorporates interactive visualization to harmonize, manage, and explore data integrated from various data sources.	Staff (2024)
		Tableau Software, LLC. (n.d.)
Looker Studio	A data visualization tool to create modifiable charts and tables with collaborative report to share insight to team member or broader audience. Reports can be personalized using pre-built template to present data from various data source.	Casarotto (2021)
		Google LLC. (2025)
	Helps business analysts and consumer with advanced, AI-driven self-service tools for data	Argano, (2022)

Oracle Analytics Cloud	preparation, visualization, enterprise reporting, augmented analysis, and natural language processing.	Oracle Corporation, (n. d.)
------------------------	--	-----------------------------

3.2.2.2. Execution

With everything prepared, the observation process begins by gathering features from existing data visualization tools. Table 3.2 displays the list of features gathered across the different tools and its generalized description.

Table 3.2: Data visualization tools features

Feature	Description	Cited Sources
Data Connection	Connect to various data sources ranging from excel workbook, CSV, XML, JSON to relational database such as MySQL, Oracle, and db2.	Microsoft Corporation, Inc. (2024)
		Tableau Software, LLC., (n. d.)
		Google LLC. (2025)
		Oracle Corporation. (n. d.)
Report	Create report with visualization and data insight.	Microsoft Corporation, Inc. (2023)
		Google LLC. (2025)
		Oracle Corporation. (n. d.)
Chart/Visualization	Create suitable visualization for representing data	Microsoft Corporation, (Inc., 2023)

		Microsoft Corporation, Inc. (2025)
		Tableau Software, LLC. (n. d.)
		Google LLC. (2025)
		Oracle Corporation. (n. d.)
Dashboard	Create customized dashboard with selected visualization	Microsoft Corporation (Inc., 2025)
		Tableau Software, LLC. (n. d.)
		Oracle Corporation. (n. d.)
Collaboration	Collaborate with other user on a shared data	Microsoft Corporation (Inc., 2024)
		Google LLC. (2025)
Automated Visualization	Automatically generate data visualization based on the data	Tableau Software, LLC. (n. d.)
Advanced Analysis	Automatically generate analysis from data	Tableau Software, LLC. (n. d.)
Data Storytelling	Automatically generate story to present data	Tableau Software, LLC. (n. d.)
Data Modelling	Create data model to display relationships	Microsoft Corporation (Inc., 2024)
		Oracle Corporation. (n. d.)

3.2.2.3. Result

The result of the observation is presented in table 3.3 To grasp each feature's importance in a data visualization tool, comparison is done across various data visualization tools to see how frequent a feature exists amongst data visualization tools.

Table 3.3: Features' existence across data visualization tools

Features	Power BI	Tableau	Looker Studio (Google Data Studio)	Oracle Analytics Cloud
Data Connection	yes	yes	yes	yes
Report	yes		yes	yes
Chart/Visualization	yes	yes	yes	yes
Dashboard	yes	yes		yes
Collaboration	yes		yes	
Automated Visualization		yes		
Advanced Analysis		yes		
Data Storytelling		yes		
Data Modelling	yes			yes

Based on table 3.3, two features that stand out are Data Connection and Chart/Visualization as these exist in all four of the observed data visualization tools. Report and Dashboard exist in most of the tools, while features like Collaboration and Data Modelling only exist in half of the observed tools. The remainder, Automated Visualization, Advanced Analysis, and Data Storytelling only exist in only one tool.

3.2.2.4. Analysis

To summarize the findings from the observation, a feature's importance in a data visualization tool is measured based on how often it appears in various data visualization tools. It was found that Data Connection and Chart/Visualization are the most important, while Report and Dashboard are considered highly important in a data visualization tool. The remainder of the features are deemed useful but not as crucial and important.

3.2.3. Questionnaire

3.2.3.1. Preparation

With the features classified from the observation of its usage amongst users, the questionnaire is to help with analyzing its usefulness from user's perspective. To design a questionnaire, the project's aims, questions, and hypotheses need to be established (Kircher & Zipp, 2022).

The background study proposed the goal of leveraging LLM into data visualization tool. The questionnaire will help with answering question of what features makes up an up-to-date data visualization tool. This project is predicted to solve the problems of usability for non-technical user within existing data visualization tools.

With the baseline of the questionnaire in mind, the designing phase of the questionnaire begins. The questionnaire consists of both closed-ended question and open-ended question that starts with respondent demographic information. Afterwards, the questions are directed towards data visualization features and its

usefulness. Lastly, questions on respondent opinions on data visualization tools will be asked towards the end of the questionnaire.

For closed-ended questions, multiple-choice questions are used on questions with multiple possible answer options. Since the answer choices may consist of too many options, an 'other' option is available when deemed necessary where respondent can specify any specific answer not listed as the options. The use of rankings is to order the answer in terms of frequency, proficiency, and usefulness of a subject.

Subjective answers are gathered through open-ended questions with the goal of understanding further the limitation and features of data visualization tools from respondent's perspective.

The questionnaire consists of three pages that group the questions based on the demographic, features, and personal opinions. The demographic section of the questionnaire is to gather the respondent's background through questions relating to professional status, data visualization usage and proficiency, and experience with existing data visualization tools. For questions relating to the features, each feature's usefulness from the observation process is ranked from a scale of 1 to 5 with 1 being least useful and 5 being very useful. The last section is designed to gather more insight into data visualization tools' limitation and features the respondent wish to see in data visualization tools.

3.2.3.2. Execution

The questionnaire is distributed through shareable links using Microsoft Forms. A window of 10 days is reserved for responses from the questionnaire.

3.2.3.3. Result

A total of 26 response were collected. Figure 3.1 to figure 3.4 shows the response from the demographic section, figure 3.5 to figure 3.9 shows the response from the features section, and table 3.4 and table 3.5 shows the response from the open-ended section.

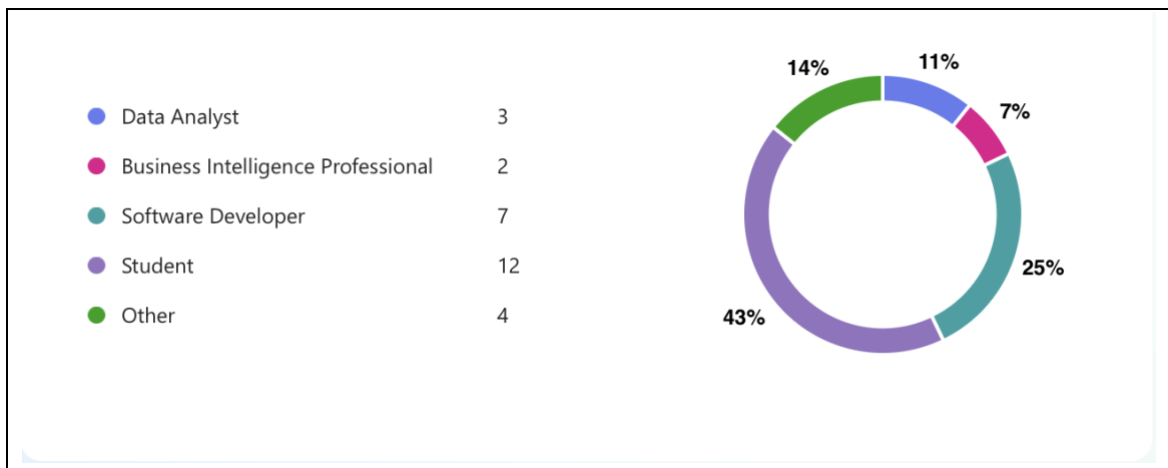


Figure 3.1: Respondents professional status

Most of the respondents are student and software developers. Few of the respondents are data analyst and business intelligence professional, while the 'Other' professional status is 'Visual Editor', 'Customer Service', and 'Managers'.

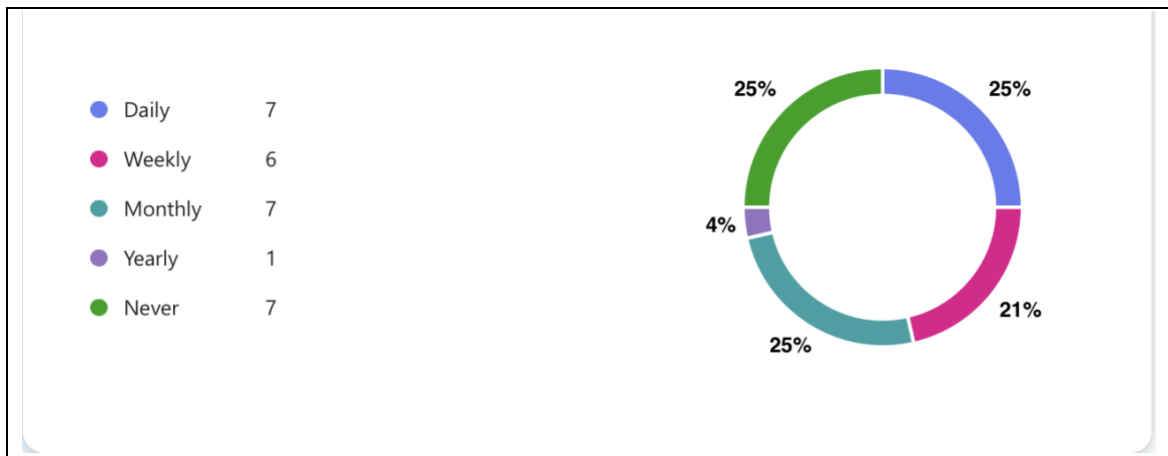


Figure 3.2: Respondents data visualization tools usage

The usage frequencies between daily, weekly, monthly and never are fairly balanced with only one respondent have the usage frequency of yearly.

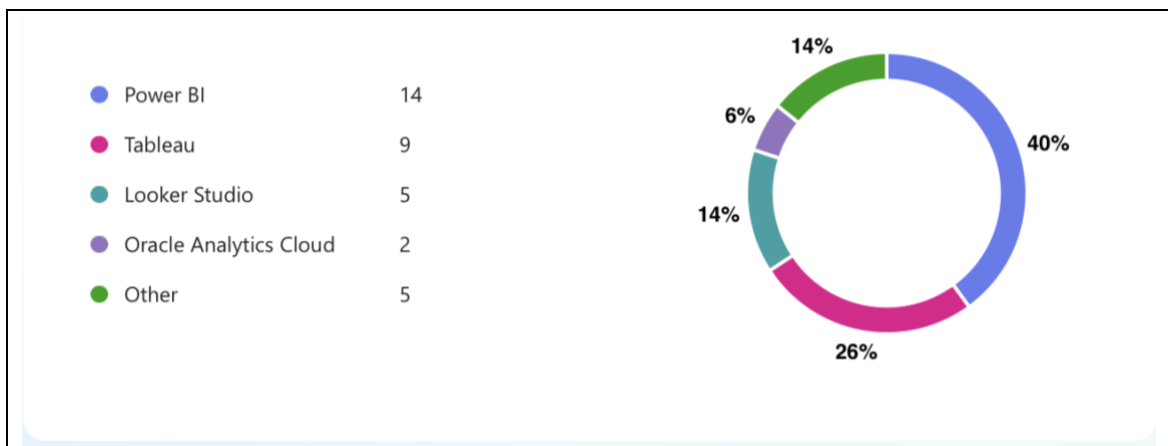


Figure 3.3: Data visualization tools used by respondents

Power BI stands out as the most popular data visualization tool amongst the respondent, followed up by tableau. Looker Studio and Oracle Analytics Cloud are in the minorities as both are rarely used amongst the respondent. The

'Other' data visualization tools are 'Jupyter Notebook', 'SPSS', 'Metabase', and 'Kibana'.

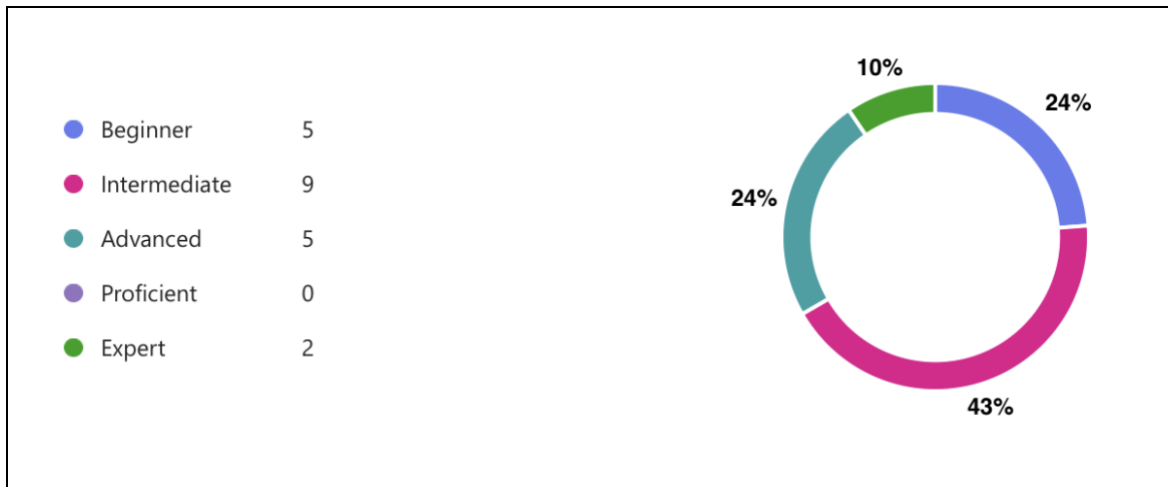


Figure 3.4: Respondents proficiency with data visualization tools

Most respondents consider themselves as intermediate while using data visualization tools. Beginners and advanced are evenly split but are the majority when combined. Two of the respondents are expert in using data visualization tools.

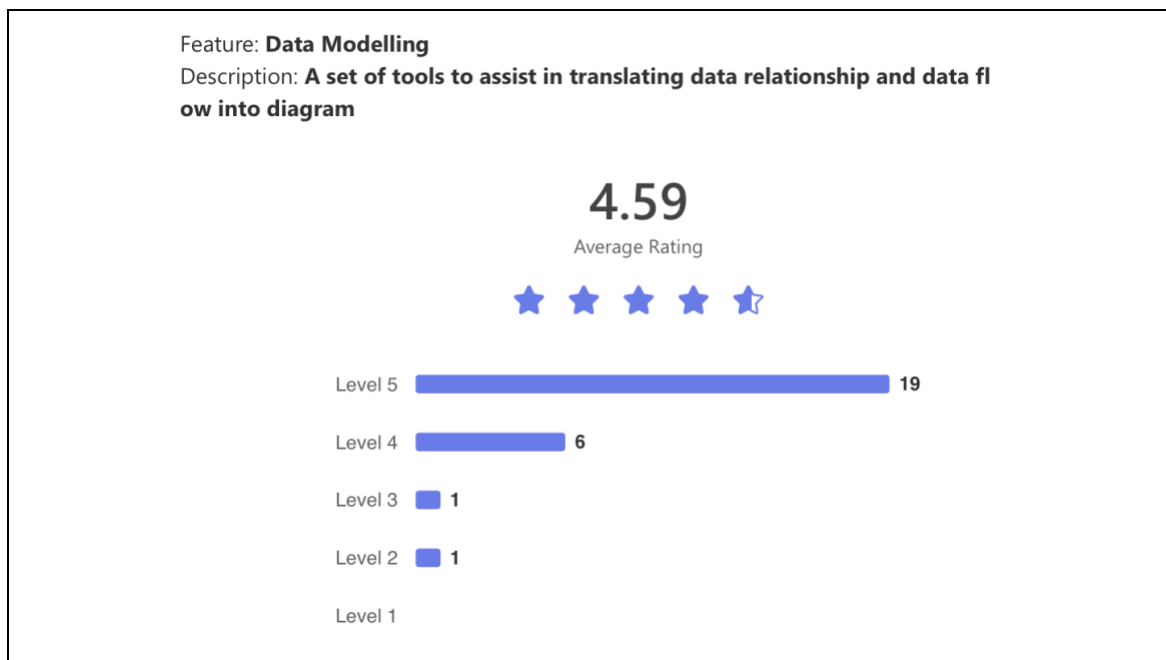


Figure 3.5: Data modelling feature rating

Data Modelling received the highest rating of 4.59 with 19 users rating it at Level 5. This indicates that users find it highly effective for translating data relationships into diagrams. 6 users rated it at Level 4, showing strong satisfaction with minor areas for improvement. A very small number of users rated it at Level 3 and Level 2 which suggests that most find this feature useful and well-implemented.

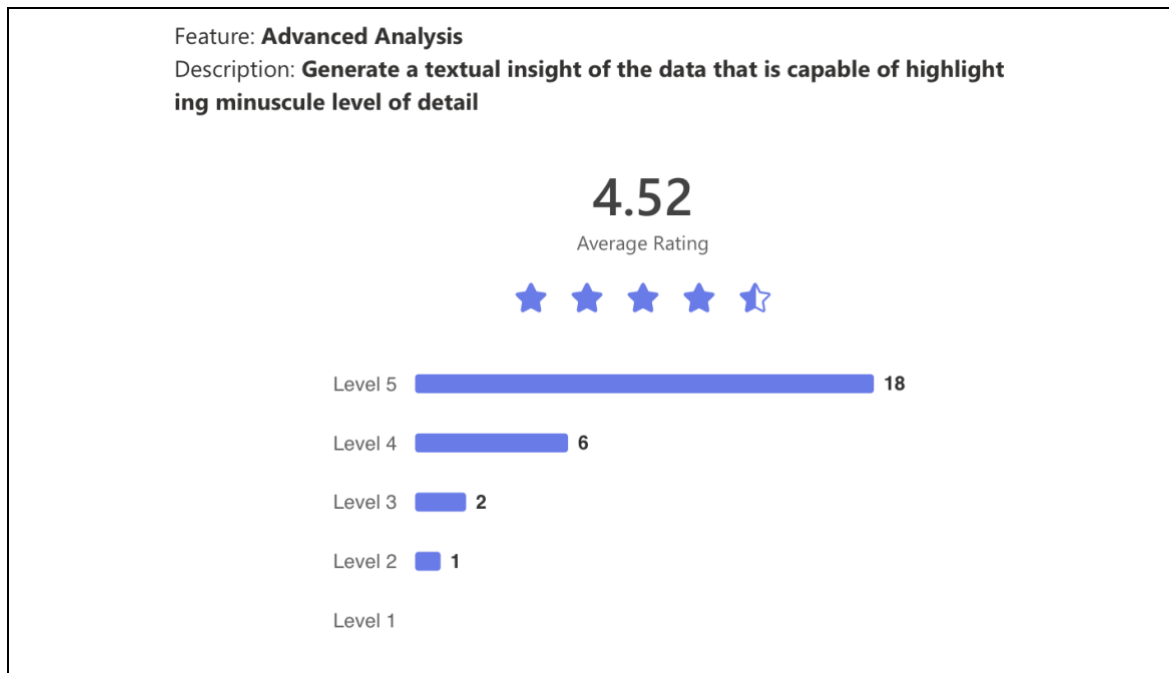


Figure 3.6: Advanced analysis feature rating

Advanced Analysis had an average rating of 4.52 with 18 users rating it at Level 5. This suggests that users appreciate its ability to generate detailed textual insights from data. 6 users rated it at Level 4, indicating a positive experience with slight room for enhancement. 3 users rated it at Level 3 or below.

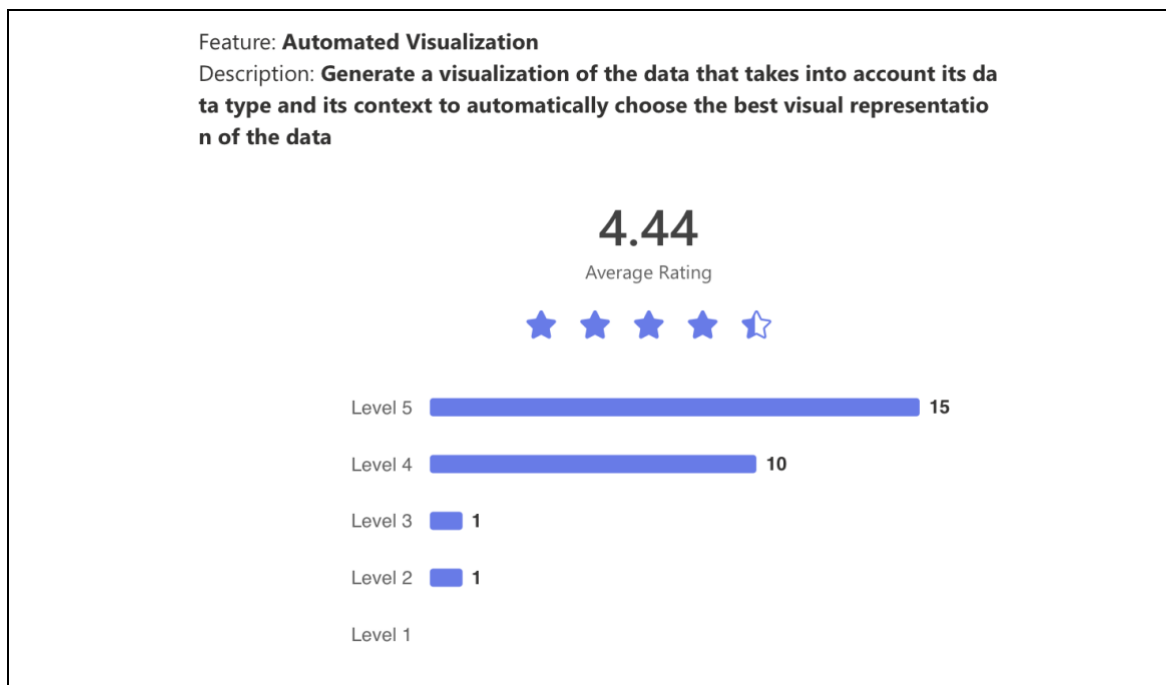


Figure 3.7: Automated visualization feature rating

Automated Visualization received an average rating of 4.44 with 15 users rating it at Level 5. Users find it valuable for selecting the best visual representation of data based on context. 10 users rated it at Level 4, reflecting a generally positive experience. Only 2 users rated it at Level 3 or Level 2 which suggests that the feature performs well for most users with minimal complaints.

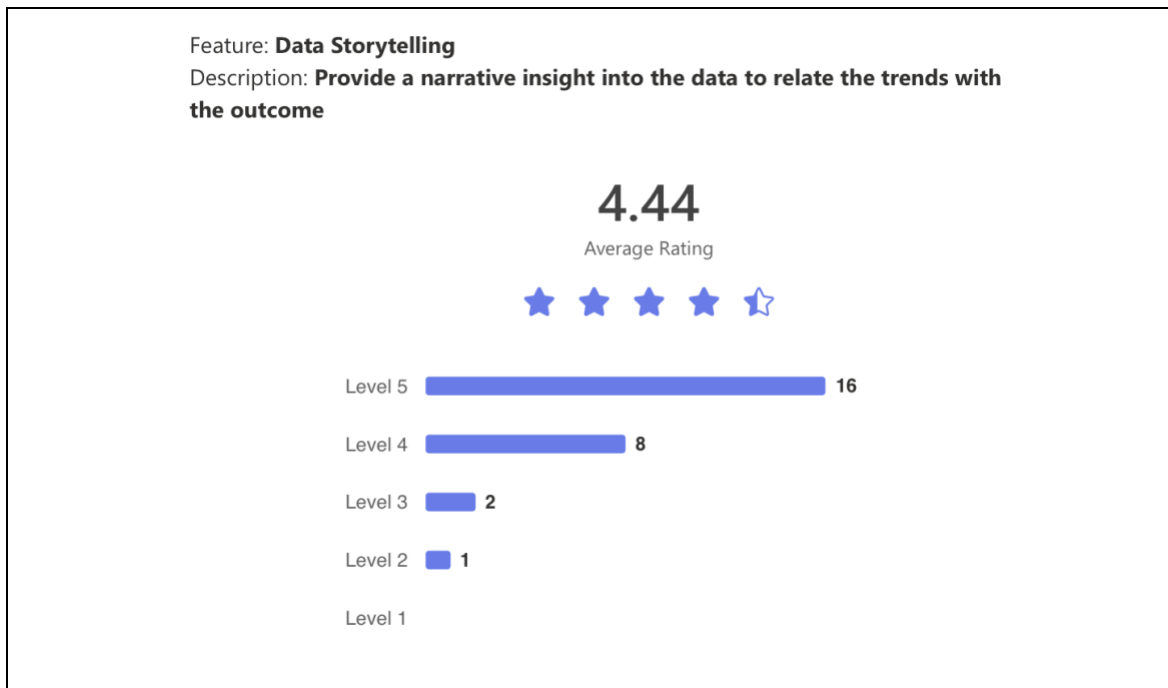


Figure 3.8: Data storytelling feature rating

Same with Automated Visualization, Data Storytelling also had an average rating of 4.44 with 16 users rating it at Level 5. This suggests that many users find it effective in providing narrative insights that relate trends to outcomes. 8 users rated it at Level 4, indicating a positive response with some minor areas for refinement. 3 users rated it at Level 3 or Level 2, showing that a small number of users may not find it as effective for storytelling purposes.

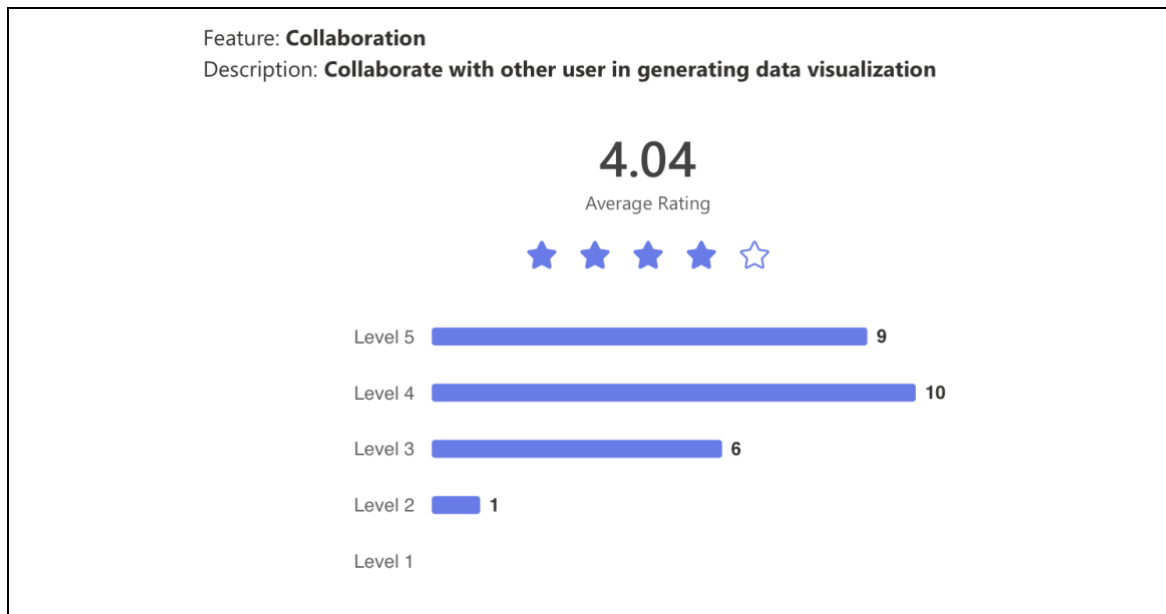


Figure 3.9: Collaboration feature rating

Lastly, Collaboration had the lowest average rating of 4.04 with 9 users rating it at Level 5. 10 users rated it at Level 4 which shows that while many users find it useful, it is not as highly rated as other features. 6 users rated it at Level 3, suggesting that a notable portion of users found it to be average, while 1 user rated it at Level 2.

Table 3.4: Features respondents wish to see in data visualization tools

No.	Responses
1	<i>so far no</i>
2	<i>Good</i>
3	<i>Spatial data analysis</i>
4	<i>Nope.</i>
5	<i>capability to import open source viz, i.e d3 to be integrate in the tool</i>
6	<i>Integration of LLM in BI tools to provide insights</i>
7	<i>No.</i>
8	<i>One feature I love to see in data visualization tools is an AI-powered Insight Generator. Imagine if the tool didn't just display charts but also analyzed your data and highlighted key patterns or trends for us and make it easier way.</i>
9	<i>Not sure</i>
10	<i>Ai helper</i>
11	<i>-</i>
12	<i>auto size the size of column in power bi</i>
13	<i>No</i>
14	<i>In power bi, i wish to see data cleansing features. Although there is other data cleansing tools, but integrating data cleansing features could improve the efficiency of work.</i>
15	<i>1. Adaptive Visualization AI Assistant: A smart, context-aware assistant integrated into visualization tools that dynamically adapts the visualizations based on the dataset, user intent, and target audience. 2. Textual narrative capability using LLMs to interpret and describe charts. This is especially good when working with data simulation when users are able to adjust variables and the the tools reactively regenerate the charts while describing what's happening</i>
16	<i>No</i>
17	<i>Automatic visualization of tabular data from SQL</i>

Table 3.5: Limitations on existing data visualization tools

No.	Based on your personal experience, what are the limitations with the existing data visualization tools? Does it affect the goal you are trying to achieve with these tools?
1	<i>no</i>
2	<i>Good</i>
3	<i>The dataset itself</i>
4	<i>High licensing fees for advanced tools can be a barrier for individuals or small organizations. Budget restrictions may limit access to better tools, forcing compromises on visualization quality.</i>
5	<i>limitation number of data points, require certain tweak to viz big data</i>
6	<i>Performance bottleneck when dealing with large dataset, since the visualization cannot utilize all data that is available.</i>
7	<i>No.</i>
8	<i>I think limited interactivity in some tools. While many allow basic filters and drill-downs, they often dont support more dynamic exploration, like asking follow-up questions or visualizing?</i>
9	<i>Yes</i>
10	<i>limited GB for unuser</i>
11	<i>-</i>
12	<i>No</i>
13	<i>Limitations in data size and loading time for big data.</i>
14	<i>The performance of the existing data visualisation tools depends on the data volume as it tends to slow down when handling massive volumes of data and it can affect the work process.</i>
15	<i>Some tools are overly complicated and requires programming skills. Need to be able to tell or select type of charts to draw and the tools should guide users accordingly</i>
16	<i>No</i>
17	<i>Too many steps</i>

3.2.3.4. Analysis

The respondent background consists mostly of students and software developers with a smaller number identifying as data analysts and business intelligence professionals. A few respondents fall under other professional

categories, including visual editors, customer service representatives, and managers.

The usage frequency of data visualization tools is fairly balanced, with most respondents using them daily, weekly, or monthly. Only one respondent reported using them yearly, and a notable number indicated they never use data visualization tools.

Power BI is the most widely used tool, followed by Tableau. These two are considered the dominant data visualization tools amongst respondents. Looker Studio and Oracle Analytics Cloud are less commonly used which indicates that they are less favored in comparison. Some respondents mentioned other tools, such as Jupyter Notebook, SPSS, Metabase, and Kibana, though Jupyter Notebook would not be considered as a data visualization tool, it is able to generate data visualization with the help of supported libraries and packages.

In terms of proficiency, most respondents consider themselves intermediate users. The number of beginners and advanced users is evenly split, but together they form a significant majority. Only two respondents identified as experts, meaning that high-level proficiency in data visualization tools is less common amongst the surveyed group.

From this group of respondents, the feature ratings indicate that respondents highly value tools that enhance data interpretation and visualization with Data Modelling receiving the highest rating of 4.59. Users find this feature effective in translating data relationships into diagrams with 19 users rating it at Level 5. The small number of lower ratings suggests that most users find it well-implemented with minimal issues.

Advanced Analysis follows closely with a rating of 4.52, showing that users appreciate the ability to generate textual insights from data. The high number of Level 5 ratings indicates that this feature is widely accepted.

Automated Visualization and Data Storytelling both received a rating of 4.44 which indicates that users see them as valuable tools for selecting visual representations and generating narrative insights. The strong number of Level 5 ratings suggests that most users benefit from these features, but some minor areas for improvement remain.

Collaboration received the lowest rating at 4.04 which suggests that while some users find it useful, it does not perform as well as other features. With more users rating it at Level 3 and Level 4, the feedback indicates that collaborative functionalities may need enhancements to be implemented inside of a data visualization tool.

Overall, the analysis shows that features focused on data modeling, analysis, and visualization are highly rated and well-received, while collaborative aspects may require refinement. Users seem to prefer tools that simplify complex data relationships and provide automated insights,

From the open-ended questions, features that respondents wish to see are spatial data analysis, integration of open-source visualization libraries, auto-sizing of columns in Power BI, and built-in data cleansing feature. In addition, some of the features relates closely to the features observed earlier are AI-powered insight generator and textual narrative generation using LLM, which is similar to the advanced analysis feature from the questionnaire and observation. In addition, the automated visualization feature is also mentioned in the open-ended question responses where respondents wish to see automatic visualization from SQL tables and adaptive visualization AI assistant. Lastly, respondents also wish to see LLM integrated in BI tools, which relates very closely to the solution proposed in this project.

From the limitations of existing data visualization tools, user struggles with high licensing costs, handling large datasets, efficient data processing, enhanced

creativity, storage limitations, and improved data loading speed. The limitations of simplified user experience and reduction steps for visualization relates closely to the project goal which could possibly be solved through this proposed data visualization tool.

From here, the rated features will be mapped against the result from the observation and the open-ended questions with the scope and the goal of the project in mind.

3.2.4. Requirement Analysis Summary

After mapping the result from the observation and analysis, it is concluded that data connection, chart & visualization, dashboard, and advanced analysis are highly prominent in defining a data visualization tool based on the observation and questionnaire closed-ended and open-ended response. They are a combination of a must in every data visualization tools and user sees it as very useful and important in order to execute data visualization tasks. In addition, some of these features are features that was mentioned by the questionnaire respondent under the 'features they wish to see' question.

The features that were discarded are deemed not important enough as it does not exist in every data visualization tool and the responses from the questionnaire are not enough to justify the importance of its presence in a data visualization tool.

3.3. Functional Requirements

Table 3.6 displays the finalized functional requirements of the system after going through the requirements elicitation process. These functions are expected

to be available in the system and are expected to solve the problems that exists within existing data visualization tools.

Table 3.6: System's functional requirements

Requirement
The user shall be able to create an account by registering their credentials into the system
The system shall store user credentials during registration
The user shall be able to login to the system using their credentials
The system shall validate user credentials when logging in
The user shall be able to enter the data source credentials into the system
The system shall connect to the data source provided by the user
The system shall retrieve the database schema from the data source provided by the user
The system shall be able to display data preview
The user shall be able to submit a prompt into the system
The LLM shall generate executable SQL query
The system shall extract data from the data source and store the data temporarily within the system
The LLM shall convert the extracted data from the data source into a structured format
The LLM shall choose and produce visualization of the data to be displayed on the interface
The LLM shall generate analysis of the data to be displayed on the interface
User shall be able to choose the visuals for the LLM to consider when choosing a visual for the data
User shall be able to save the generated visual and analysis from the LLM
User shall be able to view the saved visual and analysis
The system shall display the output from the agents

Table 3.7 to table 3.18 displays the use cases of the system and its details of actors, description, precondition, postcondition, and its scenarios.

Table 3.7: Register use case description

Requirement ID	REQ_F001	Requirement Name	Register
Actor / User	User		
Description	User enters their email, username, and password into the system to create an account and access the system		
Precondition	User has not created an account yet		
Postcondition	User credentials are stored within the system		
Main Success Scenario	1. User input their email, username, and password 2. The inputs are validated by the system 3. The inputs are stored as in the system 4. The system displays the success message		
Alternative Scenario	1. User inputs their email, username, and password 2. The inputs are validated by the system 3. The account credentials already exist in the system 4. The system displays the alert message		
Exception Scenario	1. User inputs their email, username, and password 2. The inputs are invalidated by the system 3. The system displays the error message		
Author	Luqman Hakim bin Noorazmi		

Table 3.8: Login use case description

Requirement ID	REQ_F002	Requirement Name	Login
Actor / User	User		
Description	User enters their username or email and password into the system to access the system		
Precondition	User has registered to the system		
Postcondition	User has access to the system using their profile		
Main Success Scenario	1. User inputs their email or username and password 2. The inputs are validated by the system 3. The system displays the success message 4. User are given access to the system		
Alternative Scenario	-		
Exception Scenario	1. User inputs their email or username and password 2. The inputs are invalidated by the system 3. The system displays the error message		
Author	Luqman Hakim bin Noorazmi		

Table 3.9: Connect to data source use case description

Requirement ID	REQ_F003	Requirement Name	Connect to data source
Actor / User	User		
Description	User enters database connection credentials: hostname, username, password, and database name and connect to the data source		
Precondition	The user has access to the system		
Postcondition	The database credentials are stored in the system and the system is connected to the data source, and the database schema is fetched		
Main Success Scenario	1. User inputs the database hostname, username, password, and database name. 2. The inputs are validated by the system 3. The inputs are stored as database credentials in the system. 4. The system attempts to connect to the data source 5. The connection attempt successful 6. Database schema is fetched 7. The system displays the success message.		
Alternative Scenario	-		
Exception Scenario	1. User inputs the database hostname, username, password, and database name. 2. The inputs are validated by the system 3. The inputs are stored as database credentials in the system. 4. The system attempts to connect to the data source 5. The connection attempt unsuccessful 6. The system displays the error message.		
Author	Luqman Hakim bin Noorazmi		

Table 3.10: Submit prompt use case description

Requirement ID	REQ_F004	Requirement Name	Submit Prompt
Actor / User	User		
Description	The user enters prompt into the system		
Precondition	The user has access to the system The connection to the data source is established		
Postcondition	The prompt is stored in the system		
Main Success Scenario	1. The user enters the prompt 2. The prompt is validated by the system 3. The prompt is stored in the system		
Alternative Scenario	-		
Exception Scenario	1. The user enters the prompt 2. The prompt is invalidated by the system 3. The system displays the failure message 4. The system displays the error log		
Author	Luqman Hakim bin Noorazmi		

Table 3.11: Select visuals use case description

Requirement ID	REQ_F005	Requirement Name	Select Visuals
Actor / User	User		
Description	User selects the visuals for the LLM to consider when generating visualization		
Precondition	The user has access to the system		
Postcondition	The selected tools are ready to be passed to the LLM		
Main Success Scenario	1. User selects the visual from the available visual choices 2. The selections are updated within the system 3. The system displays the success message.		
Alternative Scenario	-		
Exception Scenario	-		
Author	Luqman Hakim bin Noorazmi		

Table 3.12: Generate SQL query use case description

Requirement ID	REQ_F006	Requirement Name	Generate SQL Query
Actor / User	LLM		
Description	The LLM translates the prompt by the user into an executable SQL query		
Precondition	The user has access to the system The connection to the database is established The database schema is fetched		
Postcondition	The prompt by the user is translated into an executable SQL query		
Main Success Scenario	1. The system gets the stored database schema and prompt 2. The LLM generates the SQL with consideration to the database schema and prompt 3. The SQL query is validated 4. The SQL query is stored in the system 5. The system displays the success message		
Alternative Scenario	1. The system gets the stored database schema and prompt 2. The LLM generates the SQL with consideration to the database schema and prompt 3. The SQL query is invalidated The SQL query is generated again with consideration to the previous SQL query		
Exception Scenario	1. The system gets the stored database schema and prompt 2. The LLM generates the SQL with consideration to the database schema and prompt 3. The SQL query is invalidated 4. The SQL query is generated again with consideration to the previous SQL query 5. The LLM reaches the maximum attempt for SQL query regeneration 6. The system displays the failure message		
Author	Luqman Hakim bin Noorazmi		

Table 3.13: Generate data dictionary use case description

Requirement ID	REQ_F007	Requirement Name	Generate Data Dictionary
Actor / User	LLM		
Description	LLM converts the queried data from the data source into a structured format		
Precondition	The data has been queried from the data source		
Postcondition	The data are transformed into a dictionary		
Main Success Scenario	1. The system gets the stored data from data source 2. LLM generate dictionary based on the queried data 3. The dictionary is validated by the system 4. The dictionary is stored in the system 5. The system displays the success message		
Alternative Scenario	-		
Exception Scenario	4. The system gets the stored data from data source 5. LLM generate dictionary based on the queried data 6. The dictionary is invalidated by the system 7. The system displays the failure message		
Author	Luqman Hakim bin Noorazmi		

Table 3.14: Generate visual use case description

Requirement ID	REQ_F008	Requirement Name	Generate Visual
Actor / User	LLM		
Description	LLM generates the best suited visual for the data		
Precondition	The prompt has been entered The data dictionary has been generated		
Postcondition	The visual representation of the data is generated		
Main Success Scenario	1. The system gets the data dictionary and prompt 2. The LLM evaluates the visual selection 3. The LLM chooses the visual 4. The visual is stored in the system 5. The visual is displayed		
Alternative Scenario	-		
Exception Scenario	1. The system gets the data dictionary and prompt 2. The LLM evaluates the visual selection 3. The LLM is unable to chooses the visual 4. The system displays the failure message		
Author	Luqman Hakim bin Noorazmi		

Table 3.15: Generate analysis use case description

Requirement ID	REQ_F009	Requirement Name	Generate Analysis
Actor / User	LLM		
Description	The LLM generates a textual analysis of the data		
Precondition	The prompt has been entered The data has been queried from the data source		
Postcondition	The analysis of the data is generated		
Main Success Scenario	1. The system gets the retrieved data and prompt 2. The LLM generates the analysis of the data 3. The system displays the analysis of the data		
Alternative Scenario	-		
Exception Scenario	1. The system gets the retrieved data and prompt 2. The LLM is unable to generate the analysis of the data 3. The system displays the failure message		
Author	Luqman Hakim bin Noorazmi		

Table 3.16: Save visual and analysis use case description

Requirement ID	REQ_F010	Requirement Name	Save Visual and Analysis
Actor / User	User		
Description	User saves the generated visual and analysis		
Precondition	The visual and analysis are displayed on the user interface		
Postcondition	The visual and analysis instance is stored within the system based on the user ID		
Main Success Scenario	1. The system displays the visual and analysis 2. User choose the save visual and analysis option 3. Visual and analysis are stored within the system 4. The system displays the success message		
Alternative Scenario	-		
Exception Scenario	1. The system displays the visual and analysis 2. User choose the save visual and analysis option 3. Visual and analysis fail to be stored within the system 4. The system displays the failure message		
Author	Luqman Hakim bin Noorazmi		

Table 3.17: View saved visual and analysis use case description

Requirement ID	REQ_F011	Requirement Name	View Saved Visual and Analysis
Actor / User	User		
Description	User saves the saved visual and analysis		
Precondition	The user saved the generated visual and analysis		
Postcondition	The saved visual and analysis are displayed		
Main Success Scenario	1. User navigates to the saved visual and analysis list 2. User selects from the saved visual and analysis 3. System displays the chosen saved visual and analysis		
Alternative Scenario	-		
Exception Scenario	1. User did not save the generated visual and analysis 2. The visual and analysis is not displayed within the saved visual and analysis page		
Author	Luqman Hakim bin Noorazmi		

Table 3.18: View agents output use case description

Requirement ID	REQ_F012	Requirement Name	View Agents Output
Actor / User	User		
Description	User views the output from each agent		
Precondition	The visual and analysis are generated successfully		
Postcondition	The output from each agent is displayed		
Main Success Scenario	4. The visual is displayed 5. The analysis is displayed 6. User clicks the view agent output button 7. System displays the agents output		
Alternative Scenario	-		
Exception Scenario	3. The visual and analysis generation failed 4. The view agent output button is not displayed		
Author	Luqman Hakim bin Noorazmi		

3.4. Quality Requirements

For every process that involves LLM, the quality of the response is important. The quality requirements are considered based on the current resources' quality. Table 3.19 to table 3.26 describes the quality requirements of the system through several different metrics of performance, portability, and usability (Krüger, 2024).

To evaluate the performance metric, the LLM's response time can be used to measure the model's response time based on the unit of text input, or token, to the model. For rough estimation purpose, the expected input for each of LLM's use case are defined.

Based on the Llama 3.3 70B Versatile model's performance on Groq hardware we can estimate execution times for key operations. Generating an SQL query takes approximately 0.36 seconds, as it is expected to process around

100 tokens at 276 tokens per second. Creating a data dictionary requires about 150 tokens, leading to an estimated execution time of 0.54 seconds. Selecting the best visualization involves approximately 200 tokens, making the process take 0.72 seconds. Finally, generating an in-depth analysis using also processes around 150 tokens, resulting in an execution time of 0.54 seconds. These estimates are based on the reported inference speed of 276 tokens per second on Groq hardware (Groq, Inc., n. d.), though actual times may vary depending on input complexity and system performance.

Table 3.19: Visualization per minute quality description

Requirement ID	REQ_Q001	Requirement Metric	Performance
Requirement Sub-Metric	Visualization per minute		
Description	Amount of visualization request per minute.		
Requirement	Based on the open-source model to be integrated, the Rate Per Minute (RPM) is 30 requests per minute (Groq, Inc., n. d.).		
Author	Luqman Hakim bin Noorazmi		

Table 3.20: SQL Generation response time quality description

Requirement ID	REQ_Q002	Requirement Metric	Performance
Requirement Sub-Metric	SQL Generation Response Time		
Description	Amount of time taken to generate SQL query		
Requirement	Approximately 0.36 seconds for around 100 tokens		
Author	Luqman Hakim bin Noorazmi		

Table 3.21: Amount of time taken to generate data dictionary quality description

Requirement ID	REQ_Q003	Requirement Metric	Performance
Requirement Sub-Metric	Generate Data Dictionary Response Time		
Description	Amount of time taken to generate data dictionary		
Requirement	Approximately 0.54 seconds for around 150 tokens		
Author	Luqman Hakim bin Noorazmi		

Table 3.22: Amount of time taken to generate visual quality description

Requirement ID	REQ_Q004	Requirement Metric	Performance
Requirement Sub-Metric	Generate Visual Response Time		
Description	Amount of time taken to generate visual		
Requirement	Approximately 0.72 Seconds for around 200 tokens		
Author	Luqman Hakim bin Noorazmi		

Table 3.23: Amount of time taken to generate analysis quality description

Requirement ID	REQ_Q005	Requirement Metric	Performance
Requirement Sub-Metric	Generate Analysis Response Time		
Description	Amount of time taken to generate analysis		
Requirement	Approximately 0.54 seconds for 150 tokens		
Author	Luqman Hakim bin Noorazmi		

Table 3.24: Time for user to familiarize with the system quality description

Requirement ID	REQ_Q006	Requirement Metric	Usability
Requirement Sub-Metric	Training Time		
Description	Amount of time for user to familiarize with the system on the first interaction.		
Requirement	Without considering the registration and login processes, time between setting up the data source and generating visualization is approximately 10 minutes. The simple interface and familiarity with existing AI-driven system that accepts prompt could heavily influence this metric.		
Author	Luqman Hakim bin Noorazmi		

Table 3.25: Percentage portable code quality description

Requirement ID	REQ_Q007	Requirement Metric	Portability
Requirement Sub-Metric	Percentage of non-portable code		
Description	Percentage of the code that cannot be converted into different platform (web app, mobile phone, etc.).		
Requirement	75% of the code should be non-portable due to the implementation of Python-specific library for the interface, PyQt5.		
Author	Luqman Hakim bin Noorazmi		

Table 3.26: Number of operating systems supported quality description

Requirement ID	REQ_Q008	Requirement Metric	Performance
Requirement Sub-Metric	Number of systems where software can run.		
Description	Number of operating systems the proposed system can be functional.		
Requirement	Three (3) operating systems will be capable of running the system. The operating systems are Windows, Linux, and Mac.		
Author	Luqman Hakim bin Noorazmi		

3.5. System Context Diagram

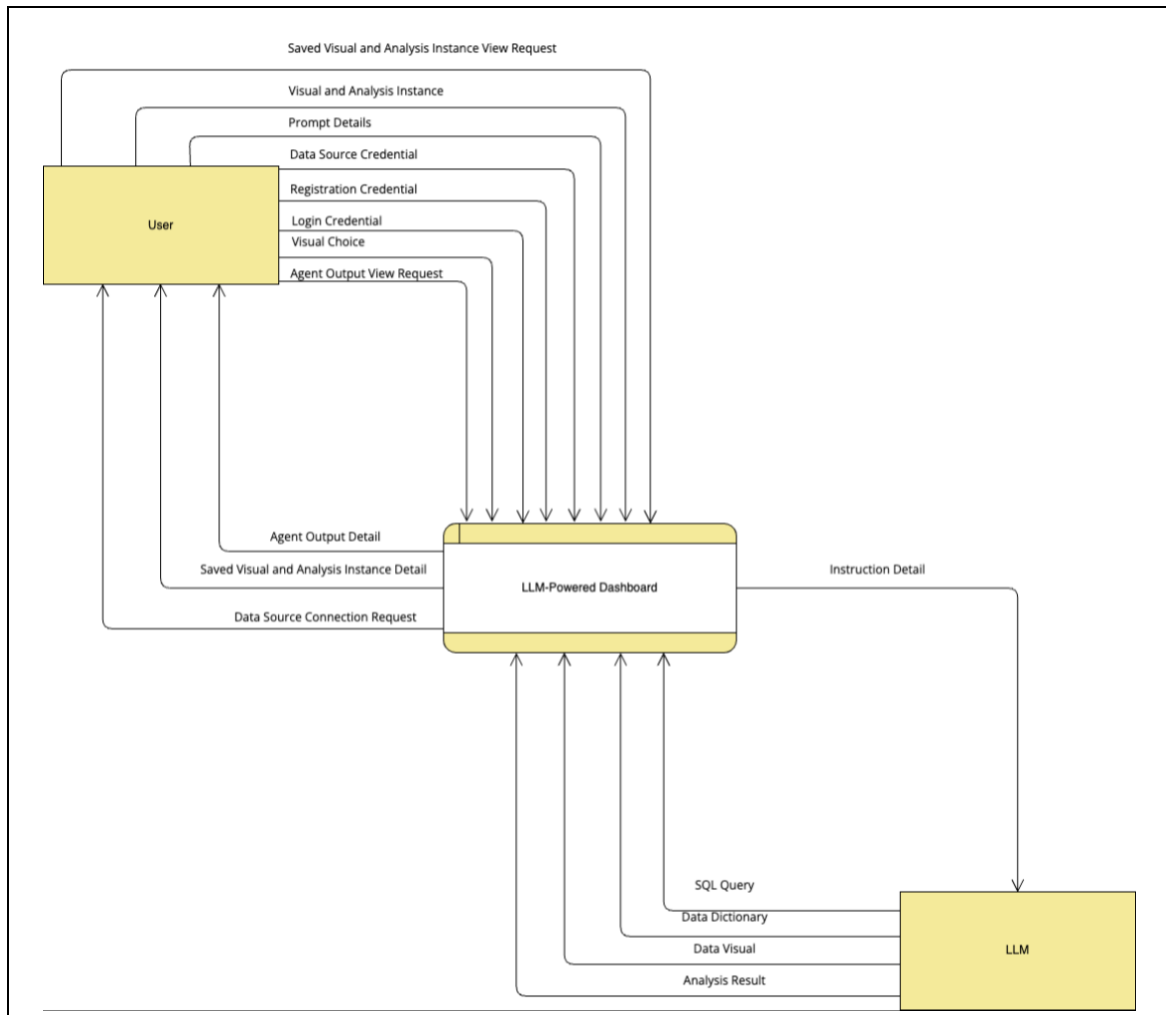


Figure 3.10: System's context diagram (Anisya et al., 2021)

Figure 3.10 depicts the context diagram of the proposed data visualization tool. A context diagram provides a high-level overview of the data visualization tool by defining its boundaries and interactions with external entities through data that flow between them. It serves as a tool for global system design that illustrate

the data visualization tool and its components in a broad and generalized manner (Anisya et al., 2021).

The external entities are user which also acts as the data source provider, and the LLM. User interacts with the system through the visual choice, visual and analysis instance, prompt detail, data source credential, registration credential, login credential, saved visual and analysis instance view request, and agent output view request. The system's interaction with the user is through the data source connection request, saved visual and analysis instance, and agent output detail, while the system's interaction with the LLM is through the instruction detail. The LLM is capable of interacting with the system through the generated SQL query, data dictionary, data visual, and data analysis.

3.6. Data Analysis Plan

Data storage is required for the system to store user's credentials and preferences. A lightweight and easy-to-use database is sufficient to store user's credentials and setting preferences locally. After some consideration, it was found that for Windows, macOS, and Linux, SQLite is a straightforward and portable database that may be used to store user data locally. Python is one of the numerous programming languages that it supports by default. SQLite also makes data management and use simple by storing it in a single file.

The first step to designing the data is to paint the data structure visually. A visual model used to depict corporate entities, their characteristics, and their relationships is called an entity-relationship diagram (ERD). Database design begins with the creation of an ER model. After it is finished, tables with primary and foreign keys are produced by using translation rules to transform it into a logical data model. In order to cut down on redundancy and boost efficiency, the

model is then examined for normalization (Rashkovits & Lavy, 2021). Figure 3.11 displays the ERD of the system.

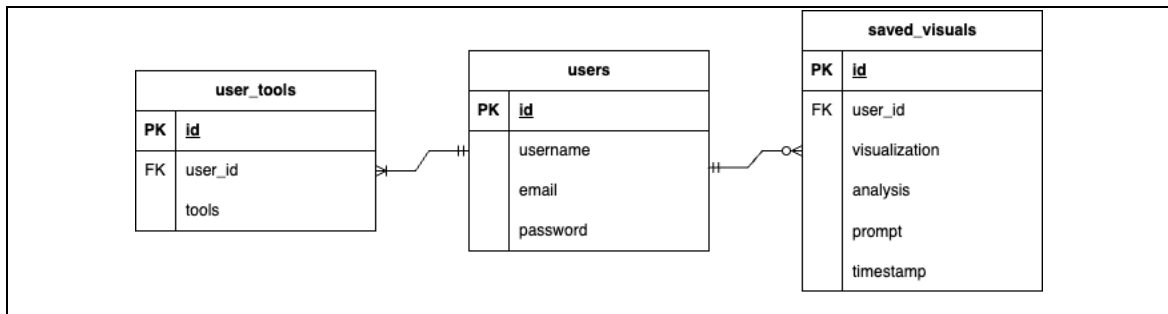


Figure 3.11: System's entity relationship diagram (Rashkovits & Lavy, 2021)

User's preferences such as their visual choices and saved visual and analysis will be stored through the *user_tools* and *saved_visual* table.

User has the option to save any visual and analysis. In addition, each user must have their own visuals choices and data source credentials. Table 3.27 depicts the data dictionary of the system based on the presented ERD.

Table 3.27: System's data dictionary

Field Name	Data Type	Data Format	Field Size	Description
id	TEXT	XNN-NNX-NXX	11	Unique identifier for each user, user_tools, and saved_visuals instance
username	TEXT	XNNNNNNNN	8	User's chosen username
email	TEXT	XXX@XXX.com	50	User's email address
password	TEXT	XNNNNNNNNNNN	11	Encrypted password for authentication
tools	JSON	{X:X}	1000	User's selected visualization option JSON

prompt	TEXT	XNNNNNNNNNNNNNNNNNN	17	User's input query for visualization
visualization	TEXT	XNNNNNNNNNNNNNNNNNN	100	Generated visual representation URL
analysis	TEXT	XNNNNNNNNNNNNNNNNNN	100	Interpretation of the visualized data URL
timestamp	TEXT	YYYY-MM-DD HH:MM:SS	19	Timestamp when the schema was stored

Chapter 4: System Design

4.1. Use Case Diagram

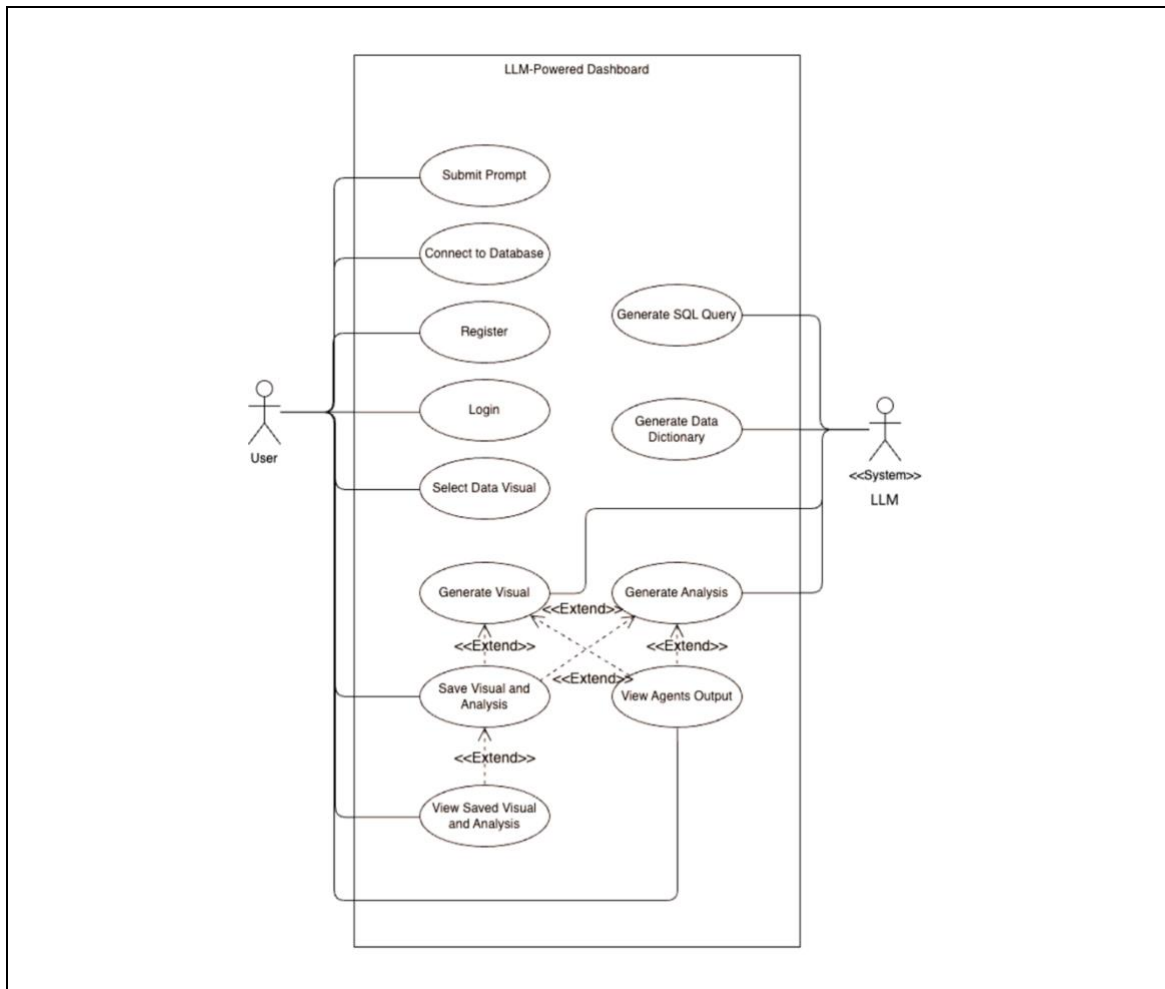


Figure 4.1: System's use case diagram (Sudiarta et al., 2021)

Figure 4.1 displays the use cases of the systems. A use case represents the interaction between a user (actor) and a system. Use case diagrams illustrate a series of interconnected interactions between actors and the system, detailing

the types of interactions a user can perform within a program. These diagrams help visualize the functionalities and features available in the system by mapping out how users engage with it (Sudiarta et al., 2021).

The user will be capable of login and register to the system, save the visual and analysis, enter data source credentials and the prompt for visualizing data into the system, view the agent output, save the generated visual and analysis and view the saved visual and analysis. The database credentials will be used by the system to make connection to the data source. Using the database schema from the data source, the LLM will generate SQL query to extract the data from the data source using the generated query and converts the data into a data dictionary. Lastly, the LLM will generate visual and analysis of the data.

4.2. Sequence Diagram

Micskei and Waeselynck argue that Sequence Diagrams (SDs) serve multiple purposes, such as illustrating the flow of method calls within a program or providing a partial specification of interactions in distributed systems (Al-Fedaghi, 2021). Hamza and Hammad state that SDs can also be used for automating test case generation, making them valuable in software validation processes (Al-Fedaghi, 2021). Lu and Kim explain that SDs depict how events or activities in a use case correspond to operations in class diagrams. They describe events as fundamental behavioral constructs that can be combined into larger behavioral fragments to represent system interactions (Al-Fedaghi, 2021).

For this data visualization tool, the sequence follows the Model View Controller (MVC) architecture. The output from the LLM is encapsulated within the functions from the controllers. The responses are stored within the models

and the response are displayed to the user through the interface where it is the only component user will be interacting with the system.

4.2.1. Submit Prompt

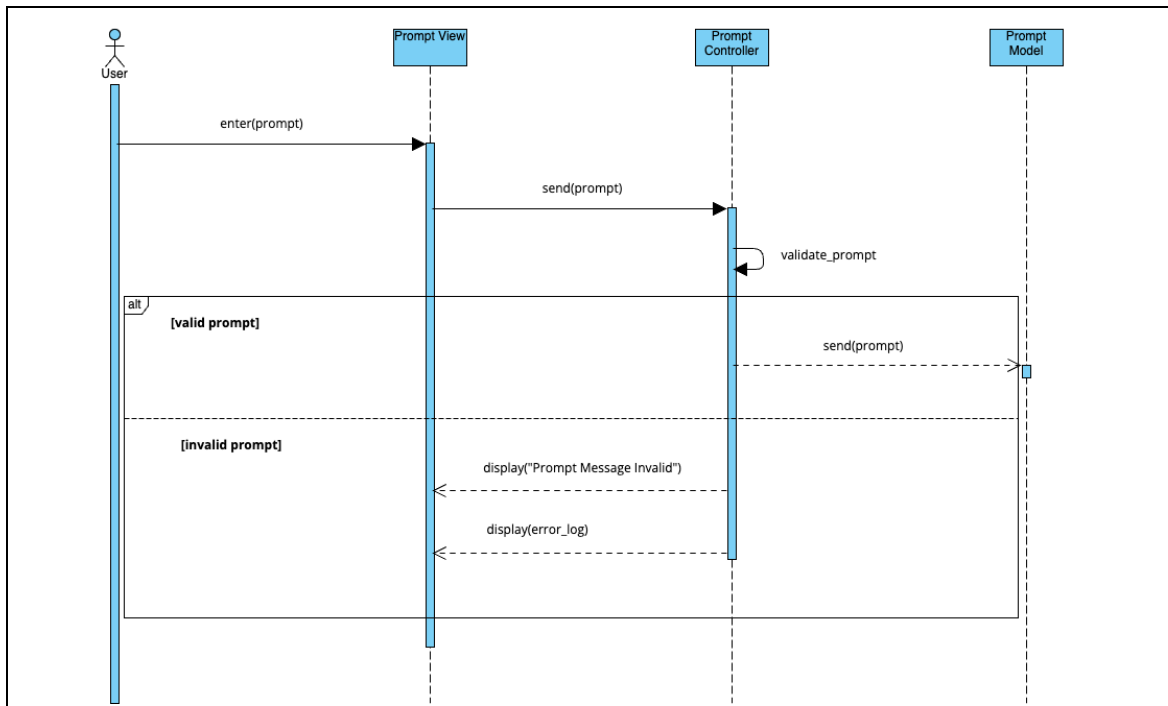


Figure 4.2: Submit prompt sequence diagram (Al-Fedaghi, 2021)

4.2.2. Connect to Data Source

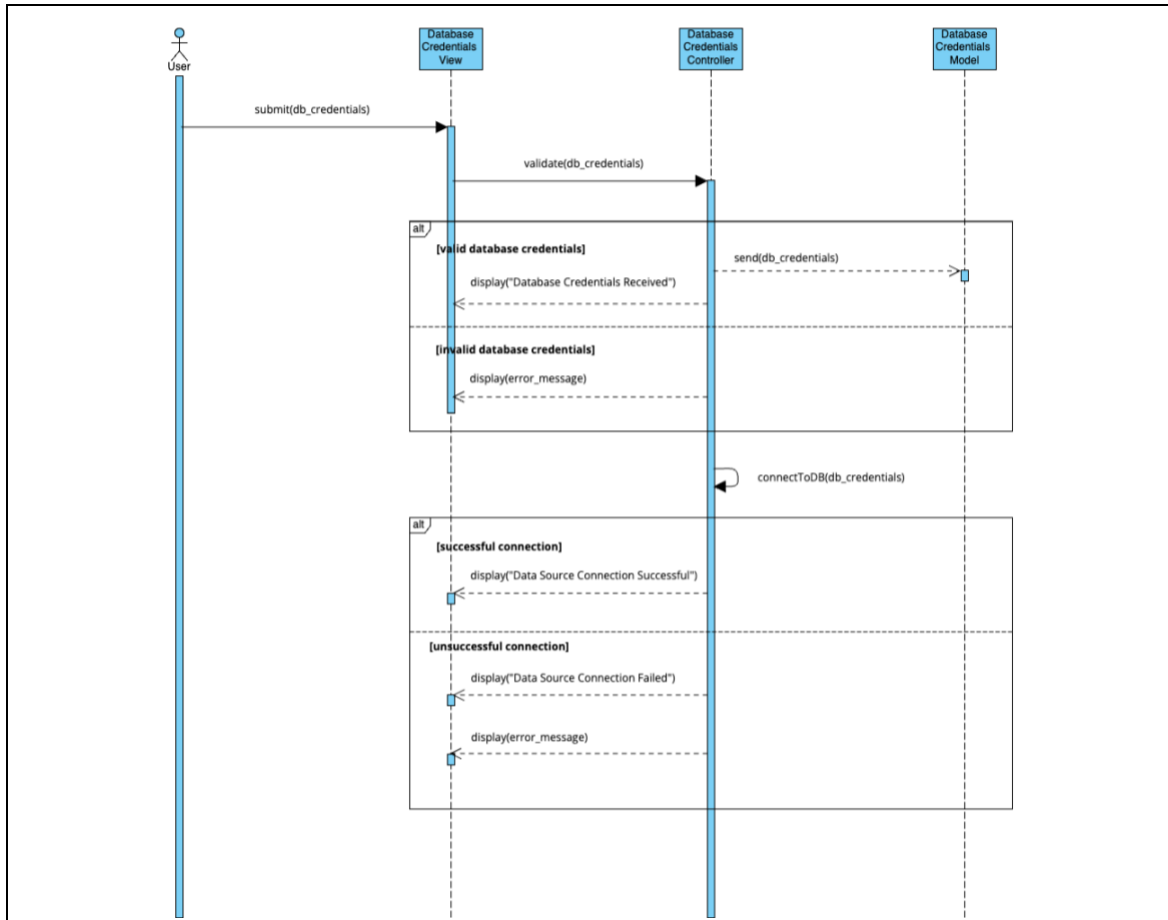


Figure 4.3: Connect to data source sequence diagram (Al-Fedaghi, 2021)

4.2.3. Select Visuals

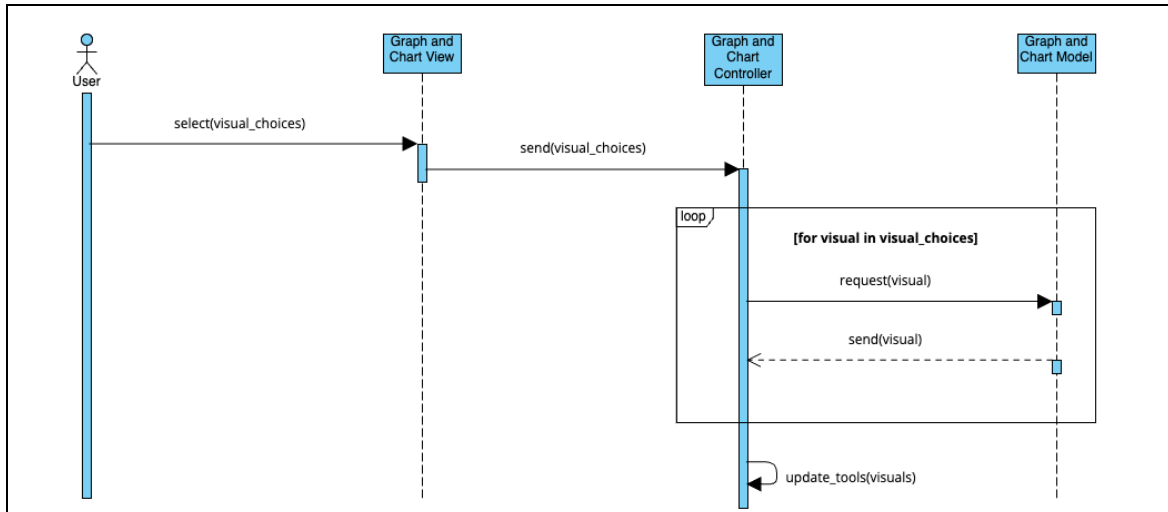


Figure 4.4: Select visuals sequence diagram (Al-Fedaghi, 2021).

4.2.4. Save Visual and Analysis

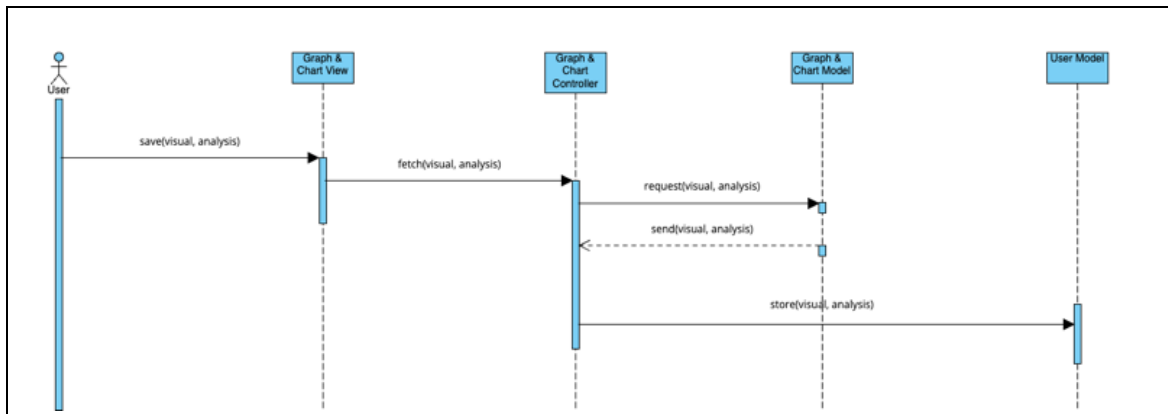


Figure 4.5: Save visual and analysis (Al-Fedaghi, 2021).

4.2.5. Generate SQL

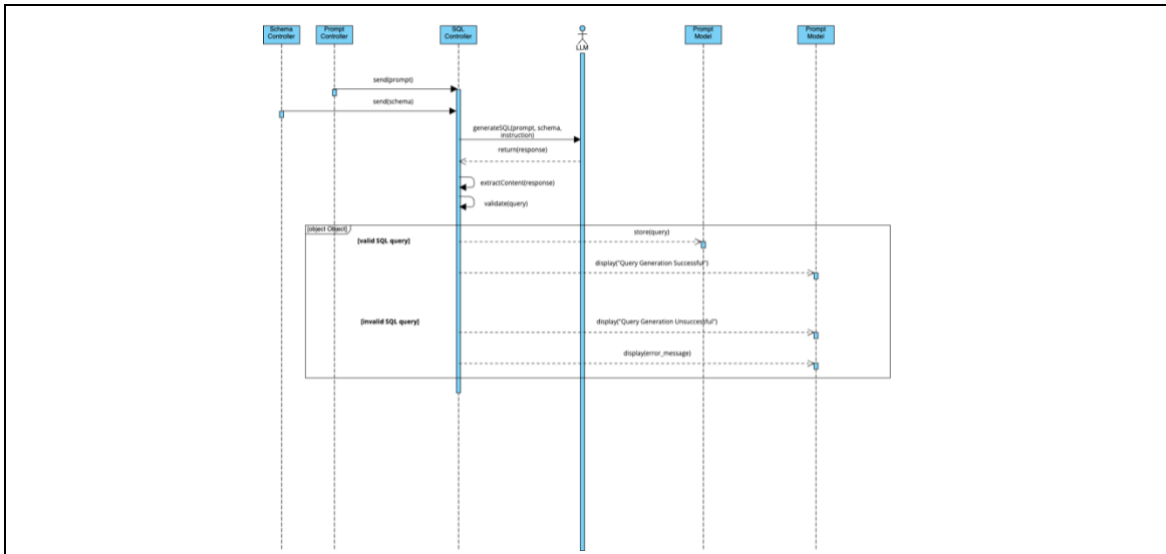


Figure 4.6: Generate SQL sequence diagram (Al-Fedaghi, 2021)

4.2.6. Generate Data Dictionary

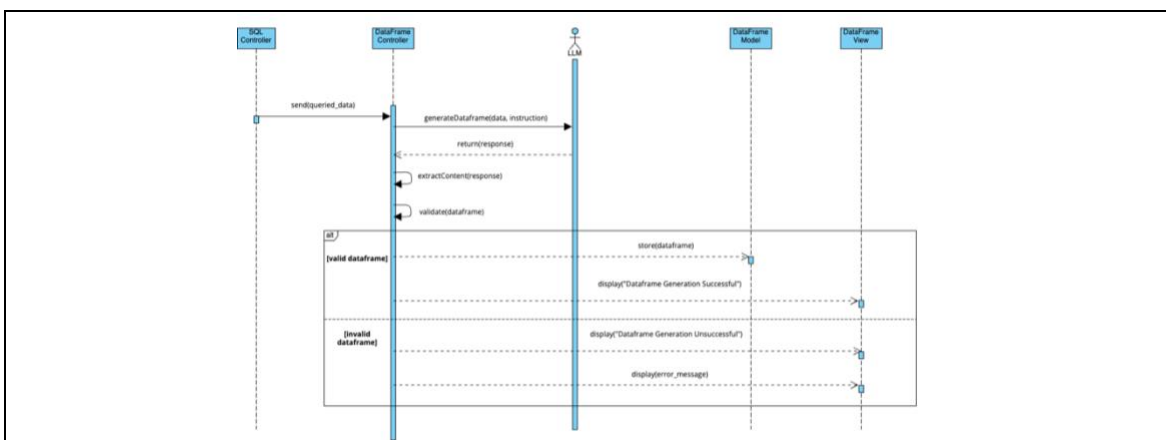


Figure 4.7: Generate data dictionary sequence diagram (Al-Fedaghi, 2021)

4.2.7. Generate Visual

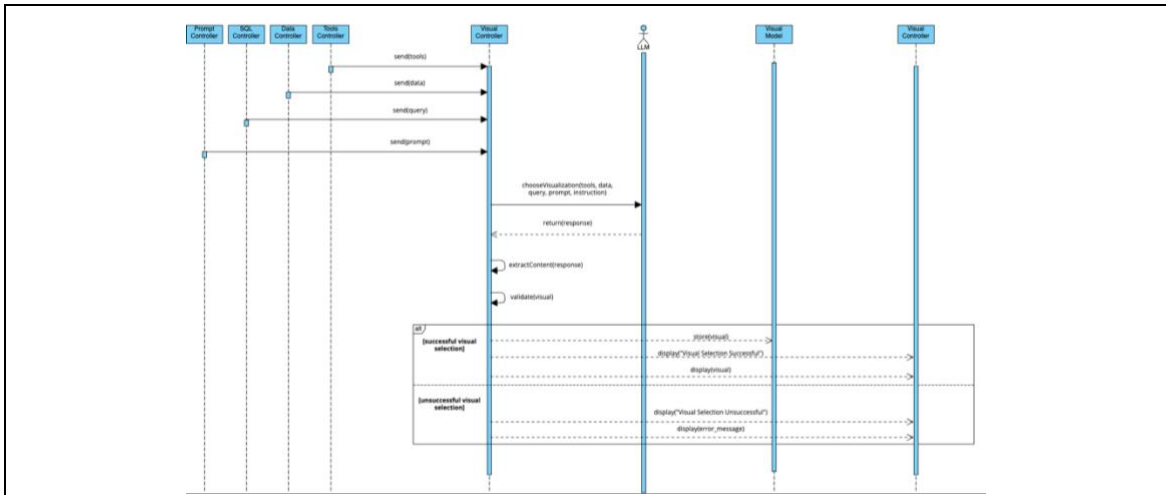


Figure 4.8: Generate visual sequence diagram (Al-Fedaghi, 2021).

4.2.8. Generate Analysis

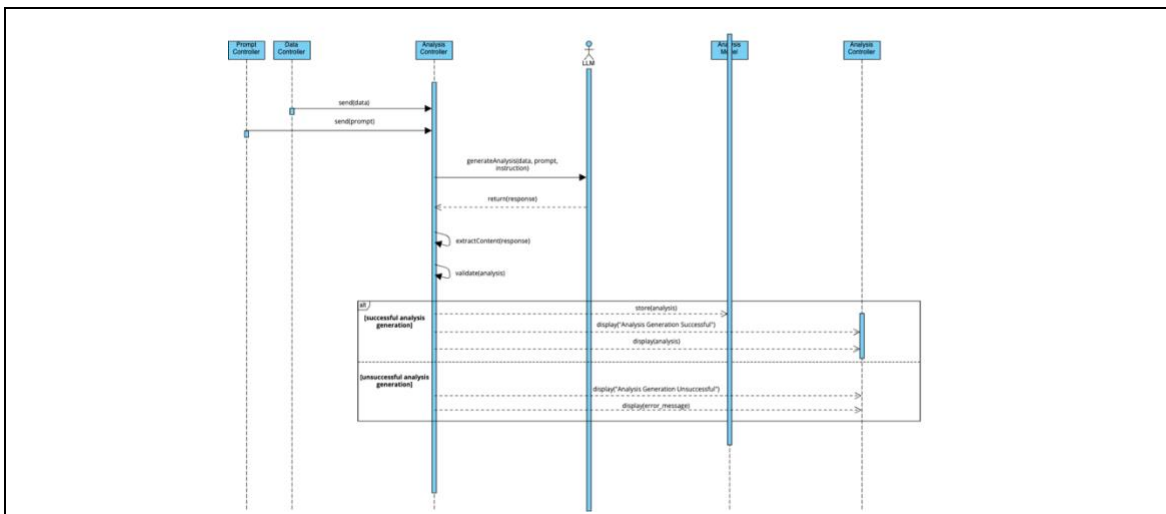


Figure 4.9: Generate analysis sequence diagram (Al-Fedaghi, 2021)

4.2.9. View Saved Visual and Analysis

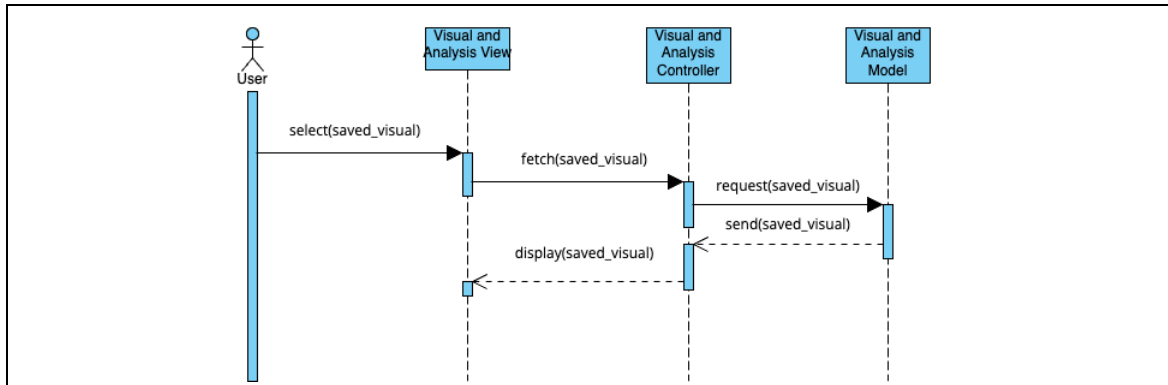


Figure 4.10: View saved visual and analysis sequence diagram (Al-Fedaghi, 2021)

4.2.10. View Agents Output

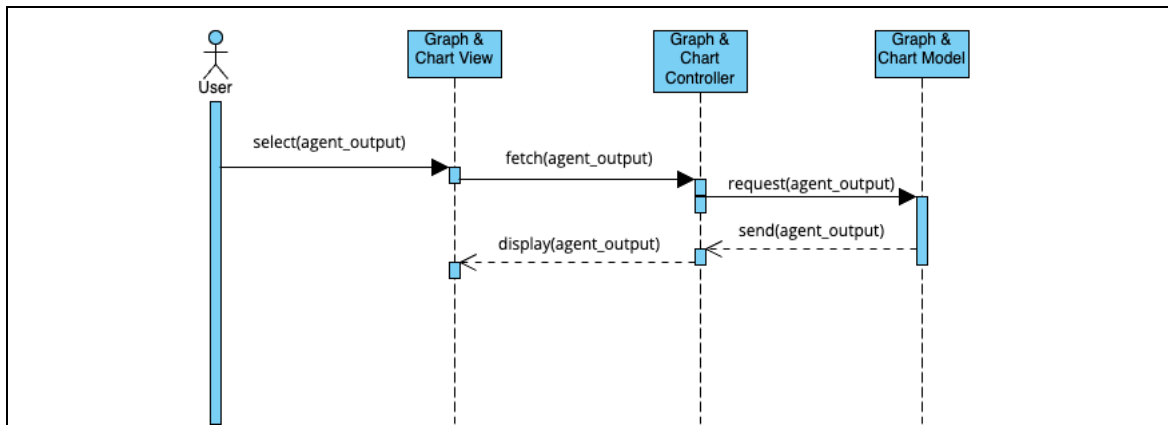


Figure 4.11: View agents output sequence diagram (Al-Fedaghi, 2021)

4.3. System Interface

Figure 4.10 to figure 4.15 shows the initial system's interface concept. The background color is a deep, muted blue-gray. It creates a calm and professional look. This color contrasts with the white text to improve readability. The design stays modern and clean.

The color palette follows a minimalist style. White buttons give a neutral and polished look. Black button text ensures clear legibility. The hover effect adds a slightly richer shade for interactive elements. The background provides a stable foundation to enhance usability and focus. The dashboard will appear structured, user-friendly, and visually appealing.

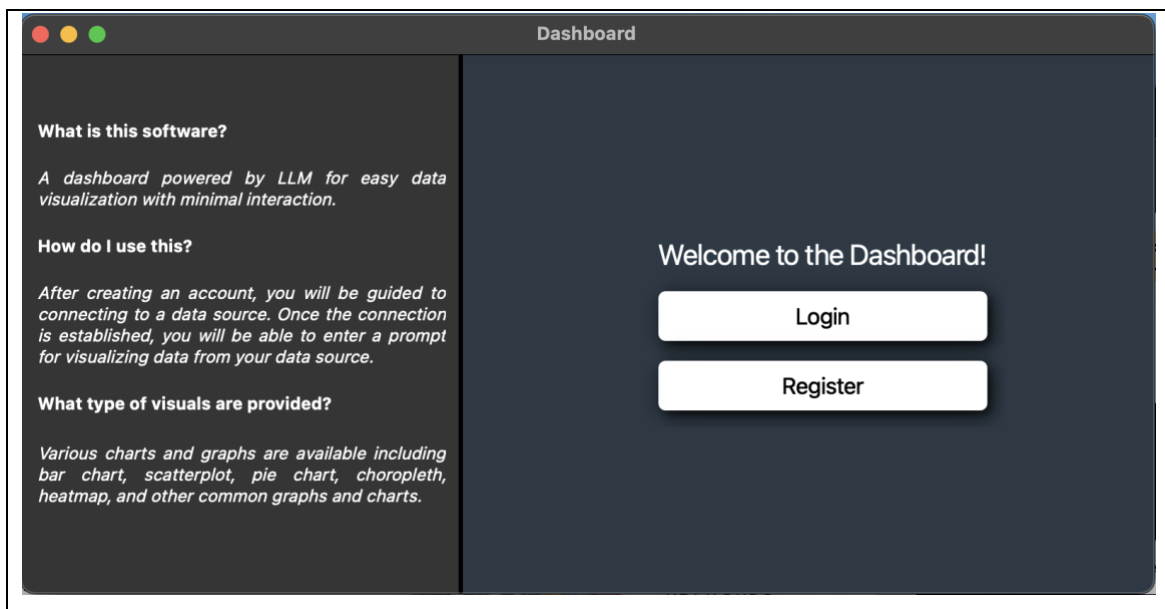
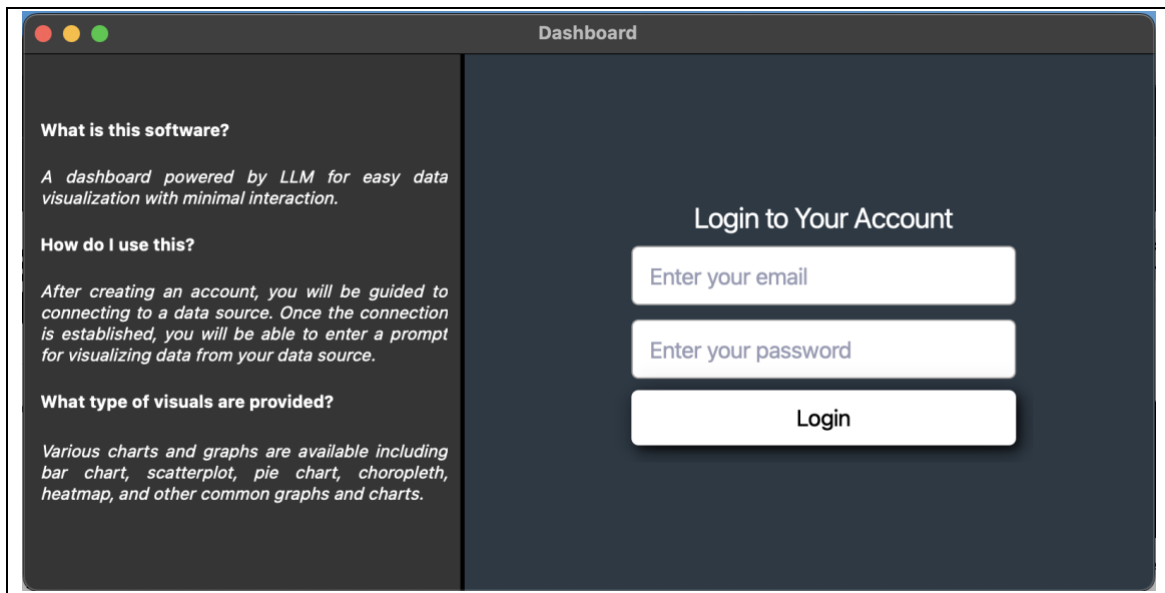
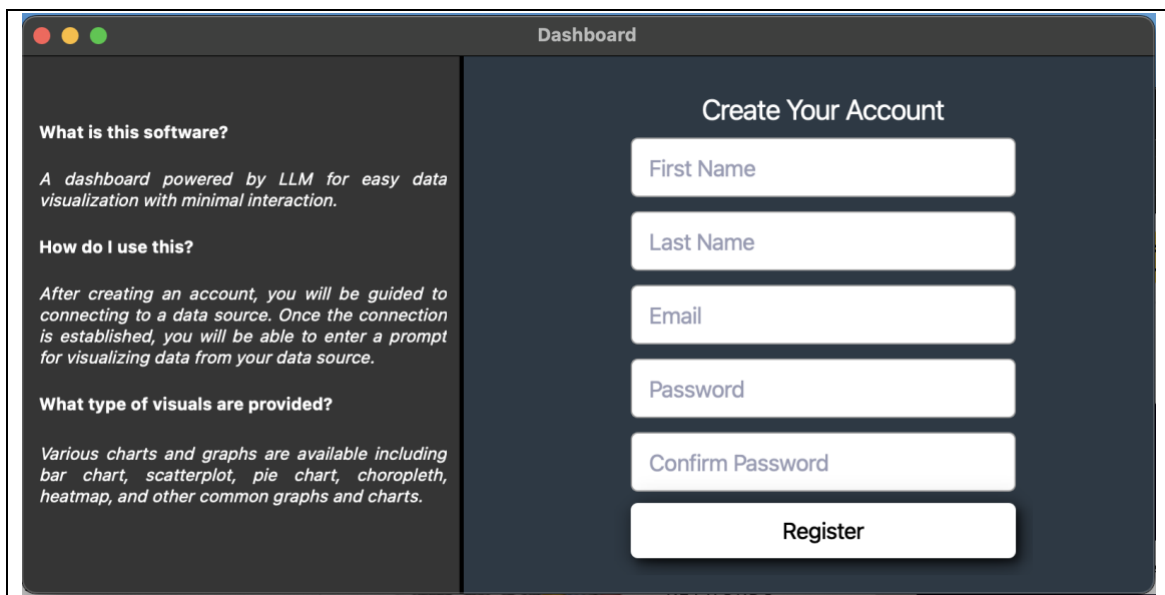


Figure 4.12: Welcome screen



The screenshot shows a web application window titled "Dashboard". The window is divided into two main sections. On the left, there is a dark sidebar with white text. It contains three sections: "What is this software?" with a description "A dashboard powered by LLM for easy data visualization with minimal interaction.", "How do I use this?" with instructions "After creating an account, you will be guided to connecting to a data source. Once the connection is established, you will be able to enter a prompt for visualizing data from your data source.", and "What type of visuals are provided?" with a list of chart types "Various charts and graphs are available including bar chart, scatterplot, pie chart, choropleth, heatmap, and other common graphs and charts." On the right, the main content area has a dark background. It features a "Login to Your Account" heading, followed by three input fields: "Enter your email", "Enter your password", and a "Login" button.

Figure 4.13: Login screen



The screenshot shows a web application window titled "Dashboard". The window is divided into two main sections. On the left, there is a dark sidebar with white text, identical to the one in Figure 4.13. It contains three sections: "What is this software?" with a description "A dashboard powered by LLM for easy data visualization with minimal interaction.", "How do I use this?" with instructions "After creating an account, you will be guided to connecting to a data source. Once the connection is established, you will be able to enter a prompt for visualizing data from your data source.", and "What type of visuals are provided?" with a list of chart types "Various charts and graphs are available including bar chart, scatterplot, pie chart, choropleth, heatmap, and other common graphs and charts." On the right, the main content area has a dark background. It features a "Create Your Account" heading, followed by five input fields: "First Name", "Last Name", "Email", "Password", and "Confirm Password", and a "Register" button.

Figure 4.14: Registration screen

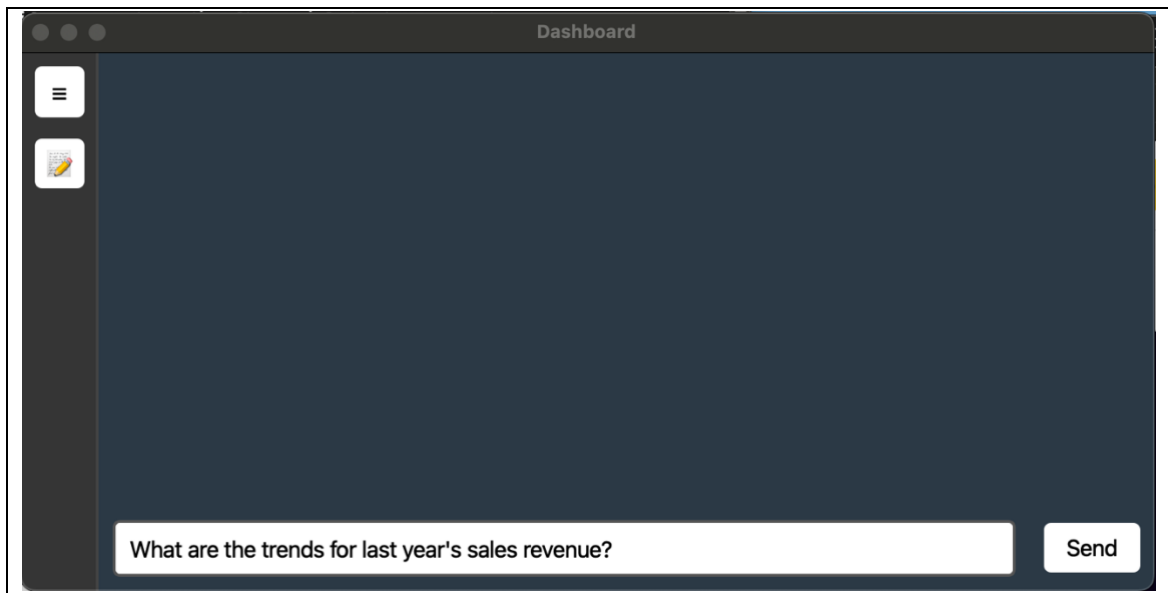


Figure 4.15: Prompt field screen

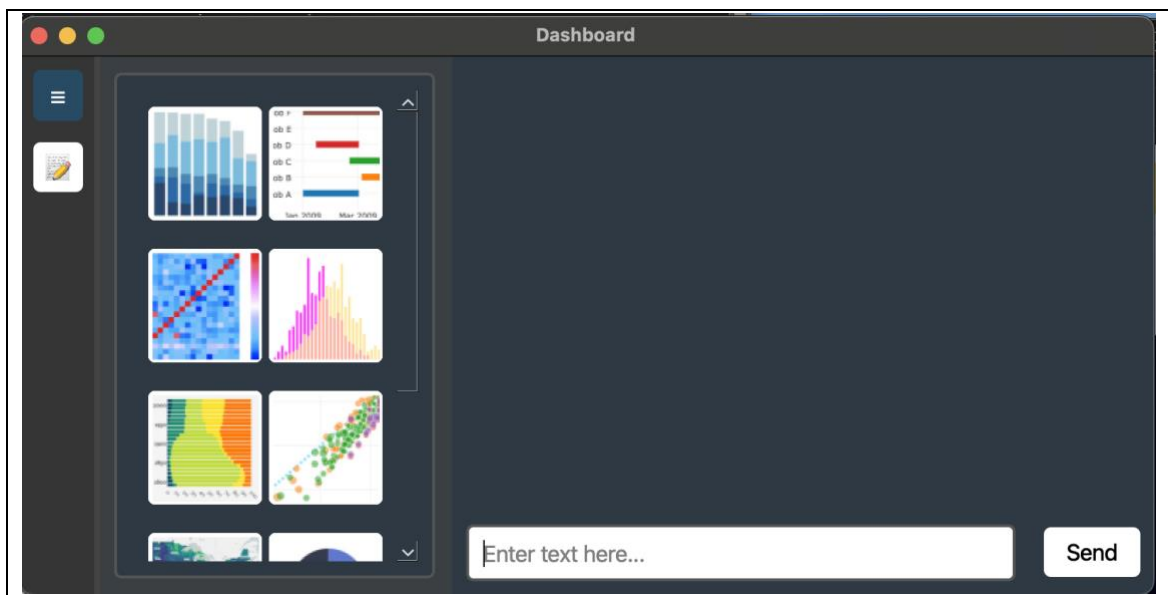


Figure 4.16: Prompt field screen (with sidebar open)

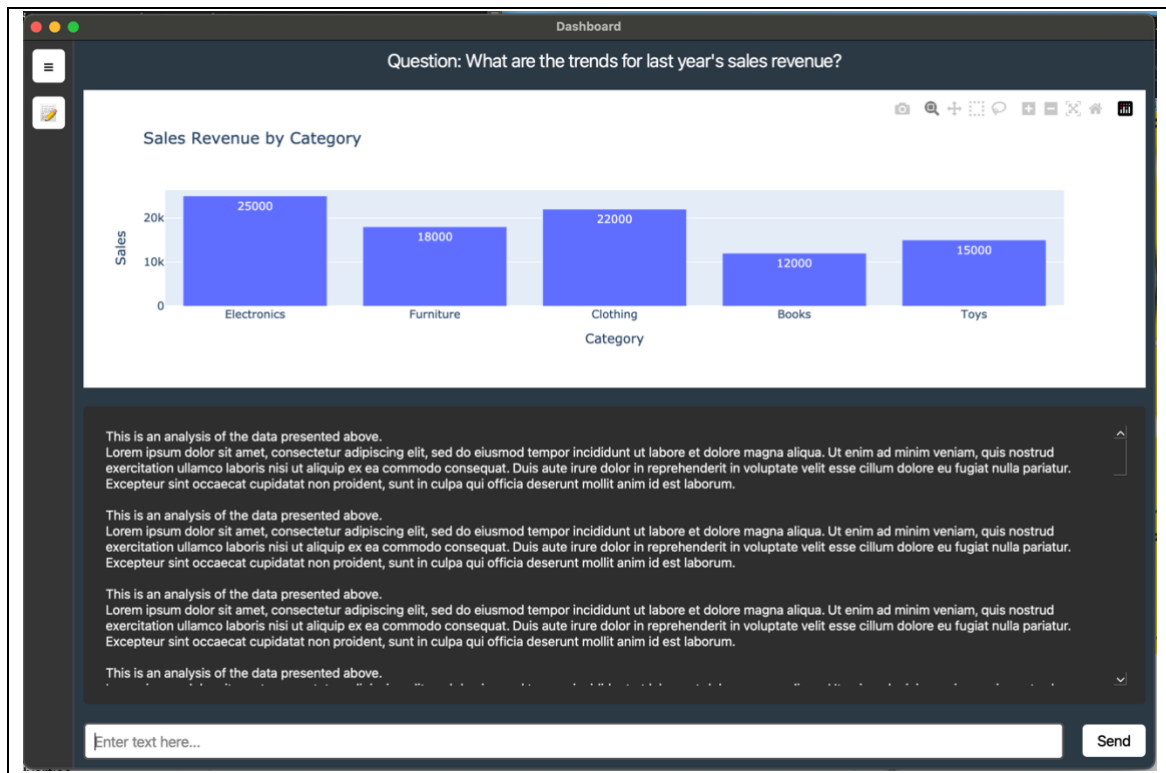


Figure 4.17: Prompt field screen (with visual and analysis)

Chapter 5: Implementation

5.1. Software Development Model

The software development model used for developing this software is the traditional waterfall model of SDLC. The software development life cycle (SDLC) is a process used to design and develop reliable and cost-effective software within a set time. It is also known as the software development process model. SDLC models provide a structured approach to creating high-quality software. As technology changes and market demands grow, organizations must choose the right SDLC model to meet project needs. Developers, project managers, and quality engineers need to understand the benefits and limitations of different models to ensure software quality.

The Waterfall model follows a strict sequence where each phase must be completed before moving to the next. It does not allow returning to a previous phase for changes which makes this suitable for small projects where revisions are minimal. In this model, problems cannot be fixed until the maintenance stage. SDLC models help provide a clear roadmap for software projects by ensuring careful planning and execution at each stage (Pargaonkar, S., 2023). Figure 5.1 displays the waterfall model diagram this project incorporates.

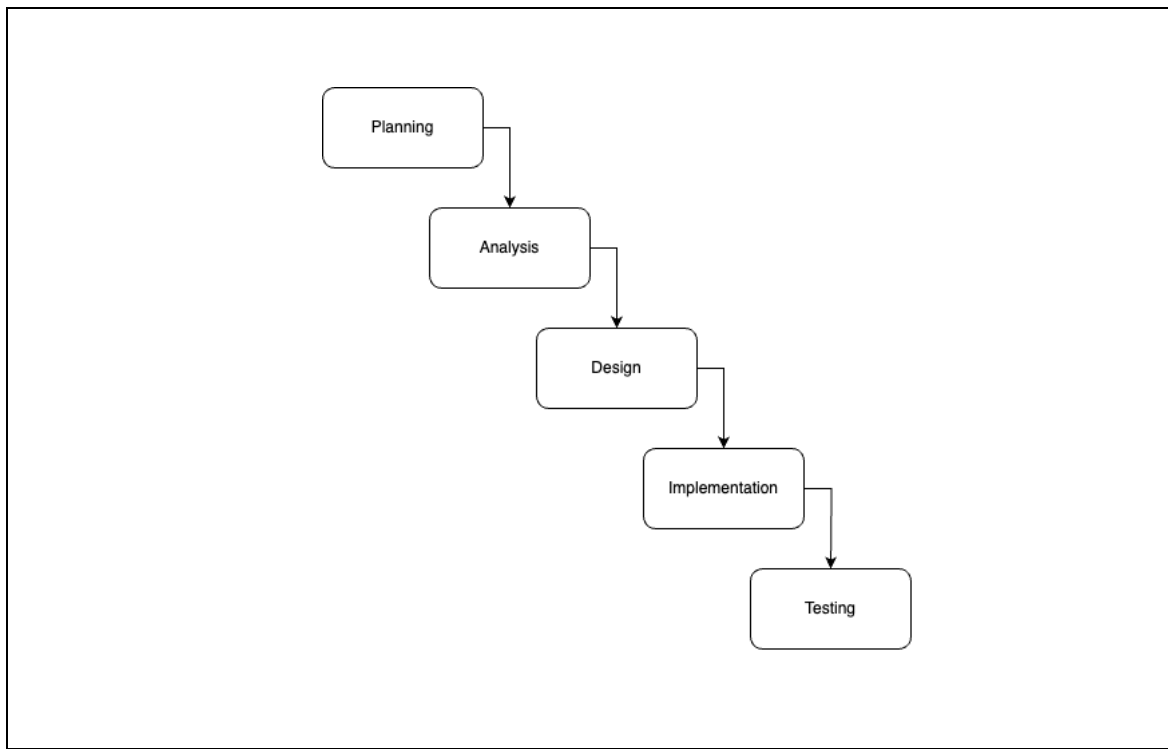


Figure 5.1: Waterfall model diagram (Pargaonkar, S., 2023)

The SDLC begins with planning the baseline and define the scope of the project. The consideration for this project includes the trimester length, classes enrolled and the classwork, the resources, and technical skills.

Afterwards, the analysis phase begins by studying the background of the project. The study done is to introduce theories, concepts, terms, and ideas that may be unfamiliar to the target audience. Concepts such as data visualization, NLP, and LLM are introduced to understand the problems and how these concepts could potentially solve it. Afterward, the project continues by recognizing similar system to the project to gather existing features. The features then are assessed further through questionnaire to further understand which of these features make a proper data visualization tool that follows the current trend.

By the end of this phase, the functional and quality requirements of the system are finalized.

The design stage continues from the analysis phase where the high-level design is created to depict the system's functionalities, interaction with external system, the data design, and the function sequence.

The implementation and testing are done to verify and validate the system while at the same time make changes to the system based on the response from the User Acceptance Test (UAT) result. The purpose of these phases is to ensure the system is outcome is as expected or not.

For FYP2, the data visualization tool will be developed. The developed data visualization tool will undergo the testing phase to verify the output, alongside identifying errors generated from different components and functions of the data visualization tool. When the final product is developed, the process will be properly documented. The presentation for FYP2 of the project will involve the demonstration of the final product.

5.2. Deployment

This section describes the whole development and deployment process of the project.

5.2.1. Implementation Scope

The implementation of this system focuses on delivering a functional multi-agent data visualization tool powered by LLM. The data visualization tool is developed within this scope:

1. Integrating LLM agents via LangChain & LangGraph
2. Implementing a Flask backend with API endpoints
3. Building a React frontend for prompt submission and result display
4. Enabling MySQL connection for data queries and a NoSQL database for storing saved visuals and tool preferences
5. Supporting 5 visualization tools (scatter, bar, pie, choropleth, table)
6. Allowing user to save the generated visual and view the saved visual
7. Allowing user to choose the tools to be considered by the agent
8. Viewing of the output from each agent after the visual is generated

Incorporating large language model (LLM) agents using LangChain and LangGraph to enable multi-agent coordination and workflow management. Developing a Flask-based backend that exposes API endpoints for communication between the frontend and the agent system. Creating a React.js frontend to handle user input prompts, database credentials, and display generated visualizations and analysis. Enabling user-supplied MySQL connection details, fetching of database schema, and retrieving relevant data for visualization. Use of NoSQL is to store user's saved visual and selected tools.

The data visualization tool will support five types of data visualizations: Scatter Plot, Bar Chart, Pie Chart, Choropleth Map, and Table, using Plotly and LangChain tools. The data visualization tool will also implement the ability for users to save generated visualizations and retrieve/view them later through the frontend interface. Users can select which visualization tools should be considered by the agent during the generation process. A detailed view of each agent's output during the workflow can be viewed by the user after the visualization succeeded.

The following features are not included in the current implementation scope but may be considered for future development:

1. Support for Non-MySQL Databases
2. Natural Language Conversation Interface
3. Mobile or Responsive Design
4. User Registration and Profile Management
5. Exporting Visualizations
6. Concurrent Multi-User Collaboration

Only MySQL is supported for database connections. Other database types are not covered. The system does not support multi-turn chat-style interaction. Prompts are processed as single, standalone inputs. The frontend is designed for desktop usage. Mobile optimization and responsive layout are not implemented at this stage. Only login functionality is supported. Features like user signup, profile editing, or role-based access control are not implemented. Generated visuals cannot be downloaded or exported to formats like PNG, PDF, or CSV. Real-time collaboration between multiple users is not supported. All interactions are handled individually.

5.2.2. Software Modules

This system is built using several software modules. Each module has a specific role in making the system easier to understand and maintain. The system is modularly designed to separate responsibilities, improve maintainability, and enable clear interaction between the frontend, backend, and multi-agent components.

Table 5.1: Software modules for development

Software Module	Purpose
User authentication	Handles login. It checks the username and password and gives access to the system if the user is valid.
Database connection	Let users enter their MySQL connection details. It connects to the database and automatically fetches the schema so the agents can use it. Also handles a local SQLite database stored in the website folder. It is used to store and retrieve saved visuals, selected tools, and prompt history for each user. It does not require manual setup or external credentials.
Prompt handling	Takes the user's input and sends it to the router agent to begin the workflow.
LLM agent	Runs the multi-agent system using LangChain and LangGraph. It includes an SQL generator agent, an

	SQL validator agent, an SQL improvement agent, a dictionary generator agent, a dictionary output validator agent, a visualization agent, and an analysis agent. Each agent has its own job and passes data to the next one.
Visualization tools	Provides five tools for generating graph and chart. These tools are scatter plot, bar chart, pie chart, choropleth map, and table. Each one is registered as a LangChain tool and includes a short description of what data it needs to let the agent decide the parameter for the graph and chart function.
Frontend UI	Allows users to log in, connect to a database, submit a prompt, view agent outputs, see the generated visual and analysis, choose tools, and view saved visuals.

5.3. Development Environment

This section explains the programming languages used, framework and libraries, IDEs and tools, version control system, and operating system.

5.3.1. Programming Languages Used

This system was built using a few different programming languages.

Python is a high-level and general-purpose programming language used for many types of problems. It is interpreted, interactive, and object-oriented. Python includes features like modules, exceptions, dynamic typing, and high-level data types. It supports different programming styles, such as object-oriented, procedural, and functional programming. Python is powerful and has clear syntax. It connects to system calls, libraries, and window systems. Developers can extend it using C or C++. It can also be used to add scripting capabilities to applications. Python is portable and works on Linux, macOS, and Windows (Python Software Foundation, n.d.). For this system, Python is used for the backend and agent logic.

The frontend of the system is developed using the combination of HTML, CSS and JavaScript.

An HTML document is a plain text file made up of elements. Elements are defined using opening and closing tags inside angle. It is used to define the overall structure of the website. HTML tags can include attributes. Attributes give extra details that change how the browser handles the element (MDN, 2024).

JavaScript is a programming language used to create interactive and dynamic features on web pages. It runs functions like content updates, interactive

maps, animations, and video controls. JavaScript is part of the three main web technologies, along with HTML and CSS (MDN, 2025).

CSS, or Cascading Style Sheets, is a language used to define the appearance of web pages. It controls layout, colors, fonts, and other visual aspects. The browser applies CSS rules to selected elements based on property-value pairs. CSS works mainly with HTML but it can also be used with other markup languages like SVG and XML (MDN, 2024).

5.3.2. Framework and Libraries

Several frameworks and libraries are used in this project.

For this web-based system, React is used for the frontend. React is a JavaScript library for building user interfaces using components. A component is a reusable piece of the interface, such as a button or image. User can create custom components like Thumbnail or Like Button and combine them to form full pages or applications. React components are JavaScript functions. They receive data and return what should be shown on the screen. User can use if statements to show content conditionally or `map()` to display lists. JSX is the syntax used in React. It mixes markup with JavaScript logic to make components easier to manage. React updates the user interface automatically when the data changes. User can add React to an existing HTML page and use it to build only the parts that need interactivity. React does not control routing or data fetching, but user can use frameworks like Next.js or Remix to build full applications. React is also an architectural pattern. It supports data fetching on the server or during build time. User can read data from files or databases and pass it to interactive components. React works on both web and mobile platforms. This allows teams to build cross-platform apps using shared skills. React helps organizations deliver

consistent user experiences and manage features across different platforms (React, n.d.).

Flask is responsible for handling the backend of the system, mostly the website routing. Flask is a web framework written in Python. It allows developers to build web applications with minimal setup, which is perfect for a system of this scope where the complexity is more emphasized on the LLM agents. Flask has a small and simple core, which makes it a microframework. It does not include built-in features like an ORM. Developers can add such features using extensions. Flask provides important tools like URL routing and a template engine. It is based on the WSGI toolkit Werkzeug and the Jinja2 template engine. Both are part of the Pocco project. Flask was created by Armin Ronacher and the Pocco team. Flask is designed to be simple, readable, and scalable. It does not hide how things work, which makes it easy to understand. A basic web app like “Hello World” can be written in just a few lines. Flask supports modular development. Large projects can be organized into multiple files. Although it is a microframework, Flask can be used to build complex applications. Developers are free to choose how to handle databases and other features. Flask is popular and widely used. It supports modern development practices and can be extended easily. It gives developers control over how their application is built and scaled (What is Flask Python—Python Tutorial, n.d.).

Plotly is used to generate charts. Plotly.py is an open-source graphing library for Python. It creates interactive, browser-based charts. The library is powered by plotly.js and supports over 30 chart types. These include 3D graphs, scientific plots, statistical charts, financial charts, and SVG maps (Plotly, n.d.). Plotly is often compared to Matplotlib and Bokeh. Matplotlib is older and very reliable but produces static visuals. Plotly and Bokeh offer more interactive and visually rich charts. Plotly is widely used for its ease of use and strong visual output (Domino, n.d.).

LangChain and LangGraph manage the agent system. To implement the LLM into the data visualization tool, the open source LLM framework, LangChain, will be utilized for integrating the LLM into the system. LangChain is a framework that allows LLM to be integrated into a traditional software workflow. It caters for various LLM use cases such as chatbot, question-answering, summarization, and agent (IBM, 2023). The LLM agents will be working together using LangGraph, a module built on top of LangChain that provides a library for deploying complex agent workflow. It uses graph to dictate the interactions between agents (LangChain, Inc., 2024).

Groq is used to run the LLM, the llama-3.3-70b-versatile model.

Llama-3.3-70b-versatile is a large language model developed by Meta. It has 70 billion parameters and supports a wide range of natural language processing tasks. It is designed for high performance and strong multilingual capabilities. The model is based on the Llama 3.3 architecture. It uses Grouped-Query Attention to improve efficiency and scalability during inference. It is fine-tuned with supervised learning and reinforcement learning using human feedback. This helps align responses with human preferences in terms of safety and usefulness. Llama-3.3-70b-Versatile performs well across key benchmarks. It scores 86.0% on MMLU for multitask understanding. It achieves 88.4% on HumanEval for code generation. It scores 77.0% on MATH and 91.1% on MGSM for mathematical reasoning. These results show strong performance in language, coding, and analytical tasks. The model is useful for applications like multilingual chat, code generation, and complex problem-solving. It works best when given clear task instructions and enough context. Tool use should also be defined with clear parameters and outputs (Groc Inc., n.d.). The strength of the model in doing analytical tasks and complex problem-solving, combined with the ability to utilize the tool use feature makes this model the most suitable LLM to be used for this system's scope.

Groq AI is a technology company that develops high-performance hardware and software for artificial intelligence. It was founded by former Google engineers. The company focuses on speed, efficiency, and accessibility in AI development. The core of Groq's system is the Language Processing Unit, or LPU. It is designed specifically for AI inference and language tasks. Unlike traditional GPUs built for graphics, the LPU is optimized for low-latency, high-speed computation (Groq, Inc., 2021).

Groq's approach emphasizes low energy use, consistent performance, and easy scalability. The system architecture avoids unnecessary complexity. It enables AI systems to operate efficiently in both small and large-scale environments. Groq AI supports various applications. In natural language processing, it improves model performance and response accuracy. In computer vision, it enhances image and video analysis for tasks like surveillance. In high-performance computing, it accelerates complex calculations used in fields such as genomics. The technology also supports emerging fields. In autonomous vehicles, Groq AI enables fast, reliable decision-making. In robotics, it improves control and precision for industrial and service applications. Users can access Groq AI through the GroqCloud platform. This cloud service allows developers to use Groq hardware without managing physical systems. The platform supports integration with AI frameworks such as TensorFlow and PyTorch. It provides tools and documentation for quick development and deployment. GroqCloud allows users to test and optimize models using a simple interface. Groq AI offers a combination of speed, simplicity, and scalability. Its architecture enables efficient performance across a wide range of AI tasks (Groq, Inc., n.d.).

Groq hardware supports models like Llama-3.3-70b-versatile. It enables fast and efficient inference without relying on traditional GPUs. Users can run Llama models on GroqCloud or on local systems. Groq's architecture ensures consistent performance, even for large models. Together, Llama-3.3-70B-

Versatile and Groq provide a powerful solution. The model offers advanced AI capabilities. Groq provides the hardware to run it quickly and reliably. This combination supports real-time applications across language, science, and industry (Groq, Inc., n.d.).

In short, the Llama-3.3-70b-versatile language model is run on Groq hardware while LangChain is responsible to integrate the model with the system logic. Groq runs the model and returns answers fast, while LangChain decides what to ask, how to structure the workflow, and what to do with the result.

Another library used is Pandas. It's used to handle data to be passed as the parameter for the Plotly function. Pandas is an open-source data analysis and manipulation tool. It is built on the Python programming language. It is fast, flexible, powerful, and easy to use (Pandas, n.d.).

This system primarily uses pandas DataFrame structure to be passed within the Plotly function's parameter. A DataFrame is a two-dimensional labeled data structure. It consists of rows and columns. Columns can hold data of different types. A DataFrame is similar to a spreadsheet, a SQL table, or a dictionary of Series. A DataFrame can be created using different input types. These include dictionaries of one-dimensional arrays, lists, Series, or other dictionaries. It also accepts two-dimensional NumPy arrays, structured arrays, Series, or another DataFrame. Users may also specify index labels for rows and column labels for columns. If provided, these labels define the structure of the resulting DataFrame. Any data not matching the given index or columns will be discarded. If labels are not specified, pandas generates them based on the input data using default rules. The DataFrame is the most commonly used object in pandas for data manipulation and analysis (Pandas, n.d.).

To summarize the frameworks and libraries used, the website itself is built on top of React for the frontend and Flask for the backend. Within Flask, the

website route and the agent logic is implemented while React reflects the operations to the user's view. The agent tasks and workflow is defined with the help of LangChain and LangGraph. Groq is responsible for running the model within the website. Lastly, pandas DataFrame is used for passing the data to the Plotly functions as a parameter.

5.3.3. IDEs and Tools

Development is done using a few main tools.

Visual Studio Code is a lightweight source code editor with advanced developer tools. It supports IntelliSense for code completion and built-in debugging features. The editor is designed for a smooth edit-build-debug workflow with minimal setup. It includes fast editing capabilities with syntax highlighting, auto-indentation, bracket matching, box selection, and code snippets. Keyboard shortcuts and customizable mappings improve navigation and efficiency. Visual Studio Code offers strong code understanding. It supports semantic analysis, code navigation, and refactoring. Debugging tools allow users to step through code, inspect variables, and manage call stacks. The editor integrates with build systems and scripting tools to streamline development workflows. Git support is included for version control tasks such as committing changes and viewing diffs. Users can install third-party extensions and customize the environment. Most features work out of the box. Visual Studio Code also allows configuration for specific needs. It is open-source and maintained by an active community on GitHub. The editor includes support for JavaScript and TypeScript using the same engine as Visual Studio. It also provides tools for JSX, React, HTML, CSS, SCSS, Less, and JSON development (Microsoft Corporation, Inc., n.d.).

The Groq dashboard is used to manage the LLM token (Groq, Inc., n.d.). This dashboard displays the model's live and history status. It also shows the token usage for the model. Since this is a free-tier service, the input and token output are limited to 100,000 token per day, hence this dashboard is vital for tracking the token usage for the model especially during development when to evaluate the model output using different prompts.

Jupyter Notebook is used to test early agent code and prompts. It is used to explore the test data and test the model before implementing it into the system (Jupyter Team, n.d.). Jupyter Notebook is a tool for developing and presenting data science projects. It combines code, visual output, and text in a single document. This creates a clear and interactive workflow. A notebook blends code and results in one place. It supports running code, displaying output, and adding explanations or charts. This improves transparency and makes work easier to reproduce. Jupyter Notebooks are widely used in data science workflows. They help users explore data, test ideas, and share results efficiently. The tool is open source and free. It can be downloaded from the Project Jupyter website or accessed through the Anaconda toolkit. Jupyter Notebooks support several languages. Python is the most common. Other supported languages include R, Julia, and Scala. This format helps streamline data work. It also improves collaboration and communication of insights (Pryke, B., 2025).

5.3.4. Version Control System

GitHub is used as the version control system to manage code changes and used as the interface for the Git version control (GitHub, Inc., n.d.). While this system does not benefit from the very useful collaborative feature of Git and GitHub, it is mainly used to manage and track the system's code in one place. In

case of major complication with the latest version of the code, Git allows the project to be tracked back to a certain point of the project.

5.3.5. Operating System

The system is developed on macOS Sequoia version 15.1. However, the data visualization tool runs on any operating system through a modern web browser.

5.4. System Configuration and Setup

This section explains how the backend, frontend, and build tools are set up to support the system. Each part of the system must be configured properly before it can run.

5.4.1. Backend Setup

The backend is developed using Flask. It acts as the main server and handles all API requests between the frontend and the agents. Flask runs as the local web server during development (Pallets., n.d.).

The backend also connects to a MySQL database. The user must provide host, username, password, and database name to make the connection (Oracle Corporation., n.d.). Once connected, the system fetches the database schema automatically and uses it for query generation and visualization.

In addition to MySQL, the system uses SQLite to store user-specific data. This includes saved visuals, selected tools, and prompt history. SQLite is used because it is lightweight and easy to set up (Sqlite documentation, n.d.). The backend includes functions to read from and write to the SQLite database.

5.4.2. Frontend Setup

The frontend is built using React. It is created and managed using React scripts. These scripts handle running the development server, building the project, and optimizing the frontend code (Meta Platforms, Inc., n.d.).

For the UI, the system uses basic custom CSS with optional support for component libraries (Refsnes Data., n.d.). No full Bootstrap or Material UI integration is used. Charts and graphs are rendered using Plotly.js (Plotly., n.d.), which works well with React.

All interactions with the user happen in the frontend. This includes logging in, connecting to the database, choosing tools, submitting prompts, and viewing visual and analysis results.

5.4.3. Build Tools and Package Managers

The system uses npm as the main package manager. It is used to install and manage React dependencies on the frontend. npm stands for Node Package Manager. It is both a registry and a tool for JavaScript and Node.js development. npm provides an online repository for publishing open-source JavaScript packages. It allows users to share and access libraries for different functionalities. npm also includes a command-line tool. This tool helps users install packages

and manage dependencies. It can also handle version control and script automation. npm is widely used in frontend and backend JavaScript projects. It supports efficient development through reusable modules and simplified package management (Abramowski, 2022).

Flask and backend dependencies are managed using pip. pip is the standard package manager for Python. It is used to install and manage libraries not included in the Python standard library. pip helps users handle dependencies in Python projects. It installs packages from the Python Package Index (PyPI). pip is widely used in data science, web development, automation, and general Python scripting. It supports simple installation, upgrade, and removal of packages through the command line interface (Acsany, P., 2024).

5.5. Database Implementation

This system connects to a MySQL database using user-provided connection details using Python's MySQL connector (mysql-connector-python 9.3.0., n.d.).

MySQL is a widely used open-source relational database management system. It uses SQL to manage structured data. MySQL stores data in tables with rows and columns organized into schemas. A schema defines how data is organized and how tables relate to one another. The database schema is crucial for this system since the agent is responsible for generating the accurate SQL query. This format supports data types such as text, numbers, dates, and JSON. MySQL is commonly used in software applications to store and retrieve data. MySQL supports ACID transactions. ACID stands for atomicity, consistency, isolation, and durability. These properties ensure data integrity even during system failure. MySQL is designed for performance and scalability. It handles a

high volume of concurrent connections. It supports large datasets and allows scaling through native replication features. MySQL is easy to install and configure. It suits both small projects and large-scale applications. It is backed by a strong community and extensive documentation. These factors contribute to its long-standing popularity in database development and the number one choice for the supported database the system can be connected to for data retrieving (Erickson, J., 2024).

This system also handles a local SQLite database stored in the website folder. This database stores saved visuals, selected tools, and prompt history. It does not need manual setup or external credentials as the table creation is checked every time the system runs. SQLite is a lightweight and widely used relational database management system. It is embedded, serverless, and operates as an in-memory open-source library. It requires no configuration or installation. SQLite is less than 500 KB in size. It does not need a separate server process to run. This makes it suitable for applications where simplicity and low overhead are important. SQLite supports multiple databases in a single session. It is flexible and cross-platform. It runs on macOS, Windows, Linux, and other systems. Many organizations use SQLite in embedded systems and applications. For example, Adobe uses SQLite in Photoshop Lightroom. Airbus uses it in flight software for the A350 aircraft family. SQLite is open source and does not require a license. Its ease of use and portability make it popular in both small-scale and embedded software projects. Based on the entity relationship diagram in figure 3.11, SQLite is the perfect database for this system as it is small scaled and not complex (Ravikiran, 2024).

5.5.1. Database Schema Design

The SQLite schema includes three tables. These are users, user_tools, and saved_visuals. Each table is linked using foreign keys to match data with the correct user. This structure keeps data organized and easy to manage.

5.5.2. SQL Database Tables

The SQL database in SQLite is designed based on the ERD from [figure 3.11](#) and data dictionary from [table 3.27](#).

The users table stores account information. It includes an ID, username, email, and password.

The user_tools table stores the tools selected by a user. It uses the user_id as a foreign key to link the record to the correct user.

The saved_visuals table stores the generated chart, the analysis, and the original prompt. It also saves the timestamp of when the data was created. This table also uses user_id as a foreign key.

5.5.3. Stored Procedures or Triggers

No stored procedures or triggers are used in this system. All logic is handled directly in the application using Python code.

5.5.4. Tools Used for Database Management

MySQL Workbench is used to test MySQL connections and view data. MySQL Workbench is a visual tool for database architects, developers, and

administrators. It supports data modeling, SQL development, and server administration. Users can configure servers, manage users, and perform backups. It combines multiple database tasks into one platform. MySQL Workbench runs on Windows, Linux, and macOS. It is used to simplify and streamline database management workflows (Oracle Corporation., n.d.).

SQLite is managed directly in code using Python's built-in support.

5.6. Key Modules and Features Developed

This section explains the submodules from each key modules and the features implemented from each module. Each submodules include the feature, code snippet, pseudocode, and its integration with other modules.

The full agent implementation including its expected data type, expected output structure, and its responsibilities are explained in detail within this section.

5.6.1. User authentication

User authentication module handles user registration, login, user session, and logout.

This system gives user access for a maximum of one hour before the token expires and logs out immediately. User will have to login again to regenerate the token.

Table 5.2: Register submodule

Feature	The user input their e-mail, username, and password and the submodule will attempt to create a new
---------	--

	account for the user. If the email already exists within the database, it will alert the user and user has to use a different e-mail.
Code Snippets	<pre>def register(): data=request.get_json() email=data.get("email") username=data.get("username") password=data.get("password") conn = sqlite3.connect('users.db') cursor = conn.cursor() try: cursor.execute("INSERT INTO users (username, email, password) VALUES (?, ?, ?)", (username, email, password)) conn.commit() except sqlite3.IntegrityError: return jsonify({ "ERROR": "E-mail already exists" }), 409 finally: conn.close() payload={ "id": cursor.lastrowid, "username": username,</pre>

	<pre> "email": email, "exp": int((datetime.datetime.now() + datetime.timedelta(hours=1)).timestamp()) } token=jwt.encode(payload, SECRET_KEY, algorithm="HS256") return jsonify({ "message": "Registration Successful", "token": token }), 200 </pre>
Pseudocode	<pre> BEGIN INPUT data READ email FROM data READ username FROM data READ password FROM data BEGIN CONNECT to database BEGIN ATTEMPT to insert email, username, password into user records IF insert fails due to conflict RETURN error message with status code END IF SET user_id to last inserted ID END CLOSE database connection END </pre>

	SET expiration time SET token using user_id, username, email, and expiration RETURN success message with token and status code END
Integration with other components	Connects to users.db to authenticate users during login and fetch tool preferences and saved visuals.

Table 5.3: Login submodule

Feature	Handles login. It checks the username and password and gives access to the system if the user is valid. Else, it will alert the user
Code Snippets	<pre>def login(): data=request.get_json() username=data.get("username") password=data.get("password") cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?", (username, password)) user=cursor.fetchone() if user: return jsonify({ "message": "Login Successful", "token": token</pre>

	}}, 200
Pseudocode	<pre>BEGIN INPUT username INPUT password CONNECT to database PREPARE SQL to select user where username and password match EXECUTE SQL READ user IF user exists THEN CREATE payload with user ID, username, email, and token expiration time ENCODE payload into JWT token RETURN JSON with success message and token ELSE RETURN JSON with error message ENDIF END</pre>
Integration with other components	Connects to the database to authenticate users during registration and fetch tool preferences and saved visuals.

Table 5.4: User session submodule

Feature	Handles current user session using JSON Web Token (JWT).
Code Snippets	<pre> def currentUser(): auth_header=request.headers.get("Authorization") if not auth_header: return jsonify({ "error": "Missing Token" }), 401 token=auth_header.replace("Bearer ", "") try: decoded=jwt.decode(token, SECRET_KEY, algorithms=["HS256"]) return jsonify({ "username": decoded["username"], "email": decoded["email"] }), 200 except jwt.ExpiredSignatureError: return jsonify({ "error": "Token Expired" }), 401 except jwt.InvalidTokenError: return jsonify({ "error": "Invalid Token" }) </pre>

	), 401
Pseudocode	<pre>BEGIN READ auth_header FROM request IF auth_header is missing RETURN error message for missing token with status code END IF SET token by removing prefix from auth_header BEGIN ATTEMPT to decode token using secret and algorithm IF decoding succeeds READ username and email from decoded token RETURN user information with status code END IF EXCEPTION token expired RETURN error message for expired token with status code EXCEPTION token invalid RETURN error message for invalid token with status code END END</pre>

Integration with other components	Connects to the database to authenticate users during registration and fetch tool preferences and saved visuals. Also used to display username and email in profile on the frontend.
--	--

Table 5.5: Logout submodule

Feature	End current user session.
Code Snippets	<pre>def logout(): connection_status["status"] = False connection_status["database_name"] = None connection_status["engine"] = None return jsonify({"message": "Logged out successfully and DB disconnected."}), 200</pre>
Pseudocode	<pre>BEGIN SET connection status to false SET database name to none SET engine to none RETURN success message with status code END</pre>
Integration with other components	Terminate the database connection.

5.6.2. Database connection

The connection to user's MySQL database is made using Python's mysql connector. Every time the connection is successfully made, the system will automatically fetch the database schema. It will retrieve the schema for every new database connection. The connection is terminated and the schema is deleted every time the user logs out.

Table 5.6: MySQL database connection submodule

Feature	Let users enter their MySQL connection details. It connects to the database and automatically fetches the schema so the agents can use it.
Code Snippets	<pre>def ValidateConnection(): try: data=request.get_json() username=data.get("username") host=data.get("host") database=data.get("database") password=quote_plus(data.get("password")) db_uri = f"mysql+mysqlconnector://{username}:{password}@{host}/{database}" db=SQLDatabase.from_uri(db_uri) table_names = inspector.get_table_names() for table in table_names: with engine.connect() as conn: result = list(conn.execute(text(f"SHOW CREATE TABLE `{table}`"))) if result: ddl = result[0][1] state["schema"].append(ddl)</pre>

Pseudocode	<pre> BEGIN INPUT JSON data from request READ username from data READ host from data READ database from data READ password from data ENCODE password for URL-safe usage CONSTRUCT db_uri using username, encoded password, host, and database TRY CONNECT to database using db_uri CATCH any connection error RETURN error response ENDTRY FOR each table in table_names BEGIN CONNECT to the database engine EXECUTE "SHOW CREATE TABLE" for the table READ the result IF result exists BEGIN GET the DDL statement from the result ADD the DDL to the schema in state ENDIF END END FOR END </pre>
Integration with other components	Connects to the LLM Agent by passing the MySQL schema so the agents can understand the database.

Table 5.7: Data preview submodule

Feature	Displays the preview of the database that has been connected to. It displays basic info such as database name, number of tables, tables. It also allows user to select tables from the dropdown and view the first five entries from the selected table. In addition, it also shows the schema overview and the amount of rows from selected table.
Code Snippets	<pre>def PreviewTable(table_name): if connection_status["status"]: engine=connection_status["engine"] try: inspector=inspect(engine) columns_info = inspector.get_columns(table_name) with engine.connect() as conn: preview_query = text(f"SELECT * FROM `{table_name}` LIMIT 5") result = conn.execute(preview_query) rows = [dict(row) for row in result.mappings()] count_query = text(f"SELECT COUNT(*) as total FROM `{table_name}`") total_rows = conn.execute(count_query).scalar() column_schema=[{ "name": col["name"], "type": str(col["type"]) } for col in columns_info]</pre>

	<pre> return jsonify({ "tableName": table_name, "columns": column_schema, "preview": rows, "rowCount": total_rows }), 200 except SQLAlchemyError as e: print(f"SQLAlchemyError: {str(e)}") return jsonify({ "error": "Error querying table", "details": str(e) }), 500 else: return jsonify({ "error": "No Connection" }), 400</pre>
Pseudocode	<pre>BEGIN IF connection is active SET engine from connection BEGIN ATTEMPT READ column information using inspection</pre>

	<pre> BEGIN CONNECT SET preview_query to select few rows from table EXECUTE preview_query READ rows SET count_query to count total rows EXECUTE count_query READ total count END SET column schema using column information RETURN table name, column schema, preview rows, total count with status code EXCEPTION on query error PRINT error RETURN error message with details and status code END END ELSE RETURN error message for no connection with status code END IF END </pre>
Integration with other components	This submodule is triggered after user make the connection to their MySQL database. It is displayed on the frontend within the data source page.

5.6.3. Prompt handling

The data visualization tool accepts prompt from the user to generate the visual and analysis before displaying the result to the user. The prompt field cannot be empty. The system will notify the user if user tries to submit an empty prompt.

Table 5.8: Process prompt submodule

Feature	Takes the user's input and sends it to the router agent to begin the workflow.
Code Snippets	<pre>def setQuery(): """ The processes from getting the input to generating the visualization """ global final_state try: data=request.get_json() question=data.get("question") final_state = graph.invoke(state) return jsonify({ "status": "success" }), 200</pre>
Pseudocode	BEGIN INPUT question from user

	<pre> READ data from request SET final_state by running the graph with current state IF no error RETURN success response ELSE RETURN error response ENDIF END </pre>
Integration with other components	setQuery() is the core function that handles the user's question. It extracts the prompt and prepares it for processing. The function calls graph.invoke(state) to trigger the multi-agent pipeline. This includes SQL generation agent, SQL validator agent, SQL improvement agent, Dictionary and output validator agents, Visualization and analysis agents. Each agent processes the data step by step and updates the shared state.

5.6.4. LLM agent

The LLM is the large language model used for each agent. The model used is llama-3.3-70b-versatile from Meta that is used for understanding complex tasks across different domains, problem solving, and executing analytical tasks (Groq, Inc., n. d.). To fully utilize the capabilities of the model and to ensure the best response is generated, the instruction for specific task must be defined clearly, and the context should be sufficient. The prompt requires specific format to ensure the model clearly understand the instruction by the user. Table 5.9 displays the tags included in the prompt to mark different section of the prompt.

Table 5.9: Prompt tags and purposes

Tag	Purpose	Cited Source
< begin_of_text >	Start of the prompt	Meta Platforms, Inc. (n.d.)
< start_header_id >	Enclose specific role for specific message: System, User, Assistant	
< end_header_id >		
< eot_id >	End of turn	

To incorporate agents into the system, a general workflow for generating the visualization must be defined. The workflow can be defined from the requirement analysis result. The features can be fit into a workflow that starts after user submitting the prompt and ends when the visualization and analysis are successfully generated.

Several agents can be implemented to execute specific tasks. Each agent will be equipped with instructions on what to do, the format of the output, the input to consider, and other specific constraints related to the specific task. In addition to the tasks being executed using agent, a router agent is implemented. The router will be responsible for verifying the output from other agent and decide whether the current output is good enough to move on to another agent, or the current output requires some fixing and improvement from the agent responsible for the output.

Each agent is equipped with specific purpose based on the data visualization workflow. Table 5.10 displays the details of each agent.

Table 5.10: Agents purpose, input, and output type

Agent	Purpose	Input	Output Type
writeQuery	Write SQL query	Database Schema	String of SQL query
		User Prompt	

executeQuery	Validate and Execute SQL Query	Database Schema	String of 'valid' or 'invalid' or 'end'
		User Prompt	
		SQL Query	
		Queried Data	
improveQuery	Improve existing SQL query	Database Schema	String of improvised SQL query
		SQL Query	
		Queried Data	
generateDF	Generate Dictionary of Data	User Prompt	String, formatted as dictionary, of data from database
		SQL Query	
		Queried Data	
chooseVisualization	Choose the Best Tool to Represent the Data	User Prompt	String of the graph or chart function fully equipped with the parameter
		Queried Data	
		Tools	
generateAnalysis	Generate Analysis based on the Data	SQL Query	String of the data analysis
		Queried Data	
dfValidator	Agent Router, responsible of verifying the dictionary output from agent generateDF and decide	SQL Query	String, formatted as dictionary, containing the name of the next agent, and the suggestion for improvement (if any)
		Queried Data	
		Graph or chart function	
		Data Analysis	

	whether the output require improvement.		
--	---	--	--

The output from each agent plays crucial role in the workflow as the content is directly used for the input of other agent. The instruction for generating a structured output is done through the prompt passed to the agent using few-shot prompting. Few-shot prompting allows for the model to learn something in-context by providing example of output to the agent (DAIR.AI, 2025). This method allows the agent to produce properly formatted output that can be directly converted into a python object. Table 5.11 displays the example string of the output structure and the expected data type of the output.

Table 5.11: Example of expected agents output

Agent	Example Structured Output String	Target Data Type
writeQuery	SELECT * FROM employees;	String
executeQuery	'(1, 'Alice Smith', 'Engineering', 80000), (2, 'Bob Jones', 'Marketing', 60000)'	String
generateDF	'{ 'id': [1, 2], 'name': ['Alice Smith', 'Bob Jones'], 'department': ['Engineering', 'Marketing'], 'salary': [80000, 60000] }'	Dictionary
chooseVisualization	Figure({ 'data': [{'hovertemplate': 'name=%{x} salary=%{y}<extra></extra>', 'marker': {'color': '#636efa', 'pattern': {'shape': ''}}},	plotly.graph_objs._figure.Figure

	<pre> 'orientation': 'v', 'showlegend': False, 'textposition': 'auto', 'type': 'bar', 'x': [Alice Smith, Bob Jones], 'y': [80000, 60000]]], 'layout': {'barmode': 'relative', 'legend': {'tracegroupgap': 0}, 'margin': {'t': 60}, 'template': '...', 'title': {'text': 'Employee Salaries'}}, 'xaxis': {'title': {'text': 'Employee Name'}}, 'yaxis': {'title': {'text': 'Salary'}}}))' </pre>	
dfValidator	<pre> {"agentName": "chooseVisualization", "improvementMessage": "" }' </pre>	Python dictionary

Based on table 5.11, the writeQuery agent, alongside several other agents, generate responses in form of string. This is considered the easiest form of data type as there is minimal action required to convert the agent response into a Python object.

The generateDF agent and the dfValidator agent will benefit from the few-shot prompt through its output. For example, the generateDF agent is expected to convert the result from the SQL query execution on the database into a string formatted as Python dictionary. The string is then to be directly converted into a dictionary data type. In case of the agent not producing the output in form of a dictionary, an error should be generated.

Another agent that requires precise output generation is the chooseVisualization agent. The output to be generated by this agent must be the function call from the defined tools list. This agent is responsible to select the best graph and chart from the tools list to represent the data and provide the parameter required for executing the function. This agent benefits heavily from the annotation for each parameter that defines the expected data type and the description of the parameter. Given the tools, user prompt, and the data as a Python dictionary object, the agent should be capable of generating the executable function with the necessary parameter.

With all the agents defined, the full workflow using these agents can be created. Figure 5.4 displays the workflow of the agents in a LangChain environment.

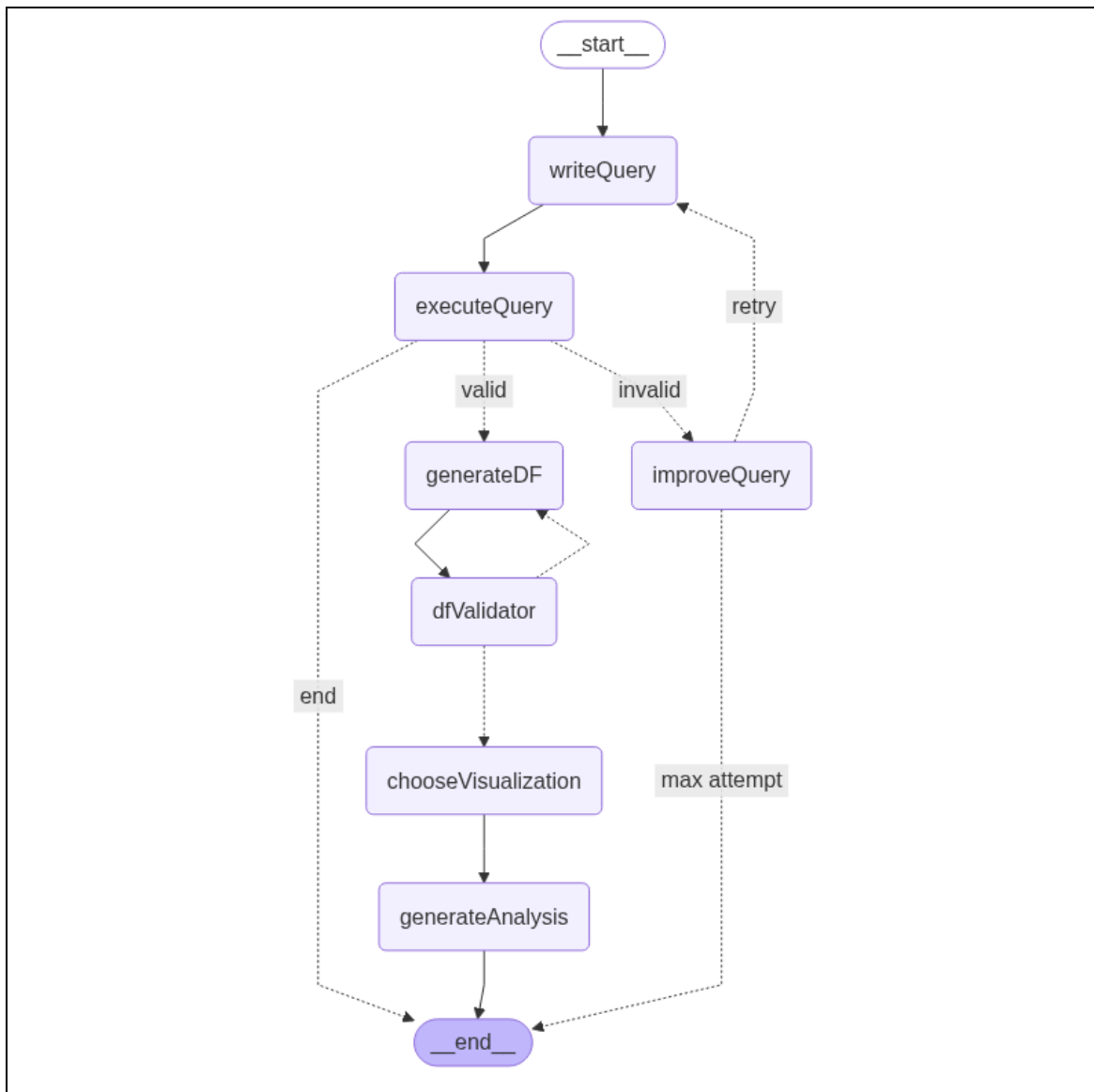


Figure 5.2: Full agents workflow

This workflow is initiated after user submitted a prompt. The writeQuery agent will analyze the prompt and the database schema to generate the SQL query. The generated SQL query is then passed to the executeQuery agent where the SQL query will be validated by the agent. There are three possible outcomes to be produced by this agent: 'valid', 'invalid', or 'end'. It generates the string 'valid' if the SQL query can be executed, and the agent sees the SQL query

matches the user prompt where it then moves to the next agent with the data resulted from the SQL query execution. If there are any error output from executing the SQL query, the executeQuery agent will include the error message in the output and moves to the improveQuery agent. Lastly, it will output 'end' if the prompt and the database schema are not related to each other.

The improveQuery agent takes the error message from the previous agent to generate the improvement needed for fixing the SQL query. The generated improvement message is then passed to the writeQuery agent for the SQL query to regenerate. The maximum amount of attempt that can be made by this agent is at most three (3) attempts.

In cases where the SQL query is executable and relates to user prompt, the result of the executed SQL query will be converted into a Python dictionary object generated through the generateDF agent. This agent is instructed to generate a single-level Python dictionary where the keys will be the column names, and the values will be the lists of column values. This structure is very important as the output from this agent, the Python dictionary string, will be used as the function parameter in the next agent. Any output that does not match the structure of a Python dictionary will fail the overall workflow.

Before passing the output from generateDF agent into the visualization function, another validator agent, dfValidator, is responsible for validating the output from generateDF agent. The purpose of this validator is to ensure the output from generateDF agent is in form of a Python dictionary. If yes, only then the workflow will progress to the next agent. Otherwise, it will redirect the workflow back to the generateDF agent with the improvement message from the dfValidator agent to fix the Python dictionary.

Based on the available graph and chart tools, the chooseVisualization agent is responsible for choosing the best tool to represent the data. The output is the string of Plotly function fully equipped with the parameter.

The last agent is responsible for generating analysis based on the user prompt and the data. It will assess the data and return the preview of the data, the data trend, and the main outcome.

The process of converting the string output into its respective data type is done using Python's built in function encapsulated within a try block. If the conversion fails, the workflow will return an error and the workflow will be halted.

These agents are wrapped inside a function. The function is responsible to convert the output of the agents into its respective data type.

A 'State' object is used to manage the agents output. It is used to store the current status of the executed workflow. Since every workflow may be using different prompt, database, and agent output, the state is cleared after each successful execution. Table 5.12 displays the objects stored within the state object.

Table 5.12: State object attributes

Attribute	Data Type	Description
db	object	User database connection
llm	object	LLM object
model	str	Model name
schema	List[str]	User database schema
prompt	object	SQL Query agent custom prompt
question	str	User prompt

query	str	SQL Query
result	str	Fetches data from user database using generated SQL query
retry	int	Agent retry count
SQLValidity	str	Validity of the SQL query
data	dict	Fetches data formatted as a Python dictionary
tools	list	List of available tools
analysis	str	Analysis text of the data
visualization	object	Visualization function
nextNode	str	Name of the next node in the workflow
routerCount	int	Amount of time the workflow has reached the router
improvement	str	Improvement message based on agent output
error	list	Error generated by the agent or output conversion during workflow execution

Out of all the agents, only the generateSQL agent uses existing prompt template from the LangChain community (LangChain, Inc., n.d.). It is instructed to convert a user's question into a syntactically correct SQL query that is valid for a specific database schema, returning a limited and relevant set of results. This

prompt guides a language model to convert a natural language question into a correct SQL query. The query must follow the given SQL dialect and use only the provided table and column names. It limits results to a maximum number unless stated otherwise. It avoids selecting all columns and prefers selecting only relevant ones. It may sort the results to return more useful or interesting answers.

Due to the token limitation, the limit of result for this system is set to 10 entries only to cater for the limited token usage for the LLM.

Table 5.13: Agents submodule

Feature	Runs the multi-agent system using LangChain and LangGraph. It includes an SQL generator agent, an SQL validator agent, an SQL improvement agent, a dictionary generator agent, a dictionary output validator agent, a visualization agent, and an analysis agent. Each agent has its own job and passes data to the next one.
Agent Code Snippet	
writeQuery agent	
Code Snippets	<pre>def writeQuery(state: State): prompt=prompt structured_llm = state["llm"].with_structured_output(QueryOutput) result = structured_llm.invoke(prompt) state["query"]=result["query"] return state</pre>
Pseudocode	BEGIN INPUT prompt

	READ tool FROM state CALL tool WITH prompt READ result SET state query TO result RETURN state END
validateQuery agent	
Code Snippets	<pre>def validateQuery(state: State): execute_query_tool=QuerySQLDatabaseTool(db=state["db"]) state["result"]=execute_query_tool.invoke(state["query"]) prompt=prompt response = state["llm"].invoke(prompt) state["SQLValidity"]=response.content return state</pre>
Pseudocode	BEGIN SET executor FROM state CALL executor WITH state query SET result TO state INPUT prompt CALL tool WITH prompt READ response SET state SQLValidity TO response RETURN state END
improveQuery agent	

Code Snippets	<pre>def improveQuery(state: State): if state["retry"]<=3: state["retry"]+=1 prompt=prompt response = state["llm"].invoke(prompt) state["improvement"]=response.content print(state["improvement"]) return state</pre>
Pseudocode	<pre>BEGIN IF state retry <= threshold THEN INCREMENT state retry INPUT prompt CALL tool WITH prompt READ response SET improvement TO state DISPLAY improvement RETURN state END IF END</pre>
generateDF agent	
Code Snippets	<pre>def generateDF(state: State): prompt=prompt response = state["llm"].invoke(prompt) data={"df": response.content} dict_data = ast.literal_eval(data['df']) state["data"]=dict_data</pre>

	<code>return state</code>
Pseudocode	<pre> BEGIN INPUT state READ question FROM state READ query FROM state READ result FROM state READ existing_data FROM state READ improvement FROM state SET prompt TO formatted instruction including all the above CALL LLM WITH prompt READ response READ 'df' FROM response CONVERT response TO dictionary STORE dictionary INTO state["data"] RETURN updated state END </pre>
chooseVisualization agent	
Code Snippets	<pre> def chooseVisualization(state: State): prompt=prompt llm_with_tools=state["llm"].bind_tools(state["tools"]) response = llm_with_tools.invoke(prompt) tool_calls = response.additional_kwargs.get("tool_calls", []) function_call = tool_calls[0] function_name = function_call["function"]["name"] </pre>

	<pre>function_args = json.loads(function_call["function"]["arguments"]) tool_mapping = {tool.name: tool for tool in state["tools"]} if function_name in tool_mapping: visualization_result = tool_mapping[function_name].invoke(input=function_args) state["visualization"] = visualization_result.to_html(full_html=False, include_plotlyjs='cdn') return state</pre>
Pseudocode	<pre>BEGIN INPUT prompt BIND tools FROM state CALL tool WITH prompt READ tool_call EXTRACT function name AND arguments MAP tool name TO tool IF function name IN tools THEN CALL tool WITH arguments SET visualization TO state RETURN state END IF END</pre>

generateAnalysis agent	
Code Snippets	<pre>def generateAnalysis(state: State): prompt=prompt state["analysis"] = state["llm"].invoke(prompt).content return state</pre>
Pseudocode	<pre>BEGIN INPUT prompt CALL tool WITH prompt READ response SET state analysis TO response RETURN state END</pre>
dfValidator Agent	
Code Snippets	<pre>def dfValidator(state: State): prompt=prompt if state["routerCount"] == 5: print("max router attempt") state["error"].append("dfValidator failed: max routing attempt reached") return state else: state["routerCount"]+=1 response = state["llm"].invoke(prompt) response_string=response.content formatted_response = json.loads(response_string) state["nextNode"]=formatted_response["agentName"]</pre>

	<pre>state["improvement"]=formatted_response["improvementMessage"] return state</pre>
Pseudocode	<pre>BEGIN INPUT prompt IF router count EQUALS limit THEN DISPLAY error message APPEND error TO state RETURN state ELSE INCREMENT router count CALL tool WITH prompt READ response EXTRACT fields FROM response SET nextNode TO state SET improvement TO state RETURN state END IF END</pre>
Integration with other components	These agents are triggered based on their sequence after user submits a prompt. The chooseVisualization agent uses the visualization tools to generate the visual based on user prompt and data from user's database.

5.6.5. Visualization tools

This module provides five tools for generating graph and chart. These tools are scatter plot, bar chart, pie chart, choropleth map, and table. Each one is registered as a LangChain tool and includes a short description of what data it needs to let the agent decide the parameter for the graph and chart function.

The tools are the graphs and charts the agent will choose from when generating visual. The graphs and charts are functions from Plotly (Plotly, n.d.) wrapped around a `@tool` decorator. The use of decorator is to register the function as a 'tool' within a LangChain environment (LangChain, Inc., n.d.). By wrapping each function with the `@tool` decorator, it allows for annotations to be passed for each function that can include the data type and the brief description of each parameter. There are 5 graphs and charts supported: Scatter Plot, Bar Chart, Choropleth, Pie Chart, and Table. The annotation of each tool's parameter is described in table 5.14.

Table 5.14: Bar chart function annotation and body

Feature	
Provides five tools for generating graph and chart. These tools are scatter plot, bar chart, pie chart, choropleth map, and table. Each one is registered as a LangChain tool and includes a short description of what data it needs to let the agent decide the parameter for the graph and chart function.	
Tools Details	
Function Name	
<code>barChart()</code>	
Parameters	Annotation

	Data Type	Description
data	<i>dict</i>	The dictionary containing the value to be plotted
x	<i>str</i>	The name of the x axis
y	<i>str</i>	The name of the y axis
Plotly Function Snippet		
<pre> return px.bar(data_frame=df, x=x, y=y) </pre>		
Bar Chart Function Pseudocode		
BEGIN INPUT data_frame df, string x, string y CREATE bar_chart using df[x] as x-axis and df[y] as y-axis RETURN bar_chart END		
Function Name		
scatterPlot()		
Parameters	Annotation	
	Data Type	Description
data	<i>dict</i>	The dictionary containing the value to be plotted
x	<i>str</i>	The name of the x axis
y	<i>str</i>	The name of the y axis
color	<i>str</i>	The key from the dictionary which contains the numeric value. the key capitalization must match exactly from the dictionary

title	str	The title of the graph
Plotly Function Snippet		
<pre> return px.scatter(data_frame=df, x=x, y=y, color=color, color_continuous_scale="plasma", render_mode="webgl", title=title) </pre>		
Scatter Plot Function Pseudocode		
BEGIN INPUT data_frame df, string x, string y, string color, string title CREATE scatter_plot using df[x], df[y], and df[color] for color intensity SET color scale to "plasma" SET render mode to "webgl" SET title of scatter_plot to title RETURN scatter_plot END		
Function Name		
choropleth()		
Parameters	Annotation	
	Data Type	Description
data	dict	The dictionary containing the value to be plotted
locations	str	The name of the column from the data that contains the locations
locationmode	str	The set of locations used to match entries in locations. acceptable parameter is either 'ISO-3', 'USA-states', or 'country names'

color	str	The key from the dictionary which contains the numeric value. the key capitalization must match exactly from the dictionary
title	str	The title of the chart
Plotly Function		
<pre> return px.choropleth(data_frame=df, locations=locations, locationmode=locationmode, color=color, color_continuous_scale='Viridis', title=title) </pre>		
Choropleth Function Pseudocode		
BEGIN INPUT data_frame df, string locations, string locationmode, string color, string title IF df contains columns locations AND color THEN CREATE choropleth_map using: - df as data source - locations for region codes - locationmode to interpret codes - color to define intensity - color scale set to 'Viridis' SET title of choropleth_map to title RETURN choropleth_map END		
Function Name		

piechart()		
Parameters	Annotation	
	Data Type	Description
data	<i>dict</i>	The dictionary containing the value to be plotted
names	<i>str</i>	The name of the column from the data that contains the names
values	<i>str</i>	The name of the column from the data that contains the values
title	<i>str</i>	The title of the chart
Plotly Function Snippet		
<pre> return px.pie(data_frame=df, names=names, values=values, title=title) </pre>		
Pie Chart Function Pseudocode		
BEGIN INPUT data_frame df, string names, string values, string title CREATE pie_chart using df[names] as labels and df[values] as sizes SET title of pie_chart to title RETURN pie_chart END		
Function Name		

table ()		
Parameters	Annotation	
	Data Type	Description
data	dict	The dictionary containing the value to be plotted
title	str	The title of the graph
Plotly Function Snippet		
<pre>fig = go.Figure(data=[go.Table(header=dict(values=list(df.columns), fill_color='paleturquoise', align='left'), cells=dict(values=[df[col] for col in df.columns], fill_color='lavender', align='left'))]) fig.update_layout(title=title) return fig</pre>		
Table Function Pseudocode		
BEGIN INPUT data_frame df, string title READ column_headers from df.columns READ column_values from each column in df CREATE table_figure using headers = column_headers and cells = column_values SET layout title of table_figure to title RETURN table_figure END		
Integration with other components		
The visualization agent selects these tools based on user intent and data. User prompt determines the need for these tools and will be routed to the right tool. The resulting plot is rendered in the UI for user viewing and interaction.		

5.6.6. Frontend UI

The UI of the website is responsible to cater for all interactions between user and the system. The information presented on user screen must match with the action to be executed when user interacts with the UI element.

Table 5.15: Various frontend UIs submodule

Feature	Allows users to log in, connect to a database, submit a prompt, view agent outputs, see the generated visual, choose tools, and view saved visuals.
Website Pages	
Login Page	
Code Snippets	<pre>const Login = () => { const [username, setUsername]=useState('') const [password, setPassword]=useState('') const handleChange=(set_value)=>(event)=>{ {set_value(event.target.value)} } const navigate = useNavigate() const handleSubmit=async (event)=>{ event.preventDefault() try{</pre>

```
const response= await
fetch("http://127.0.0.1:5001/auth/login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    username: username,
    password: password
  })
})

const data=await response.json()
if(response.ok){
  localStorage.setItem("token", data.token)
  navigate("/mainmenu")
} else {
  alert('Login failed: ${data.error}')
}
}catch(err){
  alert("Something went wrong")
  console.error(err)
}
}

return (
  <div className="container">
    <div className="wrapper">

      <div className="title">
        <span>Welcome!</span>
      </div>

      <p className='title_para'>Please enter your details to sign
in.</p>
```

```
<form
  method="POST"
  action="#"
  onSubmit={handleSubmit}>

  <div className="row">
    {/ * <i className="fas fa-user"></i> */}
    <input
      type="text"
      placeholder="Enter your Username"
      value={username}
      onChange={handleChange(setUsername)}
      required />
    </div>

    <div className="row">
      {/ * <i className="fas fa-lock"></i> */}
      <input
        type="password"
        placeholder="Password"
        value={password}
        onChange={handleChange(setPassword)}
        required />
      </div>

      <div className="pass"><a href="#">Forgot
password?</a></div>

      <div className="row button">
        <input
          type="submit"
          value="Login" />
        </div>

        <div className="signup-link"> Not a member? <Link
to="/register">Sign Up</Link></div>
```

	<pre></form> </div> </div>) }</pre>
Pseudocode	<pre>BEGIN SET username TO empty SET password TO empty PREPARE navigation handler ON input change READ input value SET corresponding state TO value WHEN form is submitted PREVENT default behavior TRY SEND login request TO server WITH username AND password READ response IF response is successful THEN STORE token IN local storage NAVIGATE to main page ELSE SHOW error message END IF</pre>

	CATCH error SHOW fallback error message LOG error END TRY DISPLAY container SHOW title AND message RENDER form WITH input for username ON change, update username WITH input for password ON change, update password WITH submit button ON click, trigger handleSubmit SHOW forgot password link SHOW signup link END
Connect to a database	
Code Snippets	<pre>const LocalDB = () => { const [username, setUsername]=useState("") const [host, setHost]=useState("") const [database, setDatabase]=useState("") const [password, setPassword]=useState("")</pre>

```
const handleChange=(setVale=>(event)=>{
  {setVale(event.target.value)}
})

const navigate = useNavigate()

const handleSubmit=async (event)=>{
  event.preventDefault()

  try{
    const response= await fetch("http://127.0.0.1:5001/data-
source/db-connection", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        username: username,
        host: host,
        database: database,
        password: password
      })
    })
    const data=await response.json()
    if(response.ok){
      alert(`${data.message}!`)
      navigate("/db-preview")
    } else {
      alert('Login failed: ${data.error}')
    }
  }catch(err){
    alert("Something went wrong")
    console.error(err)
  }
}
```

```
}

return (
  <div className="container">
    <div className="wrapper">

      <div className="back-parent">
        <div className="back-div">
          <button className="back-btn" onClick={() =>
navigate("/db-preview")}>
            ←
          </button>
        </div>
        <div className="title-ul">
          <h1>Connect to your Database</h1>
        </div>
      </div>

      <form
        method="POST"
        action="#"
        onSubmit={handleSubmit}>

        <div className="row">
          { /* <i className="fas fa-user"></i> */ }
          <input
            type="text"
            placeholder="Username"
            value={username}
            onChange={handleChange(setUsername)}
            required />
        </div>
      </form>
    </div>
  </div>
)
```

	<pre><div className="row"> /* <i className="fas fa-lock"></i> */ <input type="text" placeholder="Host" value={host} onChange={handleChange(setHost)} required /> </div> <div className="row"> /* <i className="fas fa-lock"></i> */ <input type="text" placeholder="Database" value={database} onChange={handleChange(setDatabase)} required /> </div> <div className="row"> /* <i className="fas fa-lock"></i> */ <input type="password" placeholder="Password" value={password} onChange={handleChange(setPassword)} /> </div> <div className="row button"> <input type="submit" value="Verify Connection" /> </div></pre>
--	--

	<pre></div> </form> </div> </div>) }</pre>
Pseudocode	<pre>BEGIN SET username TO empty SET host TO empty SET database TO empty SET password TO empty ON input change READ value SET corresponding field ON submit form PREVENT default action SEND data TO server WITH username, host, database, password READ response IF response is successful THEN SHOW message NAVIGATE to next page ELSE SHOW error message</pre>

	END IF DISPLAY form WITH inputs: - username - host - database - password - submit button DISPLAY back button ON click, NAVIGATE to previous page END
Submit Prompt	
Code Snippets	<pre>const Query={()=>{ const [inputText, setInputText]=useState("") const [showLLMOutput, setShowLLMOutput] = useState(false); const [LLMOutputContent, setLLMOutputContent] = useState(null); const [showDBOutput, setShowDBOutput] = useState(false); const [DBOutputContent, setDBOutputContent] = useState(null); const [showToolSelector, setShowToolSelector] = useState(false); const [isLoading, setIsLoading] = useState(false); const navigate=useNavigate()</pre>

```
const handleChange = (event) => {
  setInputText(event.target.value)
}

const handleSubmit = async(event) => {
  event.preventDefault()
  setIsLoading(true); // show loading screen

  console.log("Submitted:", inputText)

  let dbConnection=false;
  try{

    const dbStatus=await fetch("http://127.0.0.1:5001/data-
source/db-status", {
      method: "GET"
    })
    .then(res=>res.json())
    .then(data=>{
      if(data.status==="connected"){
        dbConnection=true // db status
        console.log("db found")

      }else {
        dbConnection=false;
        alert('You are NOT connected to a database. Please
make a connection to your database')
        setIsLoading(false);
        return
      }
    })
  }
  catch(err){
    console.error(err)
  }
}
```

```
    alert(err)
  }

  console.log(dbConnection);

  if(dbConnection===true){
    try {
      const response = await
fetch("http://127.0.0.1:5001/query/query-input", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({ question: inputText })
    })

    const data = await response.json()

    if (!response.ok){
      if (data.error && Array.isArray(data.error)) {
        alert("Errors:\n" + data.error.join("\n"));
      } else if (typeof data.error === "string") {
        alert("Error:\n" + data.error);
      } else {
        alert(`Unknown error occurred (HTTP
${response.status})`);
      }
      return;
    }

    if (data.status === "success"){
```



```
        console.log("backend (/query-input) return success")
        navigate("/visual-output")
    } else throw new Error("Backend responded with failure status")
    } catch (err) {
        console.error("Error submitting query:", err)
        alert(err.message)
    } finally{
        setIsLoading(false);
    }
}
}

const getToolSelector = async () => {
    const shouldShow = !showToolSelector;
    setShowToolSelector(shouldShow);
};

const getDBAttr = async () => {
    const shouldShow = !showDBOutput;
    setShowDBOutput(shouldShow);

    if (shouldShow && !IDBOutputContent) {
        const response = await fetch("http://127.0.0.1:5001/query/db-
details", {
            method: "GET",
            headers: { "Content-Type": "application/json" }
        });
        const data = await response.json();
        setDBOutputContent(data);
        if (data.error) {
            console.error("Server error:", data.error);
            alert(`Something went wrong: ${data.error}`);
        }
    }
}
```

```
    }  
  };  
  
  const getLLMAttr = async () => {  
    const shouldShow = !showLLMOutput;  
    setShowLLMOutput(shouldShow);  
  
    if (shouldShow && !LLMOutputContent) {  
      const response = await fetch("http://127.0.0.1:5001/query/llm-  
details", {  
        method: "GET",  
        headers: { "Content-Type": "application/json" }  
      });  
      const data = await response.json();  
      setLLMOutputContent(data);  
      if (data.error) {  
        console.error("Server error:", data.error);  
        alert(`Something went wrong: ${data.error}`);  
      }  
    }  
  };  
  
  return (  
    <div className="container">  
      {isLoading && (  
        <div className="loading-overlay">  
          <div className="spinner"></div>  
          <p>Processing your query...</p>  
        </div>  
      )}  
      <div className="wrapper">  
        <div className="back-parent">  
          <div className="back-div">
```

```
        <button className="back-btn" onClick={() =>
navigate("/mainmenu")}>
        ←
        </button>
    </div>
    <div className='title-ul'>
        <h1>Enter Your Query</h1>
    </div>
</div>
<form onSubmit={handleSubmit}>
    <div className="row">
        <textarea
            placeholder="Create a graph for ..."
            value={inputText}
            onChange={handleChange}
            required
        />
    </div>

    <div className="row button">
        <input type="submit" value="Submit" />
    </div>

</form>

<div className="dropdowns">
    <button onClick={getToolSelector}>Choose
Tools</button>
    {showToolSelector && (
        <ToolSelector token={localStorage.getItem("token")}
onClose={() => setShowToolSelector(false)} />
    )}

    <button onClick={getDBAttr}>DB Details</button>
```

	<pre> {showDBOutput && (<DBOutput output={DBOutputContent} onClose={() => setShowDBOutput(false)} />)} <button onClick={getLLMAttr}>LLM Details</button> {showLLMOutput && (<LLMOutput output={LLMOutputContent} onClose={() => setShowLLMOutput(false)} />)} </div> </div> </div>) }</pre>
Pseudocode	BEGIN SET input TO empty SET loading TO false SET tool section TO hidden SET db section TO hidden SET llm section TO hidden ON input change READ text SET input ON form submit PREVENT default SET loading TO true

	<pre>SEND request TO check database READ response IF database is connected THEN SEND input TO server READ result IF result is success THEN NAVIGATE to result page ELSE SHOW error END IF ELSE SHOW warning END IF SET loading TO false ON click tool button TOGGLE tool section IF section is now visible AND content is empty THEN FETCH tool content SET content IF error THEN SHOW error END IF</pre>
--	--

	<pre>END IF ON click db button TOGGLE db section IF section is now visible AND content is empty THEN FETCH db content SET content IF error THEN SHOW error END IF END IF ON click llm button TOGGLE llm section IF section is now visible AND content is empty THEN FETCH llm content SET content IF error THEN SHOW error END IF END IF DISPLAY input form DISPLAY buttons for tool, db, llm SHOW content IF sections are visible SHOW loading screen IF loading is true</pre>
--	--

	END
View Generated Visual	
Code Snippets	<pre>const VisualOutput = () => { const navigate = useNavigate(); const location = useLocation(); const [analysisText, setAnalysisText] = useState(""); const [stateOutputContent, setStateOutputContent] = useState(null); useEffect(() => { fetch("http://127.0.0.1:5001/query/generated-analysis") .then(res => res.text()) .then(text => setAnalysisText(text)) .catch(err => console.error("Failed to fetch analysis:", err)); }, []); // const handleframeLoad = () => alert("Visualization Success!"); // const handleframeError = () => alert("Failed to load the visualization."); const saveVisual = async () => { try { const token = localStorage.getItem("token"); const res = await fetch("http://127.0.0.1:5001/query/save-visual", { method: "POST", headers: { Authorization: `Bearer \${token}`,</pre>

```
        "Content-Type": "application/json",
    },
  });

  const data = await res.json();
  alert(data.message);
} catch (err) {
  console.error("Failed to save:", err);
  alert("Error saving visual and analysis");
}
};

const getStateAttr = async () => {
  try {
    const response = await
fetch("http://127.0.0.1:5001/query/state-details", {
    method: "GET",
    headers: {
      "Content-Type": "application/json"
    }
  });

    const data = await response.json();

    if (data.error) {
      console.error("Server error:", data.error);
      alert(`Something went wrong: ${data.error}`);
      return null;
    } else {
      console.log(data);
      return data;
    }
  } catch (err) {
    console.error("Failed to fetch state details:", err);
    alert("Could not retrieve state output.");
  }
}
```



```
return null;

}

};

const handleViewStateOutput = async () => {
  const stateData = await getStateAttr();
  if (stateData) {
    navigate("/state-output", { state: { stateData } });
  }
};

return (
  <div className='container'>
    <div className='wrapper'>
      <div className='back-parent'>
        <div className='back-div'>
          <button
            className="back-btn"
            onClick={() => {
              const confirmLeave = window.confirm(
                "Are you sure you want to go back?\n\nIf
you go back, the visual and analysis will be lost.\n\nIf you wish to
see this again, please save before going back."
              );
              if (confirmLeave) {
                navigate("/query");
              }
            }}
          >
            ←
          </button>
        </div>
        <div className='title-ul'>
```

```
        <h1>Result</h1>
      </div>
    </div>

    <h2>Visualization</h2>
    <iframe
      src="http://127.0.0.1:5001/query/generated-visual"
      title="Visualization"
      // onLoad={handleIframeLoad}
      // onError={handleIframeError}
      style={{
        width: '100%',
        height: '600px',
        border: 'none',
      }}
    ></iframe>

    <h2>Analysis</h2>
    <div className="analysis-box">
      <pre style={{
        background: '#f9f9f9',
        padding: '12px',
        borderRadius: '6px',
        whiteSpace: 'pre-wrap',
        wordBreak: 'break-word',
        fontFamily: 'monospace',
        fontSize: '14px'
      }}>
        {analysisText || "N/A"}
      </pre>
    </div>
```

	<pre><div className="visual-button-group"> <button className="visual-btn" onClick={saveVisual}>Save Visual and Analysis</button> <button className="visual-btn" onClick={handleViewStateOutput}>View State Output</button> </div> </div> </div>); };</pre>
Pseudocode	BEGIN SET analysis TO empty SET state output TO empty ON component load FETCH analysis text FROM server SET analysis ON click back button SHOW confirmation IF confirmed THEN NAVIGATE to previous page END IF ON click save button READ token SEND request TO save visual and analysis READ response SHOW message

	ON click state output button FETCH state details IF data exists THEN NAVIGATE to state output page WITH data END IF DISPLAY layout DISPLAY title DISPLAY visualization iframe DISPLAY analysis text box DISPLAY buttons: save button state output button END
View Agent Output	
Code Snippets	<pre>const AgentCard = ({ title, description, content, contentList }) => (<div style={{ border: '1px solid #ccc', borderRadius: '10px', padding: '16px', marginBottom: '16px', boxShadow: '0 2px 4px rgba(0,0,0,0.1)', backgroundColor: '#fff' }}></pre>

```
<h3 style={{ marginTop: 0 }}>{title}</h3>
<p style={{ fontStyle: 'italic', color: '#666' }}>{description}</p>

{contentList ? (
  contentList.map(({ label, value }, idx) => (
    <div key={idx} style={{ marginBottom: '12px' }}>
      <strong>{label}</strong>
      <pre style={{
        background: '#f9f9f9',
        padding: '12px',
        borderRadius: '6px',
        whiteSpace: 'pre-wrap',
        wordBreak: 'break-word'
      }}>
        {value || "N/A"}
      </pre>
    </div>
  ))
) : (
  <pre style={{
    background: '#f9f9f9',
    padding: '12px',
    borderRadius: '6px',
    whiteSpace: 'pre-wrap',
    wordBreak: 'break-word'
  }}>
    {content || "N/A"}
  </pre>
)}
</div>
);

const StateOutput = () => {
```

```
const location = useLocation();
const navigate=useNavigate()
const output = location.state?.stateData;

if (!output) {
  return <p>No state output available. Please try again from the
Visual Output page.</p>;
}

return (
  <div className="container">
    <div className="wrapper">

      <div className='back-parent'>
        <div className='back-div'>
          <button className="back-btn" onClick={() =>
navigate("/visual-output")}>
            ←
          </button>
        </div>
        <div className='title-ul'>
          <h1>Enter Your Query</h1>
        </div>
      </div>

      <h2 style={{ marginTop: '32px' }}>Agents Workflow</h2>

              maxHeight: '500px',         objectFit: 'contain',         marginTop: '20px',         border: '1px solid #ccc',         borderRadius: '8px'       }}     /&gt;      &lt;h2 style={{ fontSize: '1.75rem', marginBottom: '10px', marginTop: '32px' }}&gt;Agent Pipeline Output&lt;/h2&gt;      &lt;AgentCard       title="Question"       description="User Prompt"       content={output.question}     /&gt;      &lt;AgentCard       title="writeQuery Agent"       description="Generates the SQL query based on the user's question."       content={output.query}     /&gt;      &lt;AgentCard       title="validateQuery Agent"       description="Runs and validates the generated SQL query."       contentList=[         {           label: "Query Result",           value: JSON.stringify(output.result, null, 2)         },         {           label: "SQL Validity",</pre> |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre>        value: output.SQLValidity    "Unknown"       }     ]}   /&gt;    &lt;AgentCard     title="improveQuery Agent"     description="Improves the query if validation fails."     contentList=[       {         label: "Retry Attempts",         value: output.retry    "None"       }     ]}   /&gt;    &lt;AgentCard     title="generateDF Agent"     description="Parses the SQL result into a structured dictionary."     content={JSON.stringify(output.data, null, 2)}   /&gt;    &lt;AgentCard     title="chooseVisualization Agent"     description="Chooses and renders the appropriate visualization for the result."     content={output.visualization    "No visualization"}   /&gt;    &lt;AgentCard     title="generateAnalysis Agent"     description="Performs human-readable reasoning on the result."</pre> |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



|                   |                                                                                                                                                                                                                                                                                                  |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                   | <pre>        content={output.analysis}       /&gt;        &lt;AgentCard         title="dfValidator Agent"         description="Routes to the next step based on data validity."         content={ Router Count: \${output.routerCount} }       /&gt;     &lt;/div&gt;   &lt;/div&gt; ); };</pre> |
| <b>Pseudocode</b> | <pre>BEGIN    READ data FROM navigation state    IF data is missing THEN     DISPLAY message     RETURN   END IF    DISPLAY page layout    DISPLAY back button   ON click, NAVIGATE to previous page    DISPLAY title    DISPLAY image from server</pre>                                         |

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | <p>DISPLAY section for each agent:</p> <p>FOR each agent</p> <p>SET title and description</p> <p>IF agent has multiple outputs THEN</p> <p>FOR each label-value pair</p> <p>DISPLAY label and value</p> <p>END FOR</p> <p>ELSE</p> <p>DISPLAY content</p> <p>END IF</p> <p>END FOR</p> <p>END</p>                                                                                                                                                                                      |
|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Choose Tools</b>  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Code Snippets</b> | <pre>const [showToolSelector, setShowToolSelector] = useState(false);  const getToolSelector = async () =&gt; {   const shouldShow = !showToolSelector;   setShowToolSelector(shouldShow); };  ...  &lt;div className="dropdowns"&gt;   &lt;button onClick={getToolSelector}&gt;Choose Tools&lt;/button&gt;   {showToolSelector &amp;&amp; (     &lt;ToolSelector       token={localStorage.getItem("token")}       onClose={() =&gt; setShowToolSelector(false)}     /&gt;   )}</pre> |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          | <pre>    })<br/>  &lt;/div&gt;</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Pseudocode</b>        | <pre>BEGIN<br/><br/>  SET tool dropdown visibility TO hidden<br/><br/>  ON click tool button<br/>    TOGGLE tool dropdown visibility<br/><br/>  IF tool dropdown is visible THEN<br/>    DISPLAY tool selection component WITH<br/>token<br/>    ON close, SET visibility TO hidden<br/>  END IF<br/><br/>END</pre>                                                                                                                                                                             |
|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>View Saved Visual</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Code Snippets</b>     | <pre>function SavedVisualList() {<br/>  const [savedItems, setSavedItems] = useState([])<br/>  const navigate = useNavigate()<br/><br/>  useEffect(() =&gt; {<br/>    const fetchSaved = async () =&gt; {<br/>      try {<br/>        const token = localStorage.getItem("token")<br/>        const res = await<br/>fetch("http://127.0.0.1:5001/query/saved-visuals", {<br/>          headers: { Authorization: `Bearer \${token}` }<br/>        })<br/>      }<br/>    }<br/>  })<br/>}</pre> |

```
const data = await res.json()
setSavedItems(data)
} catch (err) {
 console.error("Failed to fetch saved visuals:", err)
}
}
fetchSaved()
}, [])

return (
 <div className="container">
 <div className="wrapper">
 <div className="back-parent">
 <div className="back-div">
 <button className="back-btn" onClick={() =>
navigate("/mainmenu")}>
 ←
 </button>
 </div>
 <div className="title-ul">
 <h1>Saved Visual</h1>
 </div>
 </div>
 <div className="card-container">
 {savedItems.map(item => (
 <button
 key={item.id}
 className="visual-card"
 onClick={() => navigate(`/saved-visual/${item.id}`)}>
 <h3>{item.prompt}</h3>
 <small>{new
Date(item.timestamp).toLocaleString()}</small>
 </button>
```

	<pre>    )}   &lt;/div&gt; &lt;/div&gt; &lt;/div&gt; ) }</pre>
<b>Pseudocode</b>	<pre>BEGIN    SET saved list TO empty   SET navigation    WHEN page starts     READ token     SEND request to backend WITH token     RECEIVE saved data     SET saved list TO data    DISPLAY back button   ON click     NAVIGATE to main menu    FOR each item in saved list     DISPLAY card WITH prompt and timestamp     ON card click       NAVIGATE to detailed visual view  END</pre>

<b>Integration with other components</b>	The UI is related to the most amount of modules since it is the interaction point between user and the system, hence there should be a UI for every possible action within the website.
------------------------------------------	-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

## 5.7. APIs and Integration

This section describes the API integration for the system.

### 5.7.1. Description of Internal APIs or Third-Party APIs Used

Table 5.16 provides a description of the internal APIs used in the system. These APIs are developed using Flask. They are designed to handle user actions. The internal APIs support the main functions of the system.

Table 5.16: Internal API description

Internal API	Description
Flask	Custom RESTful APIs that handle user actions.

Table 5.17 provides a description of the third-party APIs used in the system. These APIs are integrated to enhance the system's features. Each third-party API serves a specific purpose.

Table 5.17: Third-Party APIs description

Third-Party APIs	Description
LangChain	Used to integrate and orchestrate multiple LLM agents through LangGraph, enabling SQL generation, validation, data parsing, visualization selection, and analysis.
Plotly	Used in visualization agents to render charts (scatter, bar, pie, choropleth, table).
SQLAlchemy	Used to connect to and query MySQL databases via URI parsing.

### 5.7.2. API Endpoints Implemented

Table 5.18 displays all routes defined using Flask. These routes handle requests from the frontend. Each route responds to a specific user interaction. The system refers each interaction to the correct route. This ensures that the

system processes all requests in an organized way. The routes link the frontend actions to the backend functions.

Table 5.18: API endpoint method and description

Endpoint	Method	Description
<i>Authentication (@auth_bp)</i>		
/currentUser	GET	Retrieves the currently logged-in user's details via JWT token
/login	POST	Logs in a user and returns a JWT token for authenticated requests
/register	POST	Registers a new user account with username and password
/logout	POST	Invalidates the user's session
<i>Database Source (@data_source_bp)</i>		
/db-connection	POST	Accepts DB connection credentials and establishes connection
/db-status	GET	Returns the status of the current database connection
/db-tables	GET	Lists all tables available in the connected database
/db-table-preview/<table_name>	GET	Returns a preview of the specified table
<i>Language Model (@language_model_bp)</i>		



/llm-details	GET	Provides metadata about the currently used language model
<i>Query and LLM Workflow (@query_bp)</i>		
/query-input	POST	Submits the user's prompt to the agent system and triggers the multi-agent workflow
/state-details	GET	Returns debugging or intermediate states from the agent execution
/llm-details	GET	Returns LLM info used in the current query pipeline
/db-details	GET	Returns details about the connected database
/generated-visual	GET	Returns the HTML content or config for the visual generated by the agent
/generated-analysis	GET	Returns textual or LLM-generated analysis of the query result
/generated-graph	GET	Returns the backend-rendered graph component or object data

/save-visual	POST	Saves the generated visual
/saved-visuals	GET	Retrieves all visuals saved by the current user
/saved-visuals/<int:visual_id>	GET	Retrieves a specific saved visual by ID
<i>Visualization Tool Preferences (@tools_bp)</i>		
/tools-list	GET	Lists all available visualization tools supported by the system
/save-tools	POST	Stores the user's preferred visualization tools
/get-selected-tools	GET	Retrieves the user's currently selected tools from previous preference setting.

### 5.7.3. JSON Payload Structure

All payloads in the system use JSON schema. This ensures a standard format for data exchange. The system defines the structure of each payload clearly. The structure depends on the endpoint and method used. Table 5.19 shows the JSON payload structure. The payload follows the defined JSON schema. This ensures that all data sent and received by the system follows a standard format. The use of JSON schema helps the system validate data before processing it. It reduces the risk of errors caused by unexpected data type. It also

makes the system easier to maintain. Each endpoint and method has a fixed payload structure. This improves the reliability of communication between client and server. It also supports better security because the system only accepts data that matches the expected format.

Table 5.19: JSON payload structure

Endpoint	Method	Payload Structure
<i>Authentication (@auth_bp)</i>		
/currentUser	GET	{ "username": decoded["username"], "email": decoded["email"] }
/login	POST	{ "message": "Login Successful", "token": token }
/register	POST	{ "message": "Registration Successful", "token": token }
/logout	POST	{ "message": "Logged out successfully and DB disconnected." }
<i>Database Source (@data_source_bp)</i>		
/db-connection	POST	{ "message": "Connection successful" }
/db-status	GET	{ "status": "connected", "databaseName": connection_status["database_name"] }
/db-tables	GET	{ "databaseName": connection_status["database_name"], "tables": tables, "tablesCount": len(tables) }
/db-table- preview/<table_name>	GET	{ "error": "Error querying table", "details": str€ }
<i>Language Model (@language_model_bp)</i>		

/llm-details	GET	{ "model_name": model_name, "system_prompt": prompt }
Query and LLM Workflow (@query_bp)		
/query-input	POST	{"status": "success"}
/state-details	GET	{ "question": final_state["question"], "query": final_state["query"], "result": final_state["result"], "data": final_state["data"], "analysis": final_state["analysis"], "visualization": str(final_state["visualization"]) if final_state["visualization"] is not None else None, "routerCount": final_state["routerCount"], "retry": final_state["retry"], "SQLValidity": final_state["SQLValidity"] }
/llm-details	GET	{"model":str(state["model"])}
/db-details	GET	{"db": str(state["db"]), "schema":str(state["schema"]), }
/generated-visual	GET	state["visualization"], 200, {"Content-Type": "text/html"}
/generated-analysis	GET	state["analysis"], 200,
/generated-graph	GET	Response(graphVisual, mimetype="image/png")
/save-visual	POST	{"message": "Saved successfully"}
/saved-visuals	GET	jsonify([ "row": row[0], "prompt": row[1], "visualization": row[2], "analysis": row[3], ])

		<pre>                                 "timestamp": row[4] }                                 for row in rows                                 ]) </pre>
/saved-visuals/<int:visual_id>	GET	<pre> {     "id": row[0],     "prompt": row[1],     "visualization": row[2],     "analysis": row[3],     "timestamp": row[4] } </pre>
<i>Visualization Tool Preferences (@tools_bp)</i>		
/tools-list	GET	jsonify(t.get_tool_names())
/save-tools	POST	{"message": "Tools saved successfully"}
/get-selected-tools	GET	{"tools": selected_tools}

#### 5.7.4. Authentication Mechanisms

This system uses JWT for user authentication and session management. After a successful login, the backend generates and returns a JWT token that is valid for one (1) hour. This token must be included in the Authorization header for protected API requests. The format is:

Authorization: Bearer <jwt\_token>.

A token validation middleware checks if the token is present and valid before allowing access to sensitive routes like prompt submission and visual saving. If the token is missing, invalid, or expired, the server responds with a 401 Unauthorized status.

#### 5.8. Security Measures

This section explains the security measures of the system through input validation, encryption, HTTPS configuration, role-based access control, error handling and logging.

#### **5.8.1. Input Validation, Encryption, HTTPS Configuration**

The system checks all user inputs to empty prompt submission. User passwords are hashed before storing in the SQLite database to protect the passwords in case of a data breach. The system uses JWT tokens to manage user sessions. These tokens are encrypted to protect user session data. The system uses HTTPS for all communication. HTTPS ensures secure data transfer between client and server. This prevents attackers from intercepting sensitive information.

#### **5.8.2. Role-Based Access Control (RBAC)**

There is only one (1) role in the system. This role is the user. All users must log in to access the system. Only logged-in users can access protected routes. For example, only authenticated users can submit prompts. Only authenticated users can save visuals. It is not possible to access any part of the system without logging in. The system does not allow anonymous access. This approach ensures that all actions are linked to a verified user account.

### **5.8.3. Error Handling and Logging**

The system handles errors with clear messages. It avoids exposing system details. Logs are kept for debugging and monitoring.

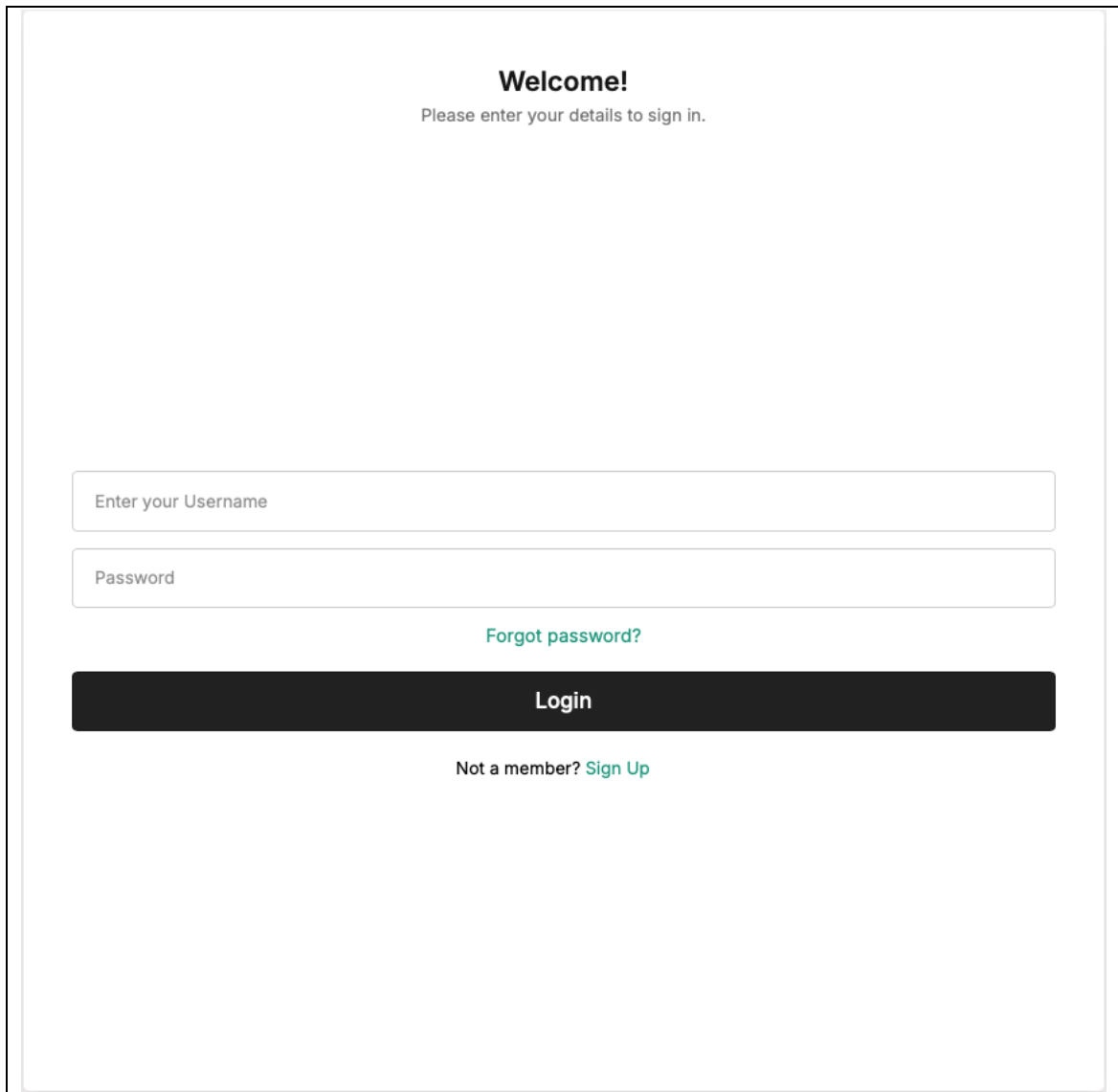
In case of agents not producing the structured output, the error would occur when Python tries to convert the output into a Python object. Hence, this system halts the visualization workflow and return the error to the user, indicating the agent is not able to produce the correct output based on current prompt. User would have to enter different prompt and submit again.

## 5.9. User Interface

This section displays the user interface of the system. Some UI changes are done based on the initial prototype UI because the color contrast between some of the generated graph and the UI does not match, hence the color code #FFFFFF is used as the primary website color to cater for this issue. Some UI is also simplified. The sidebar is changed to dropdowns as the actions to be done through the sidebar is too little, hence removing the need for a sidebar. The main part, the prompt submission field and the visual generation UI, remains.



### 5.9.1. Login Page



The image shows a login page layout. At the top, it says "Welcome!" followed by "Please enter your details to sign in." Below this are two input fields: "Enter your Username" and "Password". To the right of the password field is a link "Forgot password?". Below the input fields is a large black button labeled "Login". At the bottom, it says "Not a member? Sign Up" with "Sign Up" being a link.

**Welcome!**  
Please enter your details to sign in.

Enter your Username

Password

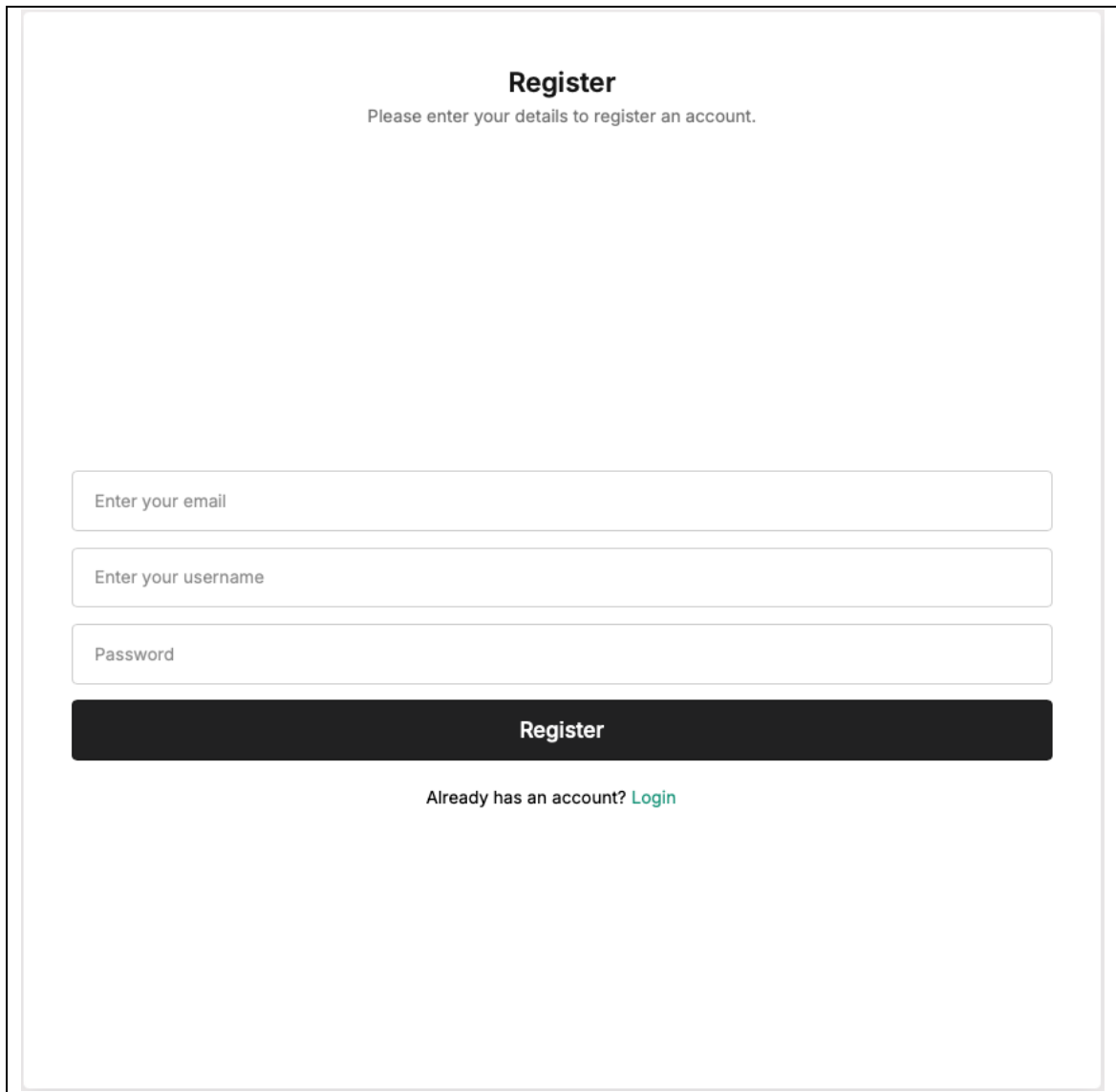
[Forgot password?](#)

**Login**

Not a member? [Sign Up](#)

Figure 5.3: Login page

### 5.9.2. Register Page



The registration page features a clean, minimalist design. At the top, the title "Register" is centered in a bold, black font, followed by a subtitle "Please enter your details to register an account." in a smaller, regular font. Below this, there are three stacked input fields with light gray borders and rounded corners. The first field is labeled "Enter your email", the second "Enter your username", and the third "Password". Each field contains a small, light gray placeholder icon. Below the input fields is a prominent, dark gray "Register" button with white text. At the bottom, a link "Already has an account? Login" is displayed in a smaller font, with "Login" in a teal color.

**Register**

Please enter your details to register an account.

Enter your email

Enter your username

Password

**Register**

Already has an account? [Login](#)

Figure 5.4: Registration page

### 5.9.3. Main Menu

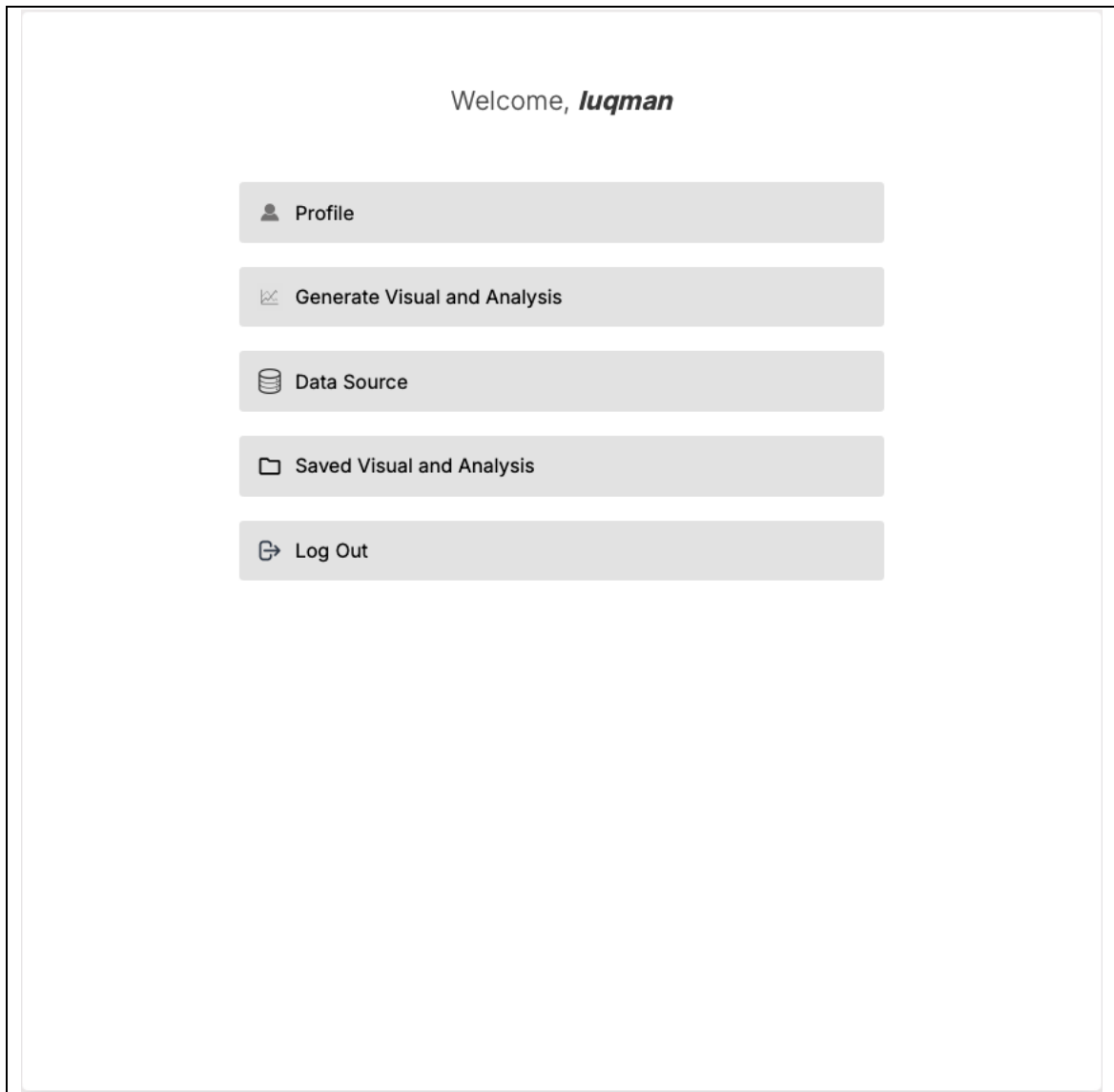
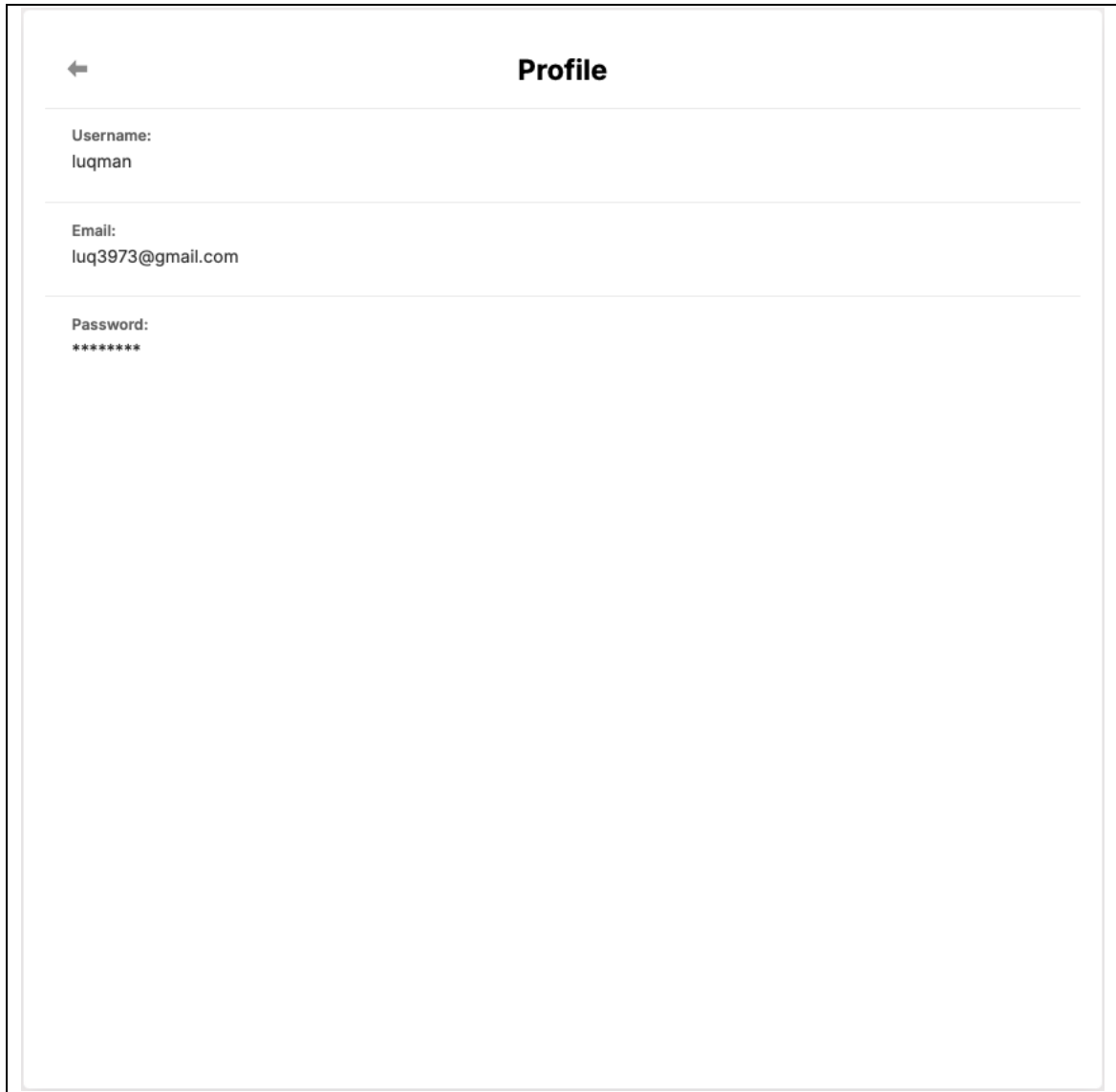


Figure 5.5: Main menu page

### 5.9.4. Profile Page



The image shows a web interface for a profile page. At the top, there is a back arrow icon and the title "Profile". Below the title, there are three sections separated by horizontal lines. The first section is labeled "Username:" and contains the text "luqman". The second section is labeled "Email:" and contains the text "luq3973@gmail.com". The third section is labeled "Password:" and contains a series of asterisks "\*\*\*\*\*".

Figure 5.6: Profile page

### 5.9.5. Data Source Credentials Page



The screenshot displays a web interface titled "Connect to your Database". At the top left, there is a back arrow icon. Below the title, there are four stacked input fields with placeholder text: "Username", "Host", "Database", and "Password". At the bottom of the form is a dark button labeled "Verify Connection".

Figure 5.7: Data source credentials page

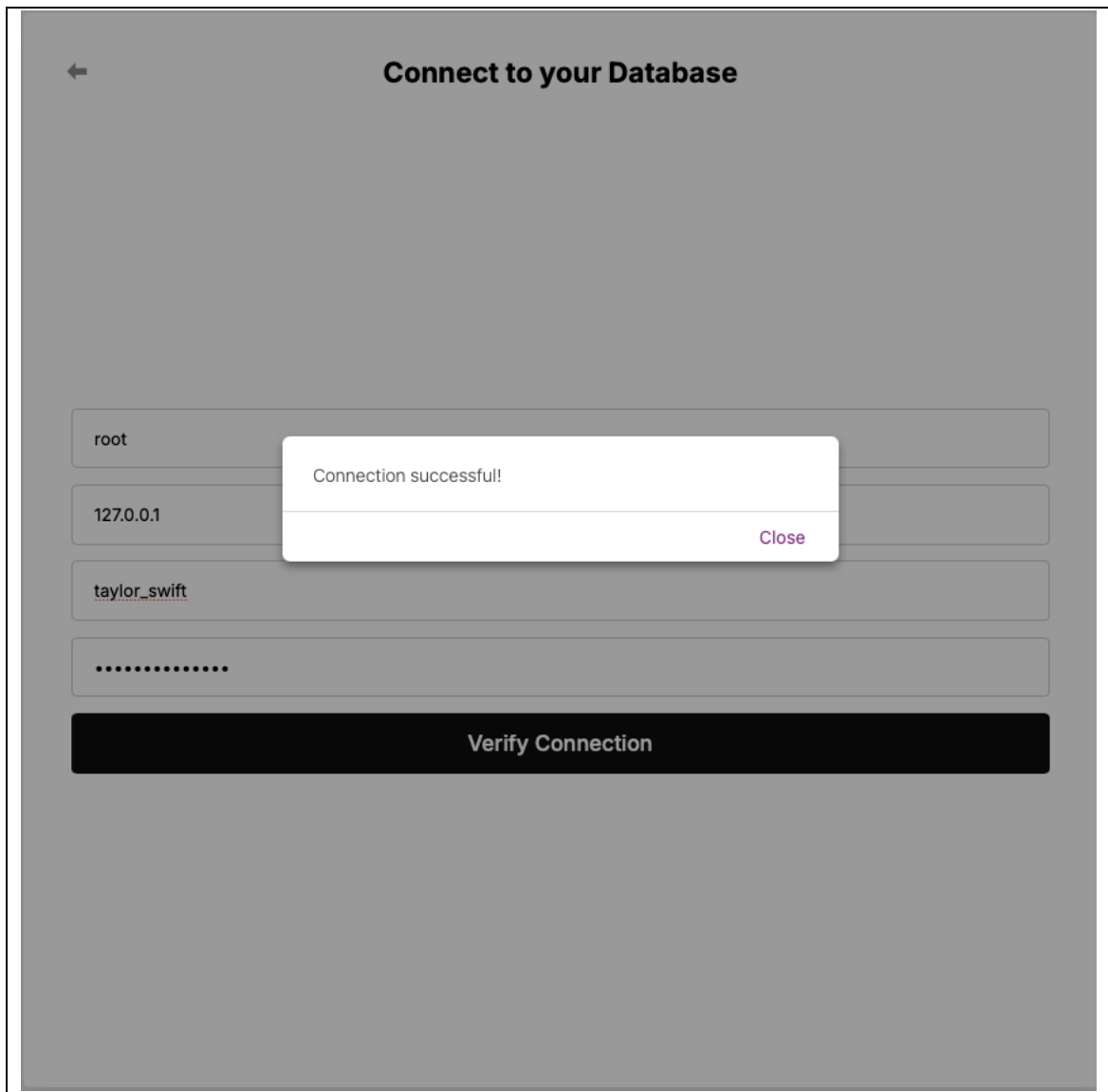


Figure 5.8: Data source credentials page after successful connection

### 5.9.6. Data Preview Page

←

Database Preview

**Basic Info**

Database Name:  
taylor\_swift

Number of Tables:  
4

Tables:  
album\_popularity, albums, lyrics, tracks

**Select Table**

-- Choose a Table --

**Schema Overview**

No schema info available.

**Table Preview (First 5 Rows)**

No preview data available.

**Basic Stats**

Row Count:

Connect to Database

Figure 5.9: Database preview page

←

Database Preview

Basic Info

Database Name:  
taylor\_swift

Number of Tables:  
4

Tables:  
album\_popularity, albums, lyrics, tracks

Select Table

tracks

Schema Overview

- track\_id (VARCHAR(255))
- track\_name (VARCHAR(255))
- album\_id (INTEGER)
- track\_number (INTEGER)
- artist (VARCHAR(255))
- featuring (VARCHAR(255))
- danceability (FLOAT)
- energy (FLOAT)
- key (INTEGER)
- loudness (FLOAT)
- mode (INTEGER)
- speechiness (FLOAT)
- acousticness (FLOAT)
- instrumentalness (FLOAT)
- liveness (FLOAT)
- valence (FLOAT)
- tempo (FLOAT)
- time\_signature (INTEGER)
- duration\_ms (INTEGER)
- explicit (TINYINT)
- key\_name (VARCHAR(255))
- mode\_name (VARCHAR(255))
- key\_mode (VARCHAR(255))

Table Preview (First 5 Rows)

track_id	track_name	album_id	track_number	artist	featuring	danceability	energy	key	loudness	n
0101	Tim	1	1	Taylor		0.58	0.491	0	-6.462	1

Figure 5.10: Database preview page after selecting table



### 5.9.7. Prompt Field Page



The screenshot shows a web interface titled "Enter Your Query". At the top left is a back arrow icon. Below the title is a large text input field with the placeholder text "Create a graph for ...". Below the input field is a dark blue "Submit" button. At the bottom, there are three light gray buttons: "Choose Tools", "DB Details", and "LLM Details".

Figure 5.11: Prompt field page



## Enter Your Query

Choose Tools

### Select Tools

  
scatterPlot

  
barChart

  
choropleth

  
pieChart

  
table

DB Details

LLM Details

Figure 5.12: Prompt field page with tool options visible

Prepared by: LUQMAN HAKIM BIN NOORAZMI

CPT6324 Project 2

2510 Term

← **Enter Your Query**

Create a graph for ...

**Submit**

Choose Tools

DB Details

**Database Connection**

Connected: <langchain\_community.utilities.sql\_database.SQLDatabase object at 0x11bcfa610>

LLM Details

Figure 5.13: Prompt field page with database connection status visible

← **Enter Your Query**

Create a graph for ...

**Submit**

Choose Tools

DB Details

LLM Details

**Large Language Model**

llama-3.3-70b-versatile

Figure 5.14: Prompt field page with model name visible

### 5.9.8. Generated Visual Page

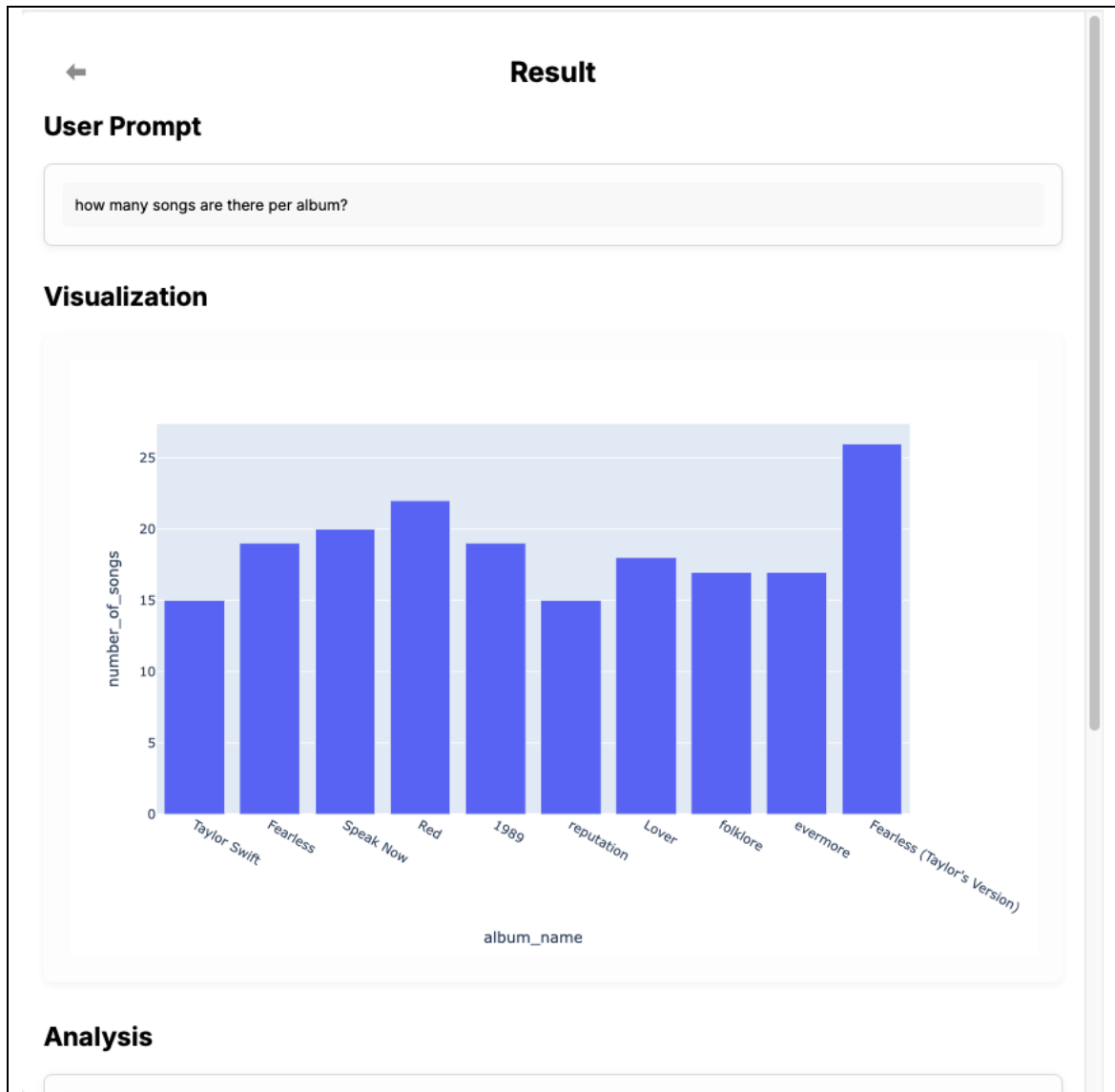


Figure 5.15: Generated visual page

### 5.9.9. Agents Output Page

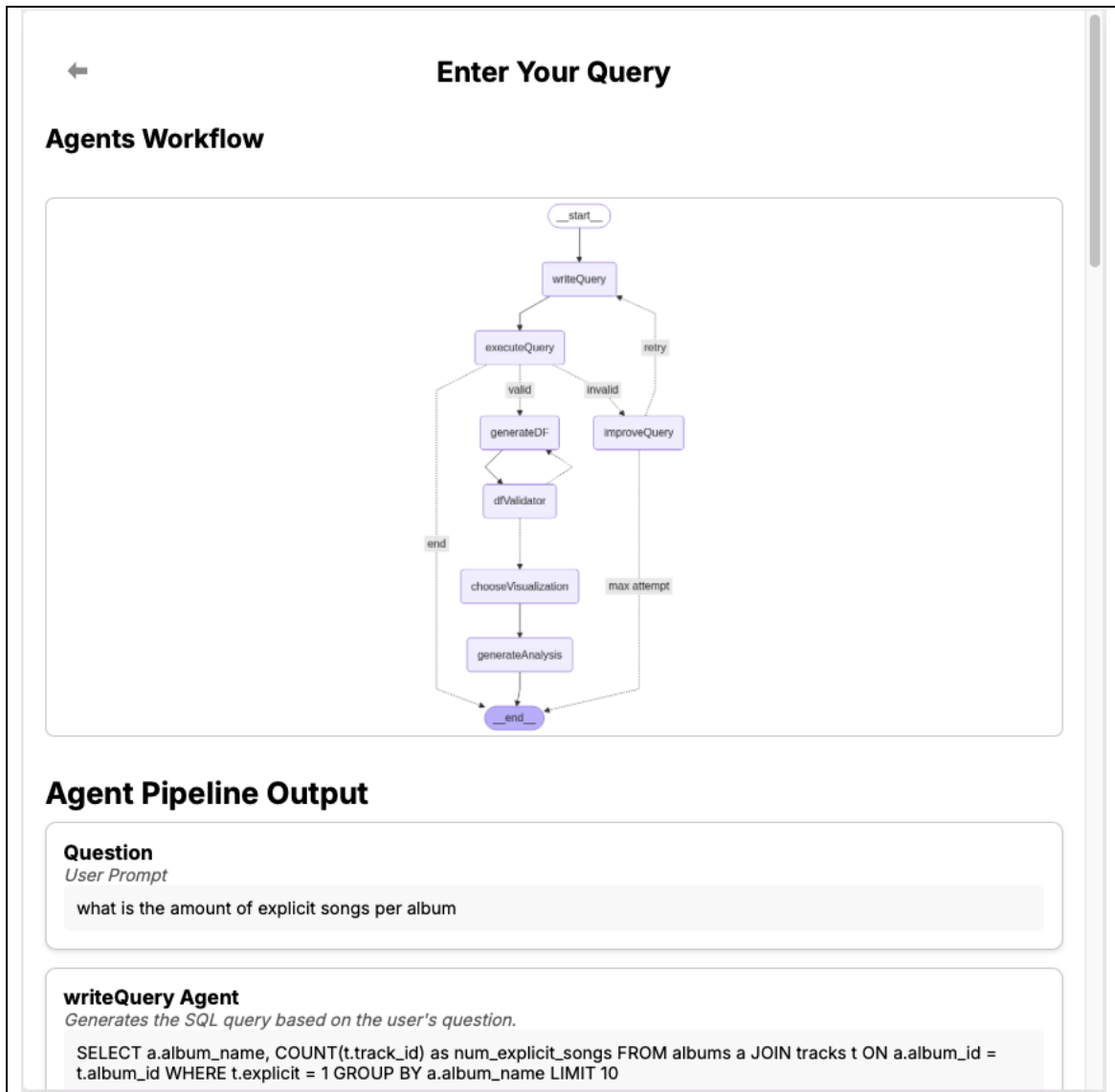


Figure 5.16: Agent output page

### 5.9.10. Saved Visual

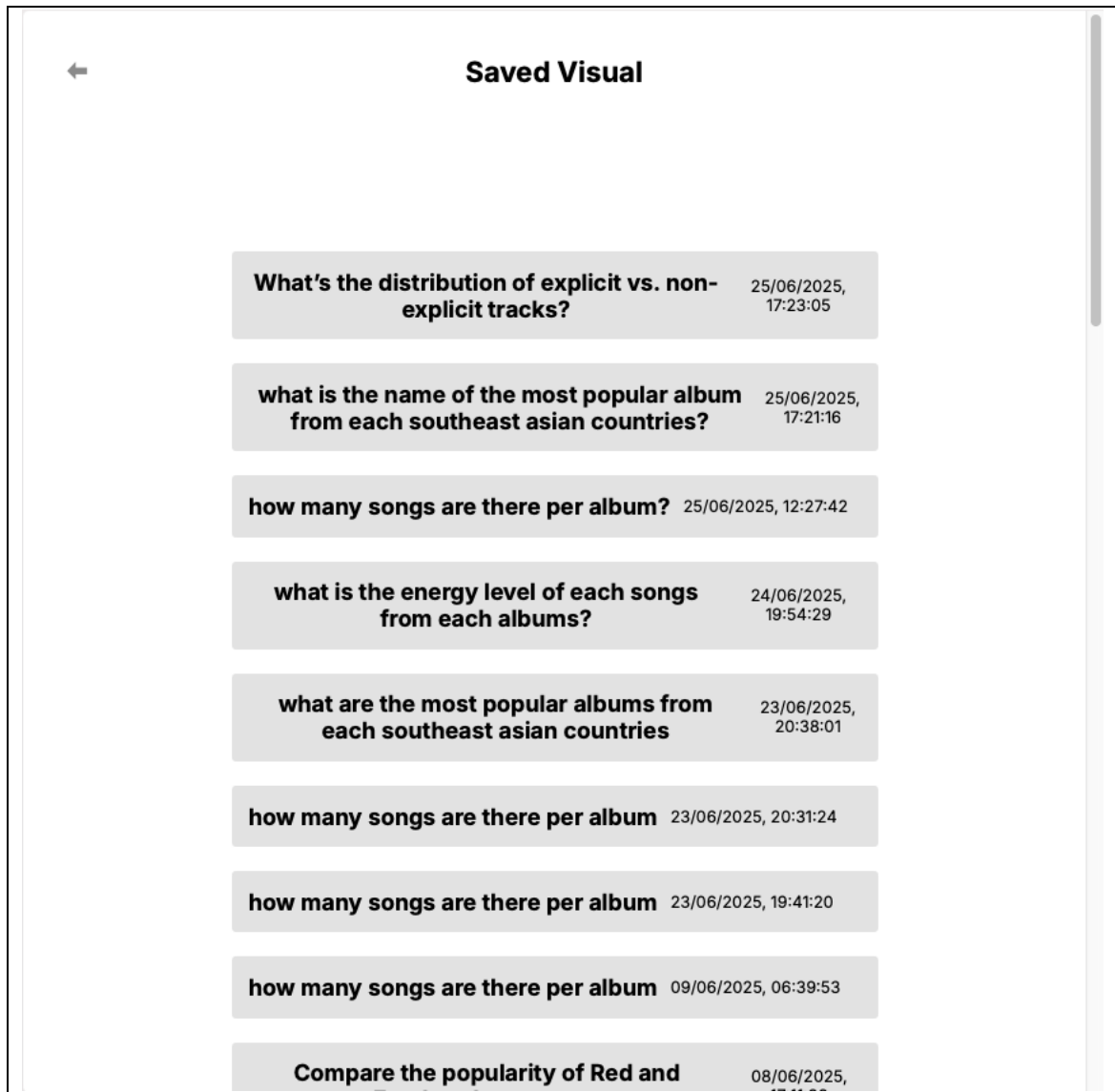


Figure 5.17: Saved visual list page

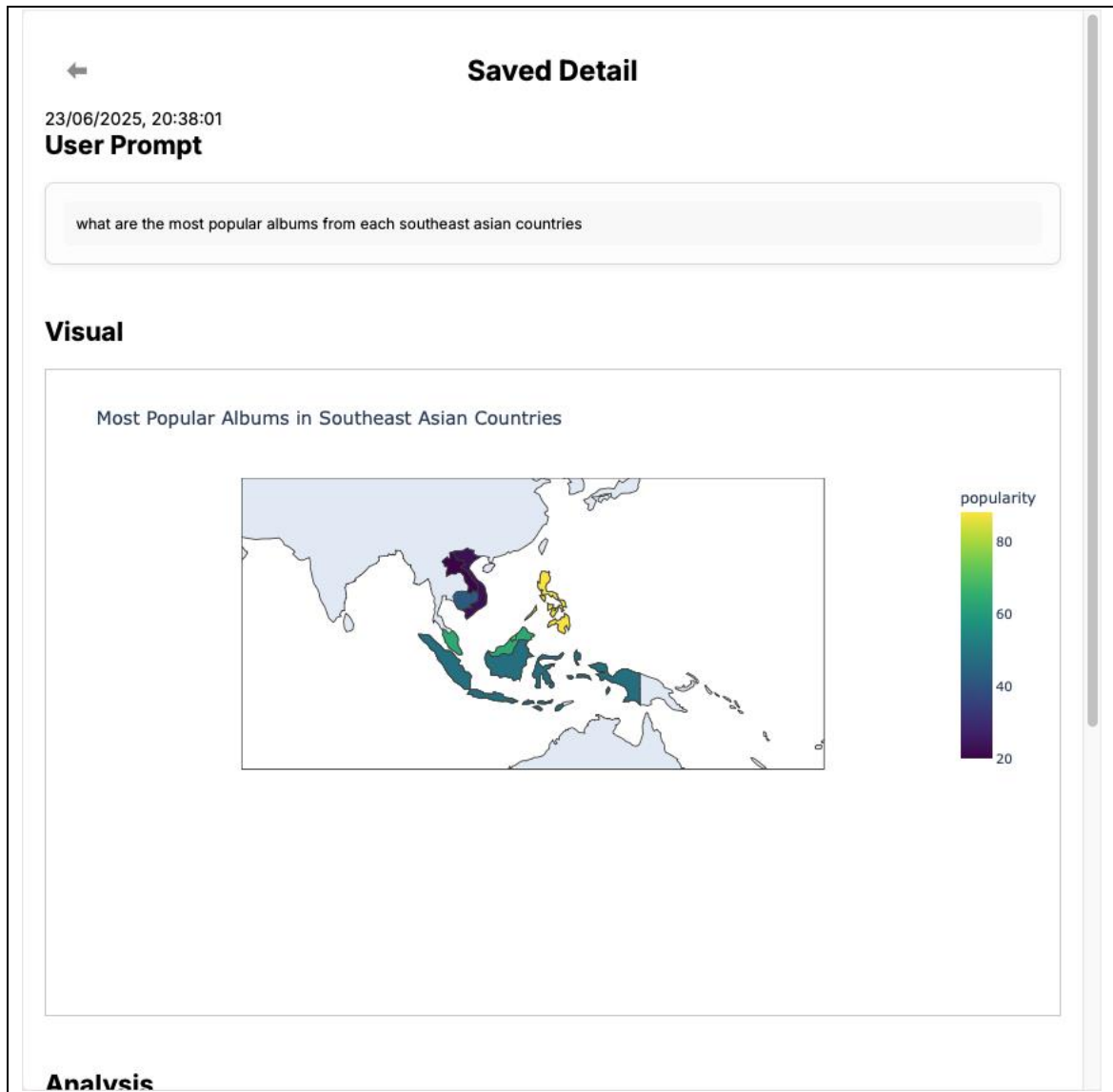


Figure 5.18: Detailed saved visual page



## 5.10. Challenges Encountered and Solutions

The project faced several challenges during development. There was limited token usage per day, which required reducing the length of prompts for each agent. Some agent outputs did not follow the expected structure, so example outputs were added to the prompts. There was a lack of references for software using the same framework. To overcome this, other unrelated software with the same framework was used as a reference. The team was also unfamiliar with the frontend and backend frameworks, so learning took place alongside development. A key issue was that agent outputs did not trigger errors immediately when they deviated from the expected format. Errors only appeared during Python operations that depended on precise output, making debugging difficult. To address this, try blocks were added to catch failures and notify the user.

### 5.11. Summary

In summary, the software was able to be developed exactly as planned out during the design phase despite the challenges faced. The developed software represents a fully working prototype of automated data visualization using multi agent framework and reflects the designed system during the analysis and design phase.

Future improvements could include features not covered within the scope of this project such as support for non-MySQL databases, natural language conversation Interface, mobile or responsive design, user registration and profile management, exporting visualizations, and concurrent multi-user collaboration

## Chapter 6: Testing

### 6.1. Unit Testing

During unit testing, each module is tested individually. Each module must produce the expected outcome to be considered as pass.

#### 6.1.1. User authentication

Table 6.1: User authentication test plan

Test Plan				
Module	No	Test ID	Function	Test Date
User Authentication	1	TC_01_01	Create Account	20/06/2026
	2	TC_01_02	Login	20/06/2026
	3	TC_01_03	User Session	20/06/2026
	4	TC_01_04	Logout	20/06/2026

Table 6.2: User authentication test data

Module	Test Case		Relevant Test Data
User Authentication	TC_01_01	Create Account	{ "username": "testuser", "email": "testuser@example.com", "password": "123123" }
	TC_01_02	Login	{ "email": "testuser@example.com", }

			"password": "Test@1234" }
	TC_01_03	User Session	Authorization: Bearer <valid_token>
	TC_01_04	Logout	Authorization: Bearer <valid_token>

Table 6.3: Create a new account test case

Test Case	Test Case ID	TC_01_01
	Description	Create a new account
	Precondition	No active session and navigated to the register page
	Postcondition	User is registered and logged in
Test Script	Test Steps	1. Navigate to the register page 2. Fill in the field with user credentials 3. Submit registration
	Expected Output	User is registered and logged in
	Actual Output	User is registered and logged in

Table 6.4: Login with valid credentials test case

Test Case	Test Case ID	TC_01_02
	Description	Login with valid credentials
	Precondition	Registered and navigated to the login page
	Postcondition	User is logged in with active session and token
Test Script	Test Steps	1. Navigate to the login page 2. Enter valid email and password 3. Click the Login button
	Expected Output	User is logged in and navigated to the home page
	Actual Output	User is logged in and navigated to the home page with access to website features

Table 6.5: Check active user session test case

Test Case	Test Case ID	TC_01_03
	Description	Check active user session
	Precondition	Logged in
	Postcondition	JSON response includes user information
Test Script	Test Steps	1. Access /currentUser endpoint with JWT in Authorization header     2. Validate user details
	Expected Output	JSON response includes user information
	Actual Output	<pre>{   "username": "testuser",   "email": "testuser@example.com" }</pre>

Table 6.6: Logout user and invalidate session test case

Test Case	Test Case ID	TC_01_04
	Description	Logout user and invalidate session
	Precondition	Logged in
	Postcondition	Session deleted
Test Script	Test Steps	1. Login successfully     2. Click Logout button or send POST to /logout     3. Try accessing a protected route
	Expected Output	JSON response: logout success
	Actual Output	User is notified and logged out and redirected to the login page

### 6.1.2. Database connection

Table 6.7: Database connection test plan

Test Plan				
Module	No	Test ID	Function	Test Date
Database Connection	1	TC_02_01	Connect to database	20/06/2026

Table 6.8: Connect to database test data

Module	Test Case		Relevant Test Data
Database Connection	TC_02_01	Connect to database	username: root host: 127.0.0.1 database: taylor_swift password (valid): *****

Table 6.9: Connect to database test case

Test Case	Test Case ID	TC_02_01
	Description	Connect to database
	Precondition	Logged in
	Postcondition	Connected to database
Test Script	Test Steps	1. Navigate to 'Data Source' 2. Click 'Connect to Database' 3. Enter database credentials 4. Submit
	Expected Output	Connection successful
	Actual Output	Received 'Connection Successful' alert and redirected to the data preview page with data from database loaded

### 6.1.3. Prompt handling

Table 6.10: Prompt handling test plan

Test Plan				
Module	No	Test ID	Function	Test Date
Prompt Handling	1	TC_03_01	Submit Prompt	20/06/2026

Table 6.11: Submit prompt test data

Module	Test Case		Relevant Test Data
Prompt Handling	TC_02_01	Submit Prompt	prompt: how many songs are there per album

Table 6.12: Submit prompt test case

Test Case	Test Case ID	TC_03_01
	Description	Submit Prompt
	Precondition	Logged in and connected to the database
	Postcondition	The website processes the prompt
Test Script	Test Steps	1. Navigate to 'Generate Visual and Analysis' 2. Enter a prompt 3. Submit the prompt
	Expected Output	The website notifies the user
	Actual Output	The website displays a loading screen with the word 'processing'

#### 6.1.4. LLM agent

Table 6.13: LLM agent test plan

Test Plan				
Module	No	Test ID	Function	Test Date
LLM Agent	1	TC_04_01	Agent Workflow	21/06/2026

Table 6.14: LLM agent test data

Module	Test Case		Relevant Test Data
LLM Agent	TC_04_01	Agent Workflow	prompt: how many songs are there per album

Table 6.15: LLM agent test case

Test Case	Test Case ID	TC_04_01
	Description	Agent Workflow
	Precondition	Logged in and connected to the database and submitted a prompt
	Postcondition	Visualization Generated
Test Script	Test Steps	1. Navigate to 'Generate Visual and Analysis' 2. Enter a prompt 3. Submit a prompt
	Expected Output	Visualization and analysis are generated based on the prompt
	Actual Output	Visualization and analysis are generated based on the prompt

### 6.1.5. Visualization tools

Table 6.16: Visualization tools test plan

Test Plan				
Module	No	Test ID	Function	Test Date
Visualization tools	1	TC_05_01	Choose Tools	21/06/2026
	2	TC_05_02	Choose no Tools	21/06/2025

Table 6.17: Choose tools test data

Module	Test Case		Relevant Test Data
Visualization tools	TC_05_01	Choose Tools	Selected Tools
	TC_05_02	Choose no Tools	No Selection

Table 6.18: Choose tools test case

Test Case	Test Case ID	TC_05_01
	Description	Choose Tools
	Precondition	Logged in and connected to the database
	Postcondition	Tools selection saved



Test Script	Test Steps	1. Navigate to 'Generate Visual and Analysis'     2. Click on 'Choose Tools'     3. Select one or more tools card     4. Save selection
	Expected Output	Tools selection saved
	Actual Output	Tools selection saved with 'Tools Saved Successfully' message

Test Case	Test Case ID	TC_05_02
	Description	Choose no Tools
	Precondition	Logged in and connected to the database
	Postcondition	Tools selection saved
Test Script	Test Steps	1. Navigate to 'Generate Visual and Analysis'     2. Click on 'Choose Tools'     3. Select no tools card     4. Unselect any selected tools card     5. Save selection
	Expected Output	Tools selection not saved
	Actual Output	Tools selection not saved with 'Invalid Tools List' message

### 6.1.6. Frontend UI

Table 6.19: Frontend UI test plan

Test Plan				
Module	No	Test ID	Function	Test Date
Frontend UI	1	TC_06_01	Full workflow	21/06/2026

Table 6.20: Full workflow test data

Module	Test Case		Relevant Test Data
Frontend UI	TC_06_01	Full workflow	user credentials, database credentials, prompt

Table 6.21: Full workflow test case

Test Case	Test Case ID	TC_06_01
	Description	Full workflow
	Precondition	Logged in
	Postcondition	All navigation leads to the correct page
Test Script	Test Steps	1. Open the web app     2. Log in using correct username and password     3. Connect to a MySQL database using correct credentials     4. Enter a valid prompt into the input field and submit     5. Wait for agents to complete processing     6. View output from each agent     7. View the generated visual and analysis     8. Click on “Choose Tools”, select tools, and save     9. Navigate to “Saved Visuals” to view saved visualizations
	Expected Output	All actions redirect to the correct page
	Actual Output	All functions performed as expected and navigated to the correct page with the correct data

## 6.2. Integration Testing

During integration testing, modules are tested together. Each test must meet its expected results to be considered pass.

Table 6.22: Integration testing results

#	Test Case	Units Integrated	Test to execute test cases	Expected results	Actual results
1	Login with connect to database	1. User Authentication 2. Database Connection	Login and connect to database	Session created and connection to database established	Session created and connection to database established
2	Enter prompt with agent output	1. Prompt Handling 2. LLM Agent	Enter a prompt and assess the generated visual	Visual is generated	Visual is generated
3	Enter prompt with failed agent output	1. Prompt handling 2. LLM Agent	Enter an unrelated prompt and assess the website response	The visual is not generated and error occurs	The visual is not generated and user is notified
4	Enter prompt with agent output and unrelated tool	1. Prompt handling 2. LLM Agent 3. Visualization Tools	Enter an unrelated prompt based on the selected tools and assess the website response	The visual is not generated and error occurs	The website displays the visual using the selected prompt, even though the prompt expect other tool

5	Save and view visual	1. UI Frontend 2. Prompt Handling 3. LLM Agent	Send prompt, wait for visual generation and save visual and view the saved visual	Visual is saved and can be viewed in the saved visual page	Visual is saved and can be viewed in the saved visual page
6	View agent output	1. UI Frontend 2. Prompt Handling 3. LLM Agent	Enter a prompt and assess the generated visual, then view the agent output	The agent output page is displayed with the actual output from the agent	The agent output page is displayed with the actual output from the agent

### 6.3. Usability Testing

For usability testing, subjects are selected from two (2) target user: student and data scientist. They are required to execute these tasks and give feedback based on the executed tasks. They will also be observed while executing these tasks.

Table 6.23: Tasks and steps to be executed for usability testing

Task	Procedure / Steps
Login	1. Enter user credentials     2. Submit user credentials
Locate main menu	1. Login to the website     2. Assess the main menu page
Navigate to the database credentials field page	1. Navigate to the main menu     2. Navigate to the data source page     3. Click the connect to database button
Enter database credentials	1. Navigate to the database credentials field page     2. Enter database credentials     3. Submit database credentials
View data	1. Make connection to the database     2. View the data preview page
Navigate to the prompt field page	1. Navigate to the main menu

	2. Navigate to the generate visual page
Choose tools	1. Navigate to the generate visual page    2. Click on the choose tools dropdown
Enter prompt	1. Navigate to the generate visual page    2. Enter the prompt on the prompt field    3. Submit the prompt
View Visual	1. Navigate to the generate visual page    2. Enter the prompt on the prompt field    3. Submit the prompt    4. Assess the displayed visual
View agent output	1. Navigate to the generate visual page    2. Enter the prompt on the prompt field    3. Submit the prompt    4. View the generated visual    5. Click on view agent output button at the bottom of the page

Save visual	1. Navigate to the generate visual page    2. Enter the prompt on the prompt field    3. Submit the prompt    4. View the generated visual    5. Click on save visual button at the bottom of the page
View saved visual	1. Navigate to the main menu    2. Navigate to the view saved visual page    3. Choose the saved visual listed from the page    4. Assess the saved visual and analysis

Table 6.24: Usability test 1

Date/Time	Task	Subject	Time	Observation	Status	Conclusion
23/06/2025	Login	Farid (Data Science Student)	7 seconds	The subject was able to enter their user credentials and login into the system without supervision.	Success	The login steps are easy to follow and the UI is intuitive and easy to use.
	Locate main menu		3 seconds	The subject was able to locate the main menu and the menu choices immediately.	Success	The UI is intuitive and easy to use.
	Navigate to the database credentials field page		14 seconds	The subject was able to find the database credentials form after	Moderat Success	The navigation for the database credentials is not straightforward



				navigating to the 'Data Source' page and click the 'Connect to Database' button		, hence users have to somewhat guess the location of the form, though it should be obvious as there is only one navigation that leads user to the database-related page.
	Enter database credentials		20 seconds	The subject was able to enter their database credentials and make the database connection the system without supervision.	Success	The database connection steps are easy to follow and the UI is intuitive and easy to use.
	View data		8 seconds	The subject was redirected	Success	The redirection to the data viewer page

				immediately to the data preview page.		after a successful database connection has been made is very intuitive for the user as the user gets to explore the data immediately and confirms the connection has been made.
	Navigate to the prompt field page		5 seconds	The subject had to go back to the main menu before navigating to the prompt field page.	Success	The navigation could be improved to allow users jump to different sections and pages without having to navigate back to the main menu.

	Choose tools		13 seconds	The subject was able to select from the choices of tools.	Success	The tools selection steps are easy to follow and the UI is intuitive and easy to use.
	Enter prompt		13 seconds	The subject enters a prompt and submits the prompt without supervision.	Success	The prompt submission steps are easy to follow and the UI is intuitive and easy to use.
	View Visual		17 seconds	The subject views the generated visual and analysis without supervision.	Success	The visual view steps are easy to follow and the UI is intuitive and easy to use as the visual is displayed to the user immediately after the prompt submission.

	View agent output		19 seconds	The subject views the agent output without supervision.	Success	The agent output view steps are easy to follow and the UI is intuitive and easy to use.
	Save visual		12 seconds	The subject navigates back to the visual output page and click on the save visual button with ease and without supervision.	Success	The save visual steps are easy to follow and the UI is intuitive and easy to use.
	View saved visual		10 seconds	The subject navigates back to the main menu and navigates to the saved visual page.	Success	The view saved visual steps are easy to follow and the UI is intuitive and easy to use.

Table 6.25: Usability test 2

Date/Time	Task	Subject	Time	Observation	Status	Conclusion
23/06/2025	Login	Asyer (Data Science Student)	9 seconds	The subject was able to enter their user credentials and login into the system without supervision.	Success	The login steps are easy to follow and the UI is intuitive and easy to use.
	Locate main menu		5 seconds	The subject was able to locate the main menu and the menu choices immediately.	Success	The UI is intuitive and easy to use.
	Navigate to the database credentials field page		10 seconds	The subject was able to find the database credentials form after navigating to the 'Data Source' page	Moderate Success	The navigation for the database credentials is not straightforward, hence users have to somewhat guess the

				and click the 'Connect to Database' button		location of the form, though it should be obvious as there is only one navigation that leads user to the database-related page.
	Enter database credentials		18 seconds	The subject was able to enter their database credentials and make the database connection on the system without supervision.	Success	The database connection steps are easy to follow and the UI is intuitive and easy to use.
	View data		3 seconds	The subject was redirected immediately to the data	Success	The redirection to the data viewer page after a successful

				preview page.		ul database connection has been made is very intuitive for the user as the user gets to explore the data immediately and confirms the connection has been made.
	Navigate to the prompt field page		3 seconds	The subject was able to navigate to the prompt field page with ease and without navigation	Success	The prompt field navigation steps are easy to follow and the UI is intuitive and easy to use.
	Choose tools		8 seconds	The subject was able to select from the choices	Success	The tools selection steps are easy to follow

				of tools. The subject chooses all the tools.		and the UI is intuitive and easy to use.
	Enter prompt		10 seconds	The subject enters a prompt and submits the prompt without supervision.	Success	The prompt submission steps are easy to follow and the UI is intuitive and easy to use.
	View Visual		17 seconds	The subject views the generated visual and analysis without supervision.	Success	The visual view steps are easy to follow and the UI is intuitive and easy to use as the visual is displayed to the user immediately after the prompt submission.
	View agent output		19 seconds	The subject views	Success	The agent output



				the agent output without supervision.		view steps are easy to follow and the UI is intuitive and easy to use.
	Save visual		12 seconds	The subject navigates back to the visual output page and click on the save visual button with ease and without supervision.	Success	The save visual steps are easy to follow and the UI is intuitive and easy to use.
	View saved visual		10 seconds	The subject navigates back to the main menu and navigates to the saved visual page. Subject seems to have	Success	The view saved visual steps are easy to follow and the UI is intuitive and easy to use.

				used to the navigatio nal structure of the website.		
--	--	--	--	-----------------------------------------------------------------------	--	--

Table 6.26: Usability test 3

Date/Time	Task	Subject	Time	Observation	Status	Conclusion
26/06/2025	Login	Azmi (Data Scientist)	8 seconds	The subject was able to enter their user credentials and login into the system without supervision.	Success	The login steps are easy to follow and the UI is intuitive and easy to use.
	Locate main menu		5 seconds	The subject was able to locate the main menu and the menu choices immediately.	Success	The UI is intuitive and easy to use.
	Navigate to the database credentials field page		10 seconds	The subject was able to find the database credentials form easily without supervision.	Moderate	The database credentials navigation steps are easy to follow and the UI is intuitive and easy to use.

	Enter database credentials		18 seconds	The subject was able to enter their database credentials and make the database connection on the system without supervision.	Success	The database connection steps are easy to follow and the UI is intuitive and easy to use.
	View data		24 seconds	The subject was redirected immediately to the data preview page. This particular subject explores the data by choosing different tables	Success	The redirection to the data viewer page after a successful database connection has been made is very intuitive for the user as the user gets to explore the data immediately and confirms the

						connecti on has been made. The data preview that is shown after selecting the table is very useful for users without prior knowled ge of the database structure .
	Navigate to the prompt field page		6 secon ds	The subject was able to navigate to the prompt field page without supervisi on.	Success	The prompt field navigatio n steps are easy to follow and the UI is intuitive and easy to use.
	Choose tools		19 secon ds	The subject was able to select from the choices of tools. Subject tries	Success	The tools selection steps are easy to follow and the UI is intuitive

				selecting and unselecting multiple tools together and trying different combination.		and easy to use.
	Enter prompt		8 seconds	The subject enters a prompt and submits the prompt without supervision.	Success	The prompt submission steps are easy to follow and the UI is intuitive and easy to use.
	View Visual		32 seconds	The subject views the generated visual and analysis without supervision. The subject hovers over the generated visual and use the built	Success	The visual view steps are easy to follow and the UI is intuitive and easy to use as the visual is displayed to the user immediately after the

				in tools by plotly.		prompt submission. The Plotly interface is very intuitive and promotes interaction between user and the interface.
	View agent output		35 seconds	The subject views the agent output without supervision. Subject zooms in to the agent workflow image for a better view.	Success	The agent output view steps are easy to follow and the UI is intuitive and easy to use. The placement of the agent workflow image is a good implementation for users with technical knowledge.

	Save visual		8 seconds	The subject navigates back to the visual output page and click on the save visual button with ease and without supervision.	Success	The save visual steps are easy to follow and the UI is intuitive and easy to use.
	View saved visual		15 seconds	The subject navigates back to the main menu and navigates to the saved visual page.	Success	The view saved visual steps are easy to follow and the UI is intuitive and easy to use.



#### 6.4. Acceptance Testing

For acceptance testing, subjects are selected from two (2) target user: student and data scientist.

Table 6.27: Acceptance test 1

Tester	Farid (Data Science Student)
Test Date	23/06/2025
Prototype developer	Luqman Hakim bin Noorazmi
Test Objective	Submit a prompt and view generated visual
Potential Test Inputs	1. Keyboard input 2. Button click
Expected Test Outputs	Visual of the data and the analysis is displayed to the user
Test Procedures	1. Navigate to the 'Generate Visual and Analysis' page 2. Enter a prompt 3. Submit the prompt
Actual Test Results	The prompt is submitted and the visual is successfully generated based on user prompt
Comments by User	Subject finds the developed system matches the expectation of the project title and requirements.

Table 6.28: Acceptance test 2

Tester	Asyer (Data Science Student)
Test Date	23/06/2025
Prototype developer	Luqman Hakim bin Noorazmi
Test Objective	Submit a prompt and view generated visual
Potential Test Inputs	1. Keyboard input 2. Button click
Expected Test Outputs	Visual of the data and the analysis is displayed to the user
Test Procedures	1. Navigate to the 'Generate Visual and Analysis' page 2. Enter a prompt 3. Submit the prompt

Actual Test Results	The prompt is submitted and the visual is successfully generated based on user prompt
Comments by User	The subject finds the system easy to use.

Table 6.29: Acceptance test 3

Tester	Azmi (Data Scientist)
Test Date	26/06/2025
Prototype developer	Luqman Hakim bin Noorazmi
Test Objective	Submit a prompt and view generated visual
Potential Test Inputs	3. Keyboard input 4. Button click
Expected Test Outputs	Visual of the data and the analysis is displayed to the user
Test Procedures	4. Navigate to the 'Generate Visual and Analysis' page 5. Enter a prompt 6. Submit the prompt
Actual Test Results	The prompt is submitted and the visual is successfully generated based on user prompt
Comments by User	The subject is satisfied with the flow of the system, mostly from submitting prompt to viewing agent output.

## Chapter 7: Conclusion

### 7.1. Future Works and Recommendation

There are several ways this developed system could be expanded.

One way this system could be expanded is through the token management. The use of open-source model limits the usage of the system to 100,000 token per day which results in the limit of fetching data from the database to only 10 entries. By allowing for more token usage, it will be possible to add more agents within the workflow for adding more validation to the agent output, add a router agent to handle the routing of the workflow, and curate richer and better prompt with possibly better instruction. More token usage also means the limit of fetched database entries could be expanded. This would also allow for more complex graphs and charts to be visualized now that the amount of data has massively increased. In short, the token limitation is the biggest obstacle that holds this software from expanding further, hence it is suggested to overcome this problem to allow more expansion to be made to the current system

The software could be modified to cater for different language model. Incorporating different model would require the prompt to be adjusted to achieve the same output as the current system.

In terms of visualization, more graph and charts could be added to expand the visual scope of the system. Other visual libraries than Plotly could also be considered as tool choices.

This system could be developed further to support different environment such as mobile and desktop app.

Lastly, more use cases could be added. This project could be expanded to include more features since most of features from existing data visualization tools that has been developed in this system is automated, other use cases could be included to allow user to interact more with the system for anything other than strictly visual generation.

## 7.2. Conclusion

In conclusion, finishing this project has expanded my understanding of how technology may be used for good. Given the existence of well-known data visualization tools, it was initially difficult to decide which features and functionalities should be included to make the project stand out. In order to improve usability, it was decided to incorporate Large Language Models (LLMs) into the visualization tool following a full background investigation. After the planning phase was finished, the development of the system was smoothly implemented, which highlights the effectiveness of good software planning. Many new frameworks and resources were utilized in developing the system, alongside some challenges that arise from the development phase, which were able to be solved effectively.

The result from the testing phase indicates the system meets its expectation. The implementation of LLM agents into a data visualization tool is considered as a success. Moving forward, this system could be expanded to cater for features not covered within the implementation scope.

Throughout this project, I gained valuable experience. I learned how to work with new technologies and frameworks. I developed skills in both frontend and backend programming. I also became more confident in problem-solving and planning. This project has helped me grow as a developer and as a person. I am

now more open to exploring opportunities in the AI industry. It has inspired me to learn more about intelligent systems and their practical uses. I believe this experience will be useful in my future academic or professional journey.

## References

- Abramowski, N. (2022, November 28). *What is npm? The complete beginner's guide*. CareerFoundry. <https://careerfoundry.com/en/blog/web-development/what-is-npm/>
- Acsany, P. (2024, December 22). *Using Python's pip to manage your projects' dependencies*. Real Python. <https://realpython.com/what-is-pip/>
- Al-Fedaghi, S. (2021). UML sequence diagram: an alternative model. *arXiv preprint arXiv:2105.15152*. doi: <https://doi.org/10.48550/arXiv.2105.15152>
- Anisya, A., & Nugroho, F. (2021). Analysis of Back-End Requirements of a Web-Based Application to Identify Damage for Building Structures. *IJISTECH (International Journal of Information System and Technology)*, 5(4), 431-435.
- Argano. (2022, March 21). *What Is Oracle Analytics Cloud? Features, Benefits & More*. Retrieved from <https://argano.com/insights/articles/what-is-oracle-analytics-cloud-features-benefits-and-more.html>
- Biswal, A. (2024, October 24). *What is Power BI?: Architecture, and Features Explained*. Retrieved from <https://www.simplilearn.com/tutorials/power-bi-tutorial/what-is-power-bi>
- Casarotto, C. (2021, May 10). *Google Data Studio: What It Is And How To Use It In 2024*. Retrieved from <https://rockcontent.com/blog/how-to-use-google-data-studio/>
- Choudhary, V. K., Kumar, G., Suman, S., & Kumar, A. (2024). A comprehensive review on data visualization techniques for real-time information. *2024 International Conference on Inventive Computation Technologies (ICICT)*, 866–871. <https://doi.org/10.1109/ICICT60155.2024.10544996>

- DAIR.AI. (2025, April 2024). *Few-Shot Prompting*. <https://www.promptingguide.ai/techniques/fewshot>
- Domino. (n.d.). *What is plotly? | domino data lab*. <https://domino.ai/data-science-dictionary/plotly>
- Erickson, J. (2024, August 2024). *MySQL: Understanding What It Is and How It's Used*. <https://www.oracle.com/my/mysql/what-is-mysql/>
- GitHub, Inc. (n.d.). *Github. Com help documentation*. <https://docs-internal.github.com/en>
- Gobov, D., & Huchenko, I. (2021). Software Requirements Elicitation Techniques Selection Method for the Project Scope Management. *ITPM*, 1-10.
- Google LLC. (2025, January 21). *Add charts and controls to your report*. Google Cloud. <https://cloud.google.com/looker/docs/studio/add-charts-and-controls-to-your-report?hl=en>
- Google LLC. (2025, January 30). *Invite others to your reports*. Google Cloud. <https://cloud.google.com/looker/docs/studio/invite-others-to-your-reports?hl=en>
- Google LLC. (2025, January 30). *Welcome to Looker Studio!* Google Cloud. <https://cloud.google.com/looker/docs/studio?hl=en>
- Google LLC. (2025, January 31). *About connectors, data sources, and credentials*. Google Cloud. <https://cloud.google.com/looker/docs/studio/about-connectors-data-sources-and-credentials?hl=en>
- Goswami, K., Mathur, P., Rossi, R., & Dernoncourt, F. (2025). *Plotgen: Multi-agent llm-based scientific data visualization via multimodal feedback* (No. arXiv:2502.00988). arXiv. <https://doi.org/10.48550/arXiv.2502.00988>

- Groq, Inc. (2021, November 18). *About groq—Fast ai inference*. <https://groq.com/about-us/>
- Groq, Inc. (n. d.). New AI Inference Speed Benchmark for Llama 3.3 70B, Powered by Groq. Retrieved from <https://groq.com/new-ai-inference-speed-benchmark-for-llama-3-3-70b-powered-by-groq>
- Groq, Inc. (n. d.). *Rate Limits*. GroqCloud Developer Console. <https://console.groq.com/docs/rate-limits>
- Groq, Inc. (n.d.). *Groq AI: What it is and how to use it*. <https://www.getguru.com/reference/what-is-groq-ai-and-how-to-use-it>
- Groq, Inc. (n.d.). *Groqdocs—Build fast*. <https://console.groq.com>
- Hegselmann, S., Buendia, A., Lang, H., Agrawal, M., Jiang, X., & Sontag, D. (2023). Tabllm: Few-shot classification of tabular data with large language models. *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, 5549–5581. <https://proceedings.mlr.press/v206/hegselmann23a.html>
- IBM. (2023, October 31). *What is langchain? | ibm*. <https://www.ibm.com/think/topics/langchain>
- Jinxin, S., Jiabao, Z., Yilei, W., Xingjiao, W., Jiawen, L., & Liang, H. (2023). Cgmi: Configurable general multi-agent interaction framework. *arXiv preprint arXiv:2308.12503*. <https://doi.org/10.48550/arXiv.2308.12503>
- Jupyter Team. (n.d.). *Project Jupyter Documentation—Jupyter Documentation 4.1.1 alpha documentation*. <https://docs.jupyter.org/en/latest/>
- Khurana, D., Koli, A., Khatter, K., & Singh, S. (2023). Natural language processing: state of the art, current trends and challenges. *Multimedia tools and applications*, 82(3), 3713-3744.



Kircher, R., & Zipp, L. (Eds.). (2022). Questionnaire for elicit Quantitative Data. Research methods in language attitudes (pp. 129-131). Cambridge: Cambridge University Press.

Krüger, G. (2024, August 8). Non-functional Requirements: What They Do, Examples, and Best Practices. Retrieved from <https://www.perforce.com/blog/alm/what-are-non-functional-requirements-examples#types-of-non-functional-requirements>

LangChain, Inc. (2024, January 17). *Langgraph*. LangChain Blog. <https://blog.langchain.com/langgraph/>

LangChain, Inc. (n.d.). langchain-ai/sql-query-system-prompt. <https://smith.langchain.com/hub/langchain-ai/sql-query-system-prompt>

LangChain, Inc. (n.d.). *Tools*. [https://python.langchain.com/docs/how\\_to/#tools](https://python.langchain.com/docs/how_to/#tools)

Lavanya, A., Sindhuja, S., Gaurav, L., Ali, W. (2023). A Comprehensive Review of Data Visualization Tools: Features, Strengths, and Weaknesses. *International Journal of Computer Engineering in Research Trends*, 10(1), 10-20.

Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., ... & Tu, Z. (2023). Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118*. doi: <https://doi.org/10.48550/arXiv.2305.19118>

MDN. (2024, December 19). *Css—Glossary* | *mdn*. <https://developer.mozilla.org/en-US/docs/Glossary/CSS>

MDN. (2024, December 19). *Html—Glossary* | *mdn*. <https://developer.mozilla.org/en-US/docs/Glossary/HTML>

- MDN. (2025, June 23). *What is JavaScript? - Learn web development | MDN*.  
[https://developer.mozilla.org/en-US/docs/Learn\\_web\\_development/Core/Scripting/What is JavaScript](https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/What_is_JavaScript)
- Mei, K., Li, Z., Xu, S., Ye, R., Ge, Y., & Zhang, Y. (2024). AIOS: LLM agent operating system. *arXiv e-prints*. doi:  
<https://doi.org/10.48550/arXiv.2403.16971>
- Meta Platforms, Inc. (n.d.). *Llama 3.1*. Model Cards & Prompt formats.  
[https://www.llama.com/docs/model-cards-and-prompt-formats/llama3\\_1/](https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_1/)
- Meta Platforms, Inc. (n.d.). *React*. <https://react.dev/>
- Microsoft Corporation, Inc. (2023, March 09). *Design Power BI reports for accessibility*. Power BI documentation. <https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-accessibility-creating-reports>
- Microsoft Corporation, Inc. (2024, January 24). *Quickstart: Connect to data in Power BI Desktop*. Power BI documentation.  
<https://learn.microsoft.com/en-us/power-bi/connect-data/desktop-quickstart-connect-to-data>
- Microsoft Corporation, Inc. (2024, March 18). *Share Power BI reports and dashboards with coworkers and others*. Power BI documentation.  
<https://learn.microsoft.com/en-us/power-bi/collaborate-share/service-share-dashboards>
- Microsoft Corporation, Inc. (2024, March 22). *Create and manage relationships in Power BI Desktop*. Power BI documentation.  
<https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-create-and-manage-relationships>

Microsoft Corporation, Inc. (2024, March 22). *What is Power BI?* Microsoft.

<https://learn.microsoft.com/en-us/power-bi/fundamentals/power-bi-overview>

Microsoft Corporation, Inc. (2025, January 06). *Dashboards for business users of the Power BI service.* Power BI documentation.

<https://learn.microsoft.com/en-us/power-bi/consumer/end-user-dashboards>

Microsoft Corporation, Inc. (n.d.). *Why did we build Visual Studio Code?*

<https://code.visualstudio.com/docs/editor/whyvscode>

Mohammadjafari, A., Maida, A. S., & Gottumukkala, R. (2024). From natural language to sql: Review of llm-based text-to-sql systems. *arXiv preprint arXiv:2410.01066*. doi: <https://doi.org/10.48550/arXiv.2410.01066>

*mysql-connector-python 9.3.0*. (n.d.). <https://pypi.org/project/mysql-connector-python/>

Nurcahya, D., Nurfauziah, H., & Dwiatmodjo, H. (2022). Comparison of Waterfall Models and Prototyping Models of Meeting Management Information Systems. *Jurnal Mantik*, 6(2), 1934-1939.

Oracle Corporation. (n. d.) *5.1 Connecting to and Disconnecting from the Server.*

<https://dev.mysql.com/doc/refman/8.0/en/connecting-disconnecting.html>

Oracle Corporation. (n. d.) *Upload and Connect.* Oracle Analytics Cloud.

<https://docs.oracle.com/en/cloud/paas/analytics-cloud/upload-data.html>

Oracle Corporation. (n. d.) *Visualize Data.* Oracle Analytics Cloud.

<https://docs.oracle.com/en/cloud/paas/analytics-cloud/visualize-data.html>

Oracle Corporation. (n. d.). *Analytics Cloud*. Oracle Cloud Infrastructure Documentation. <https://docs.oracle.com/en-us/iaas/analytics-cloud/index.html>

Oracle Corporation. (n. d.). *Build Reports and Dashboards*. Oracle Analytics Cloud. <https://docs.oracle.com/en/cloud/paas/analytics-cloud/build-reports-and-dashboards.html>

Oracle Corporation. (n. d.). *Model Data*. Oracle Analytics Cloud. <https://docs.oracle.com/en/cloud/paas/analytics-cloud/model-data-reports.html>

Oracle Corporation. (n.d.). *MySQL Workbench*. <https://www.mysql.com/products/workbench/>

Pallets. (n.d.). *Api—Flask documentation(3.1.x)*. <https://flask.palletsprojects.com/en/stable/api/>

Pandas (n.d.). *Pandas—Python data analysis library*. <https://pandas.pydata.org/>

Pandas. (n.d.). *Pandas—Python data analysis library*. <https://pandas.pydata.org/>

Pandas. (n.d.). *Pandas. Dataframe—Pandas 2. 3. 0 documentation*. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

Pargaonkar, S. (2023). A Comprehensive Research Analysis of Software Development Life Cycle (SDLC) Agile & Waterfall Model Advantages, Disadvantages, and Application Suitability in Software Quality Engineering. *International Journal of Scientific and Research Publications (IJSRP)*, 13(08), 345-358.

Plotly. (n.d.). *Plotly*. <https://plotly.com/python/>

Pryke, B. (2025, January 28). *How to use jupyter notebook: A beginner's tutorial*. Dataquest. <https://www.dataquest.io/blog/jupyter-notebook-tutorial/>

Python Software Foundation. (n.d.). *General python faq*. Python Documentation.  
<https://docs.python.org/3/faq/general.html>

Qin, L., Chen, Q., Feng, X., Wu, Y., Zhang, Y., Li, Y., ... & Yu, P. S. (2024). Large language models meet nlp: A survey. arXiv preprint arXiv:2405.12819. doi:  
<https://doi.org/10.48550/arXiv.2405.12819>

Rashkovits, R., & Lavy, I. (2021). Mapping Common Errors in Entity Relationship Diagram Design of Novice Designers. *International Journal of Database Management Systems*, 13(1), 1-19.

Ravikiran, A. S. (2024, July 8). *What is SQLite? Everything you need to know*. Simplilearn. <https://www.simplilearn.com/tutorials/sql-tutorial/what-is-sqlite>

*React*. (n.d.). Retrieved 26 June 2025, from <https://react.dev/>

Refsnes Data. (n.d.). CSS *Reference*.  
<https://www.w3schools.com/CSSref/index.php>

Ruan, J., Chen, Y., Zhang, B., Xu, Z., Bao, T., Mao, H., ... & Zhao, R. (2023). Tptu: Task planning and tool usage of large language model-based ai agents. NeurIPS 2023 Foundation Models for Decision Making Workshop.

Shen, Z. (2024). LLM With Tools: A Survey. arXiv preprint arXiv:2409.18807. doi:  
<https://doi.org/10.48550/arXiv.2409.18807>

Shi, S., Ren, T., & Hu, J. (2025). Visq2sql: Towards sql-driven data visualization via llms-grounded preference learning. *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 1–5. <https://doi.org/10.1109/ICASSP49660.2025.10888696>

Skender, F., Manevska, V. (2022). Data Visualization Tools-Preview and Comparison. *Journal of Emerging Computer Technologies*, 2(1), 30-35.

*Sqlite documentation*. (n.d.). <https://www.sqlite.org/docs.html>

Staff, C. (2024, December 11). *What Is Tableau? Features, Use Cases, and More*. <https://www.coursera.org/articles/tableau>

Sudiartha, I., Indrayana, I., Suasnawa, I. W., Atmaja, I., Indah, K. A. T., & Sunu, P. W. (2021). User Requirement and Use Case Diagram for Traveler Tracking Application in Tourist Destination. *User Requirement and Use Case Diagram for Traveler Tracking Application in Tourist Destination*, 1, 1376-1380. doi: <https://doi.org/10.5220/0010965700003260>

Tableau Software, LLC. (n. d.) *Analyze Data*. Tableau Desktop and Web Authoring Help. <https://help.tableau.com/current/pro/desktop/en-us/analyze.htm>

Tableau Software, LLC. (n. d.) *Automatically Build Views with Ask Data*. Tableau Desktop and Web Authoring Help. [https://help.tableau.com/current/pro/desktop/en-us/ask\\_data.htm](https://help.tableau.com/current/pro/desktop/en-us/ask_data.htm)

Tableau Software, LLC. (n. d.) *Build Data Views from Scratch*. Tableau Desktop and Web Authoring Help. [https://help.tableau.com/current/pro/desktop/en-us/building\\_overview.htm](https://help.tableau.com/current/pro/desktop/en-us/building_overview.htm)

Tableau Software, LLC. (n. d.) *Connect to Your Data*. Tableau Desktop and Web Authoring Help. <https://help.tableau.com/current/pro/desktop/en-us/basicconnectoverview.htm>

Tableau Software, LLC. (n. d.) *Create Dashboards*. Tableau Desktop and Web Authoring Help. <https://help.tableau.com/current/pro/desktop/en-us/dashboards.htm>

Tableau Software, LLC. (n. d.) *Create Stories*. Tableau Desktop and Web Authoring Help. <https://help.tableau.com/current/pro/desktop/en-us/stories.htm>

Tableau Software, LLC. (n. d.) *Tableau Product Overview*. Tableau Help. [https://help.tableau.com/current/tableau/en-us/tableau\\_product\\_overview.htm](https://help.tableau.com/current/tableau/en-us/tableau_product_overview.htm)

Talebirad, Y., & Nadiri, A. (2023). Multi-agent collaboration: Harnessing the power of intelligent llm agents. *arXiv preprint arXiv:2306.03314*. doi: <https://doi.org/10.48550/arXiv.2306.03314>

Wang, L., Zhang, S., Wang, Y., Lim, E.-P., & Wang, Y. (2023). Llm4vis: Explainable visualization recommendation using chatgpt. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 675–692. <https://doi.org/10.18653/v1/2023.emnlp-industry.64>

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837. [https://proceedings.neurips.cc/paper\\_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html?ref=https://githubhelp.com](https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html?ref=https://githubhelp.com)

*What is flask python—Python tutorial*. (n.d.). <https://pythonbasics.org/what-is-flask-python/>

Yang, Z., Gan, Z., Wang, J., Hu, X., Lu, Y., Liu, Z., & Wang, L. (2022). An empirical study of gpt-3 for few-shot knowledge-based vqa. *Proceedings*

*of the AAAI Conference on Artificial Intelligence*, 36(3), 3081–3089.  
<https://doi.org/10.1609/aaai.v36i3.20215>

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., ... & Wen, J. R. (2023).  
A survey of large language models. *arXiv preprint arXiv:2303.18223*. doi:  
<https://doi.org/10.48550/arXiv.2303.18223>



## **Appendix A: FYP I Meeting Logs**



**CPT6314 Final Year Project (FYP) 1 Meeting Log**  
**Trimester OCT / NOV 2024 (Trimester ID:2430)**

<b>Meeting Date:</b> 22 <sup>nd</sup> November 2024	<b>Meeting No.:</b> 1
<b>Meeting Mode:</b> Online	
<b>Project ID:</b> FYP01-SE-T2430-0033	<b>Project Type:</b> Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)

**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** Problem Formulation and Project Planning / Background Study or Literature Review / Requirement Analysis or Theoretical Framework / Design or Research Methodology / Prototype Development or Proof of Concept / Draft Report Completion

*(Please strike out the tasks, which are not applicable)*

**Details (in point form):**

- Problem Formulation and Project Planning:
  - Planned the phases of the project for FYP1 and FYP2
- Background Study:
  - Introduced theories, concepts, terms, and ideas that may be unfamiliar to the target audience
  - Introduced concepts that may have been borrowed from other disciplines that may be unfamiliar to the reader that needs explanation
  - Recognized similar system to the project or any existing software that uses the main concepts from the project

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** Problem Formulation and Project Planning / Background Study or Literature Review / Requirement Analysis or Theoretical Framework / Design or Research Methodology / Prototype Development or Proof of Concept / Draft Report Completion

*(Please strike out the tasks, which are not applicable)*

**Details (in point form):**

- Find research papers related to the system and finalize the background study
- Identify and conduct requirements analysis

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

- **Problem:** Unable to complete part of the abstract before completing the prototype
  - **Solution:** Wait until after the prototype has been developed
- **Problem:** Unable to access some existing dashboard
  - **Solution:** Rely on documentation and Youtube video of it

**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

Good

.....  
Supervisor's Signature.....  
Student's Signature.....  
Co-Supervisor's Signature  
(if applicable).....  
Company Supervisor's Signature  
(if applicable)**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student has to upload the soft copies of the meeting logs in eBwise and also attach them along with interim (FYP1) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 3 to Week 12). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and also for checking the attendance requirement of the student, by the FYP Committee.  
  
This also provide the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provide the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.
4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.



**CPT6314 Final Year Project (FYP) 1 Meeting Log**  
**Trimester OCT / NOV 2024 (Trimester ID:2430)**

<b>Meeting Date:</b> 5 <sup>th</sup> December 2024	<b>Meeting No.:</b> 2
<b>Meeting Mode:</b> Online	
<b>Project ID:</b> FYP01-SE-T2430-0033	<b>Project Type:</b> Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)

**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** ~~Problem Formulation and Project Planning /~~ Background Study / Requirement Analysis  
~~/ Design or Research Methodology /~~  
~~Prototype Development or Proof of Concept /~~ Draft Report Completion

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):**

*Background Study*

- Dissected the domain of the problem
- Analyzed recent research papers from the domain
- Identified the limitation of the state-of-the-art tools from the domain
- Proposed solutions to the identified limitation

*Requirement Analysis*

- Based on the tools gathered from the background study, analyzed the various use cases of the tools.

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** ~~Problem Formulation and Project Planning /~~ Background Study or Literature Review /  
 Requirement Analysis / ~~Design /~~ Prototype Development or Proof of Concept / ~~Draft Report~~  
 Completion

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):**

- Justify the proposed solutions through studies of the domains of the solution
- Organize and analyze the various features from the different tools
- Prepare elicitation techniques to gather requirements

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

- Limited access on some of the tools
  - o Rely on any documentation of the tools

**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

Good

.....  
Supervisor's Signature.....  
Student's Signature.....  
Co-Supervisor's Signature  
(if applicable).....  
Company Supervisor's Signature  
(if applicable)**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student has to upload the soft copies of the meeting logs in eBwise and also attach them along with interim (FYP1) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 3 to Week 12). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and also for checking the attendance requirement of the student, by the FYP Committee.

This also provide the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provide the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.

4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.



**CPT6314 Final Year Project (FYP) 1 Meeting Log**  
**Trimester OCT / NOV 2024 (Trimester ID:2430)**

<b>Meeting Date:</b> 19 <sup>th</sup> December 2024	<b>Meeting No.:</b> 3
<b>Meeting Mode:</b> Online	
<b>Project ID:</b> FYP01-SE-T2430-0033	<b>Project Type:</b> Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)



**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** ~~Problem Formulation and Project Planning / Background Study / Requirement Analysis / Design or Research Methodology /~~  
~~Prototype Development or Proof of Concept / Draft Report Completion~~

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):**

Requirement Analysis

- Justified the proposed solutions through studies of the domains of the solution
- Organized and analyzed various features from different tools
- Prepared elicitation techniques to gather requirements

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** ~~Problem Formulation and Project Planning / Background Study or Literature Review /~~  
~~Requirement Analysis / Design / Prototype Development or Proof of Concept / Draft Report Completion~~

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):**

Requirement Analysis

- Execute the elicitation process
- Finalize the functional and non-functional requirements

Design

- Define the use cases and architecture of the project

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

- Some identified existing systems from the research changed its name.
  - o Verified for any feature changes between the one from the research (old name) with the latest system's feature (new name)
- Lack of elaboration from the authors regarding the existing system's limitation
  - o Went through multiple references the limitations were sourced from

**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

Good

.....  
Supervisor's Signature.....  
Student's Signature.....  
Co-Supervisor's Signature  
(if applicable).....  
Company Supervisor's Signature  
(if applicable)**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student has to upload the soft copies of the meeting logs in eBwise and also attach them along with interim (FYP1) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 3 to Week 12). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and also for checking the attendance requirement of the student, by the FYP Committee.

This also provide the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provide the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.

4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.



**CPT6314 Final Year Project (FYP) 1 Meeting Log**  
**Trimester OCT / NOV 2024 (Trimester ID:2430)**

<b>Meeting Date:</b> 2 <sup>nd</sup> January 2025	<b>Meeting No.:</b> 4
<b>Meeting Mode:</b> Online	
<b>Project ID:</b> FYP01-SE-T2430-0033	<b>Project Type:</b> Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)

**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** ~~Problem Formulation and Project Planning / Background Study / Requirement Analysis / Design / Prototype Development or Proof of Concept / Draft Report Completion~~

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):**

Requirement Analysis

- Executed the elicitation process
- Finalized the functional requirements
- Defined some of the quality requirements
- Defined the requirements in terms of context diagram and level 0 DFD
- Defined the use cases and the high-level architecture of the system

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** ~~Problem Formulation and Project Planning / Background Study or Literature Review / Requirement Analysis / Design / Prototype Development or Proof of Concept / Draft Report Completion~~

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):**

Requirement Analysis

- Continue defining the quality requirements
- Describe and organize all the identified use cases (actors, scenarios, pre/postconditions)
- Start writing report for chapter 3

Design

- Create the sequence diagram using Visual Paradigm software
- Design the system interface

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

- Was not sure whether the use of LLM counts as external system or not
  - o Since it is integrated into the system, it is decided to be counted as the system, even though it's still dependent on external resource to get the model
- Hard time deciding the metrics for the non-functional requirements
  - o Do thorough research on existing model's response time, converting text to SQL, executing query and getting the data (might need more time for large database,

should consider limiting the amount of data from the db), and time for the model to process the data.

**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

Good



.....  
Supervisor's Signature



.....  
Student's Signature

.....  
Co-Supervisor's Signature  
(if applicable)

.....  
Company Supervisor's Signature  
(if applicable)

**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student has to upload the soft copies of the meeting logs in eBwise and also attach them along with interim (FYP1) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 3 to Week 12). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and also for checking the attendance requirement of the student, by the FYP Committee.  
  
This also provide the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provide the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.
4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.



**CPT6314 Final Year Project (FYP) 1 Meeting Log**  
**Trimester OCT / NOV 2024 (Trimester ID:2430)**

<b>Meeting Date:</b> 16 <sup>th</sup> January 2025	<b>Meeting No.:</b> 5
<b>Meeting Mode:</b> Online	
<b>Project ID:</b> FYP01-SE-T2430-0033	<b>Project Type:</b> Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)

**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** ~~Problem Formulation and Project Planning / Background Study / Requirement Analysis / Design / Prototype Development or Proof of Concept / Draft Report Completion~~

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):***Requirement Analysis*

- Defined the quality requirements
- Described and organized all the identified use cases (actors, scenarios, pre/postconditions)
- Started to design the report for chapter 3
- Modified the context diagram

*Design*

- Designed the sequence diagram for every use cases
- Started to design the system backend as part of designing the interface

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** ~~Problem Formulation and Project Planning / Background Study or Literature Review / Requirement Analysis / Design / Prototype Development or Proof of Concept / Draft Report Completion~~

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):***Requirement Analysis*

- Rework the context diagram

*Design*

- Start to design the system data
- Continue to design the system interface

*Prototype Development*

- Start to develop the system prototype

*Draft Report Completion*

- Continue to write report for chapter 3
- Continue to write report for chapter 4

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

- The are errors found on the context diagram
  - Received suggestions and modify the diagram accordingly

**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

Good



.....  
Supervisor's Signature



.....  
Student's Signature

.....  
Co-Supervisor's Signature  
(if applicable)

.....  
Company Supervisor's Signature  
(if applicable)

**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student has to upload the soft copies of the meeting logs in eBwise and also attach them along with interim (FYP1) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 3 to Week 12). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and also for checking the attendance requirement of the student, by the FYP Committee.  
  
This also provide the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provide the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.
4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.





**CPT6314 Final Year Project (FYP) 1 Meeting Log**  
**Trimester OCT / NOV 2024 (Trimester ID:2430)**

<b>Meeting Date:</b> 4 <sup>th</sup> February 2025	<b>Meeting No.:</b> 6
<b>Meeting Mode:</b> Online	
<b>Project ID:</b> FYP01-SE-T2430-0033	<b>Project Type:</b> Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)

**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** ~~Problem Formulation and Project Planning / Background Study /~~ Requirement Analysis / Design / Prototype Development or Proof of Concept / Draft Report Completion

***(Please strike out the tasks, which are not applicable)***

**Details (in point form):**

*Requirement Analysis*

- Reworked the context diagram and use case diagram

*Design*

- Designed the system data
- Designed the system interface

*Prototype development*

- Developed the system prototype based on its design

*Draft Report Completion*

- Finished writing report for chapter 1-6

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** ~~Problem Formulation and Project Planning / Background Study or Literature Review / Requirement Analysis / Design /~~ Prototype Development or Proof of Concept / Draft Report Completion

***(Please strike out the tasks, which are not applicable)***

**Details (in point form):**

Prototype Development

- Finish developing the prototype based on the system data and system interface

Draft Report Completion

- Finalize the content of the report
- Finish writing report with proper formatting

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

- Minor errors on the report content and report formatting error
  - Make adjustments and changes to the report based on the error found

**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

Need to expedite the deliverables.



.....  
Supervisor's Signature



.....  
Student's Signature

.....  
Co-Supervisor's Signature  
(if applicable)

.....  
Company Supervisor's Signature  
(if applicable)

**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student has to upload the soft copies of the meeting logs in eBwise and also attach them along with interim (FYP1) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 3 to Week 12). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and also for checking the attendance requirement of the student, by the FYP Committee.

This also provide the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provide the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.

4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.



**CPT6324 Project (FYP2) Meeting Log**  
**Trimester: March 2025 (Trimester ID:2510)**

<b>Meeting Date:</b> 11 <sup>th</sup> April 2025	<b>Meeting No.:</b> 1
<b>Meeting Mode:</b> ( <del>In-person</del> / Online)	
<b>Project ID:</b> FYP02-SE-T2510-0032	<b>Project Type:</b> <del>Research-based</del> / Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)

**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** Implementation / ~~Testing (Application-based projects)~~ or Evaluation of Findings and Research Contribution (Research-based projects) / ~~Commercialisation Proposal (Application-based projects)~~ or Research Paper (Research-based Projects) / ~~Draft Final Report / Final Report Completion~~

*(Please strike out the tasks, which are not applicable)*

**Details (in point form):**

*Implementation*

- Started preparing the software's development environment

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** Implementation / ~~Testing (Application-based projects)~~ or Evaluation of Findings and Research Contribution (Research-based projects) / ~~Commercialisation Proposal (Application-based projects)~~ or Research Paper (Research-based Projects) / ~~Draft Final Report / Final Report Completion~~

*(Please strike out the tasks, which are not applicable)*

**Details (in point form):**

*Implementation*

- Start implementing the software as planned

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

Problem	Solution
Unfamiliar with some of the tools and software for the development	Read the software and tool's documentation and watch tutorial videos of the tool and software

**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

-

.....  
Supervisor's Signature.....  
Student's Signature.....  
Co-Supervisor's Signature  
(if applicable).....  
Company Supervisor's Signature  
(if applicable)**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student must upload the soft copies of the meeting logs and attach them along with final (FYP2) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 4 to Week 14). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and for checking the attendance requirement of the student, by the FYP Committee.

This also provides the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provides the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.

4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.



**CPT6324 Project (FYP2) Meeting Log**  
**Trimester: March 2025 (Trimester ID:2510)**

<b>Meeting Date:</b> 25 <sup>th</sup> April 2025	<b>Meeting No.:</b> 2
<b>Meeting Mode:</b> ( <del>In-person</del> / Online)	
<b>Project ID:</b> FYP02-SE-T2510-0032	<b>Project Type:</b> <del>Research-based</del> / Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)

**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** Implementation / ~~Testing (Application-based projects)~~ or Evaluation of Findings and Research Contribution (Research-based projects) / ~~Commercialisation Proposal (Application-based projects)~~ or Research Paper (Research-based Projects) / ~~Draft Final Report / Final Report Completion~~

***(Please strike out the tasks, which are not applicable)***

**Details (in point form):**

*Implementation*

- Started implementing the software as planned

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** Implementation / ~~Testing (Application-based projects)~~ or Evaluation of Findings and Research Contribution (Research-based projects) / ~~Commercialisation Proposal (Application-based projects)~~ or Research Paper (Research-based Projects) / ~~Draft Final Report / Final Report Completion~~

***(Please strike out the tasks, which are not applicable)***

**Details (in point form):**

*Implementation*

- Continue implementing the software as planned

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

Problem	Solution
Unfamiliar with some of the tools and software for the development	Read the software and tool's documentation and watch tutorial videos of the tool and software



**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

-

.....  
Supervisor's Signature.....  
Student's Signature.....  
Co-Supervisor's Signature  
(if applicable).....  
Company Supervisor's Signature  
(if applicable)**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student must upload the soft copies of the meeting logs and attach them along with final (FYP2) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 4 to Week 14). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and for checking the attendance requirement of the student, by the FYP Committee.

This also provides the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provides the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.

4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.



**CPT6324 Project (FYP2) Meeting Log**  
**Trimester: March 2025 (Trimester ID:2510)**

<b>Meeting Date:</b> 9 <sup>th</sup> May 2025	<b>Meeting No.:</b> 3
<b>Meeting Mode:</b> ( <del>In-person</del> / Online)	
<b>Project ID:</b> FYP02-SE-T2510-0032	<b>Project Type:</b> <del>Research-based</del> / Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)

**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** Implementation / Testing (Application-based projects) or Evaluation of Findings and Research Contribution (Research-based projects) / Commercialisation Proposal (Application-based projects) or Research Paper (Research-based Projects) / Draft Final Report / Final Report Completion

***(Please strike out the tasks, which are not applicable)***

**Details (in point form):**

*Implementation*

- Continued implementing the software as planned
- Started preparing sample data and resources to test the AI model

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** Implementation / Testing (Application-based projects) or Evaluation of Findings and Research Contribution (Research-based projects) / Commercialisation Proposal (Application-based projects) or Research Paper (Research-based Projects) / Draft Final Report / Final Report Completion

***(Please strike out the tasks, which are not applicable)***

**Details (in point form):**

*Implementation*

- Continue implementing the software

*Testing*

- Start the testing phase of the project

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

Problem	Solution
Limited amount of token for the LLM hinders the development of the software	Be efficient with the amount of test run during the development process

**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

-

.....  
Supervisor's Signature.....  
Student's Signature.....  
Co-Supervisor's Signature  
(if applicable).....  
Company Supervisor's Signature  
(if applicable)**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student must upload the soft copies of the meeting logs and attach them along with final (FYP2) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 4 to Week 14). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and for checking the attendance requirement of the student, by the FYP Committee.

This also provides the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provides the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.

4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.



**CPT6324 Project (FYP2) Meeting Log**  
**Trimester: March 2025 (Trimester ID:2510)**

<b>Meeting Date:</b> 23 <sup>rd</sup> May 2025	<b>Meeting No.:</b> 4
<b>Meeting Mode:</b> ( <del>In-person</del> / Online)	
<b>Project ID:</b> FYP02-SE-T2510-0032	<b>Project Type:</b> <del>Research-based</del> / Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)

**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** Implementation / Testing (Application-based projects) or Evaluation of Findings and Research Contribution (Research-based projects) / Commercialisation Proposal (Application-based projects) or Research Paper (Research-based Projects) / Draft Final Report / Final Report Completion

*(Please strike out the tasks, which are not applicable)*

**Details (in point form):**

*Implementation*

- Continued implementing the software as planned
- Identified some problems regarding agent response
  - o Agent response not following the prompt
  - o Some agents consume too many token

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** Implementation / Testing (Application-based projects) or Evaluation of Findings and Research Contribution (Research-based projects) / Commercialisation Proposal (Application-based projects) or Research Paper (Research-based Projects) / Draft Final Report / Final Report Completion

*(Please strike out the tasks, which are not applicable)*

**Details (in point form):**

*Implementation*

- Continue implementing the software
- Solve identified problems regarding agent response
  - o Modify prompt and manually handle different output
  - o Reduce the amount of input token from each agent

*Testing*

- Start testing on some software modules

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

Problem	Solution
Limited amount of token for the LLM hinders the development of the software	Be efficient with the amount of test run during the development process
Some unforeseen responses from the LLM agent that halts the visualization process	Identify the problem and debug accordingly

**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

-

.....  
Supervisor's Signature.....  
Student's Signature.....  
Co-Supervisor's Signature  
(if applicable).....  
Company Supervisor's Signature  
(if applicable)**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student must upload the soft copies of the meeting logs and attach them along with final (FYP2) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 4 to Week 14). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and for checking the attendance requirement of the student, by the FYP Committee.

This also provides the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provides the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.

4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.



**CPT6324 Project (FYP2) Meeting Log**  
**Trimester: March 2025 (Trimester ID:2510)**

<b>Meeting Date:</b> 9 <sup>th</sup> June 2025	<b>Meeting No.:</b> 5
<b>Meeting Mode:</b> ( <del>In-person</del> / Online)	
<b>Project ID:</b> FYP02-SE-T2510-0032	<b>Project Type:</b> <del>Research-based</del> / Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)



**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** ~~Implementation / Testing (Application-based projects) or Evaluation of Findings and Research Contribution (Research-based projects) / Commercialisation Proposal (Application-based projects) or Research Paper (Research-based Projects) / Draft Final Report / Final Report Completion~~

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):**

- *Implementation*
  - o Continued implementing the software
  - o Solved identified problems regarding agent response
    - Modified prompt and manually handle different output
    - Reduced the amount of input token from each agent
- *Testing*
  - o Started testing on some software modules
- *Draft Final Report*
  - o Drafted the implementation section

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** ~~Implementation / Testing (Application-based projects) or Evaluation of Findings and Research Contribution (Research-based projects) / Commercialisation Proposal (Application-based projects) or Research Paper (Research-based Projects) / Draft Final Report / Final Report Completion~~

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):**

*Testing*

- Continue testing on remaining software module

*Draft Final Report*

- Start drafting the testing section

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

Problem	Solution
Limited amount of token for the LLM hinders the development of the software	Be efficient with the amount of test run during the development process
Some unforeseen responses from the LLM agent that halts the visualization process	Identify the problem and debug accordingly

**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

-

.....  
Supervisor's Signature.....  
Student's Signature.....  
Co-Supervisor's Signature  
(if applicable).....  
Company Supervisor's Signature  
(if applicable)**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student must upload the soft copies of the meeting logs and attach them along with final (FYP2) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 4 to Week 14). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and for checking the attendance requirement of the student, by the FYP Committee.

This also provides the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provides the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.

4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.



**CPT6324 Project (FYP2) Meeting Log**  
**Trimester: March 2025 (Trimester ID:2510)**

<b>Meeting Date:</b> 24 <sup>th</sup> June 2025	<b>Meeting No.:</b> 6
<b>Meeting Mode:</b> ( <del>In-person</del> / Online)	
<b>Project ID:</b> FYP02-SE-T2510-0032	<b>Project Type:</b> <del>Research-based</del> / Application-based
<b>Project Title :</b> LLM-Powered Dashboard using Multi-Agent Framework	
<b>Student ID :</b> 1211101583	<b>Student Name:</b> LUQMAN HAKIM BIN NOORAZMI
<b>Student Programme and Specialisation:</b> Bachelor of Computer Science (Software Engineering)	
<b>Supervisor Name:</b> Dr. Zarina bt Che Embi	<b>Co-Supervisor Name:</b> (if applicable)
<b>Collaborating Company:</b> (if applicable)	<b>Company Supervisor Name:</b> (if applicable)

**1. WORK DONE**

*[Please write the details of the work done, after the last meeting]*

**Tasks:** ~~Implementation~~ / Testing (Application-based projects) or ~~Evaluation of Findings and Research Contribution (Research-based projects)~~ / ~~Commercialisation Proposal (Application-based projects)~~ or ~~Research Paper (Research-based Projects)~~ / ~~Draft Final Report / Final Report Completion~~

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):**

- *Testing*
  - o Continued testing on remaining software module
- *Draft Final Report*
  - o Started drafting the testing section

**2. WORK TO BE DONE**

*[Please write the details of the work to be done, before the next meeting]*

**Tasks:** ~~Implementation~~ / Testing (Application-based projects) or ~~Evaluation of Findings and Research Contribution (Research-based projects)~~ / ~~Commercialisation Proposal (Application-based projects)~~ or ~~Research Paper (Research-based Projects)~~ / ~~Draft Final Report / Final Report Completion~~

**(Please strike out the tasks, which are not applicable)**

**Details (in point form):**

- Testing*
- Continue testing on remaining software module
- Draft Final Report*
- Continue drafting the testing section
  - Update the report to match the progress for part 2
  - Update the report based on moderator's recommendation
- Commercialisation Proposal*
- Start filling out the commercialization proposal
- Final Report Completion*
- Finalize the report

**3. PROBLEMS ENCOUNTERED AND SOLUTIONS**

*[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*

Problem	Solution
Limited amount of token for the LLM hinders the development of the software	Be efficient with the amount of test run during the development process
Some unforeseen responses from the LLM agent that halts the visualization process	Identify the problem and debug accordingly

**4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)**

-

.....  
Supervisor's Signature.....  
Student's Signature.....  
Co-Supervisor's Signature  
(if applicable).....  
Company Supervisor's Signature  
(if applicable)**IMPORTANT NOTES TO STUDENTS:**

1. Items 1 – 3 are to be completed by the students prior to the meeting. Item 4 is to be completed by the supervisor / co-supervisor / company supervisor.
2. Student must upload the soft copies of the meeting logs and attach them along with final (FYP2) report.  
Minimum requirement is SIX Meeting Logs (Period: Week 4 to Week 14). Students can have fortnightly meetings with the supervisor.
3. Log sheets provide the basis for evaluating the General Effort (Project Management, Attitude, and Technical Competency) of the student, by the supervisor and for checking the attendance requirement of the student, by the FYP Committee.

This also provides the student with feedback from the supervisor / co-supervisor / company supervisor on the tasks done and provides the plan for the upcoming tasks. This can provide the motivation for the student to give consistent and efficient effort throughout the period of FYP.

4. Student who fails to meet the minimum requirement (six nos.) of log sheets will not be allowed to submit FYP report.

## Appendix B: Turnitin Similarity Index Page

The screenshot displays the Turnitin interface for a document titled "Luqman Hakim | [Turnitin] 1211101583\_Dr Zarina\_FYP2\_Report.pdf". The document is identified by Project ID: FYP02-SE-T2510-0032. The main text area shows the "Abstract" section, which discusses data visualization tools and the integration of LLMs. The similarity index is 10%, indicated by a large red "10%" at the top of the right sidebar. Below this, a "Match Overview" table lists 10 matches with their respective similarity percentages.

Match Number	Source	Similarity Percentage
1	Submitted to Multimed... Student Paper	4%
2	Submitted to University... Student Paper	<1%
3	dev.to Internet Source	<1%
4	Submitted to University... Student Paper	<1%
5	Submitted to University... Student Paper	<1%
6	Submitted to Coventry ... Student Paper	<1%
7	Submitted to University... Student Paper	<1%
8	www.coursehero.com Internet Source	<1%
9	eprints.utar.edu.my Internet Source	<1%
10	Submitted to ESoft Met... Student Paper	<1%

Page: 1 of 223 Word Count: 30665 Text-Only Report High Resolution On



## Digital Receipt

This receipt acknowledges that Turnitin received your paper. Below you will find the receipt information regarding your submission.

The first page of your submissions is displayed below.

Submission author: Luqman Hakim  
Assignment title: Similarity check  
Submission title: [Turnitin] 1211101583\_Dr Zarina\_FYP2\_Report.pdf  
File name: \_Turnitin\_1211101583\_Dr\_Zarina\_FYP2\_Report.pdf  
File size: 3.33M  
Page count: 223  
Word count: 30,665  
Character count: 171,867  
Submission date: 27-Jun-2025 01:08AM (UTC+0800)  
Submission ID: 2700203769



Copyright 2025 Turnitin. All rights reserved.

## Appendix C: Agents Prompt

```
prompt = state["prompt"].invoke(
 {
 "dialect": "mysql",
 "top_k": 10,
 "table_info": state["schema"],
 "input": f"""
 {state["question"]}
 \n\nImportant: Available columns include: {columns}.
 When answering, use the most human-readable field available.
 If a field is a foreign key, join the referenced table and show its descriptive column.
 If the field is an ID, but there also exists a descriptive and human readable field of it within the
 same table, use the descriptive column.

 If the SQL fails, you are required to generate a new one while considering this improvement:
 {state["improvement"]}
 If the improvement is empty, ignore the improvement
 """
 }
)
```

```
prompt = f"""
 <|begin_of_text|>

 <|start_header_id|>system<|end_header_id|>

 You are a SQL query validator.
 You are required to verify the SQL query generated whether it is executable or not.

 You are required to response with only one (1) string:
 If it is executable, valid, correct, and doesn't require any fixes, response with: valid
 If it is empty, not executable, error, response with: invalid
 If the question does not relate to the database schema at all, response with: end
 """
```



```
SQL Schema: {state["schema"]}
Question: {state["question"]}
SQL Query: {state["query"]}
SQL Result: {state["result"]}

<|eot_id|>

<|start_header_id|>assistant<|end_header_id|>
"""
```

```
prompt = f"""
 <|begin_of_text|>

 <|start_header_id|>system<|end_header_id|>
 You are a SQL query validator.
 You are required to provide fix for the SQL query based on the SQL query and the error
message.
 The SQL should be syntactically correct, adhere to the schema provided, and following MySQL
dialect.

 f'Schema: {state["schema"]}'
 f'SQL Query: {state["query"]}'
 f'SQL Result: {state["result"]}'

 <|eot_id|>

 <|start_header_id|>assistant<|end_header_id|>
 """
```

```
prompt = f"""
 <|begin_of_text|>
```

```
<|start_header_id|>system<|end_header_id|>
```

You are a SQL query result parser.

Given the following user question, SQL query, and result from the SQL query, generate a dictionary containing the name of the column and the corresponding data.

Your response will be converted directly into a dataframe, hence your response should only be in form of dictionary only.

Do not generate a nested dictionary. Each column should be converted into a single key in the dictionary. Return only a single-level dictionary.

Keys are column names.

Values are lists of column values.

```
f'Question: {state["question"]}\n'
```

```
f'SQL Query: {state["query"]}
```

```
f'SQL Result: {state["result"]}'
```

Take into account the existing dictionary and the required improvement. Ignore if empty.

```
f'Existing Dictionary: {state["data"]}
```

```
f'Required Improvement: {state["improvement"]}'
```

```
<|eot_id|>
```

```
<|start_header_id|>assistant<|end_header_id|>
```

```
"""
```

```
prompt="""
```

```
<|begin_of_text|>
```

```
<|start_header_id|>system<|end_header_id|>
```

Environment: ipython

You are a data visualization expert.

You are given several tools that correspond to different types of data visualization graphs and charts.

Given the following user questions, the data, and the tools, choose the best tool to represent the data.

Question: {state["question"]}

Data: {state["data"]}

Tools: {state["tools"]}

Take into account the existing visualization and the required improvement. Ignore if empty:

Existing Visualization: {state["visualization"]}

Required Improvement: {state["improvement"]}

<|eot\_id|>

<|start\_header\_id|>assistant<|end\_header\_id|>

"""

```
prompt = (f"""
```

```
<|begin_of_text|>
```

```
<|start_header_id|>system<|end_header_id|>
```

```
You are a data analyst.
```

```
Given the following user question and the data, answer the user question using the data.
```

```
Make sure to go into detail and summarize the trends and main outcome.
```

```
You must response in normal string format.
```

```
Do not include the raw SQL data inside your response.
```

```
Format it like this example:
```

```
Data Preview:
```

```
<text of data preview>
```

```
Trend:
```

```
<text of trend>
```

Main Outcome:

<text of main outcome>

Question: {state["question"]}

SQL Result: {state["result"]}

Take into account the existing analysis and the required improvement. Ignore if empty:

Existing Analysis: {state['analysis']}

Required Improvement: {state['improvement']}

<|eot\_id|>

<|start\_header\_id|>assistant<|end\_header\_id|>

"""

prompt=f"""

<|begin\_of\_text|>

<|start\_header\_id|>system<|end\_header\_id|>

you are an agent output validator.

you must verify the string and make sure the output of an agent must be in form of a dictionary.

verify whether the output is in the form of a dictionary, if yes, move to the next agent,

chooseVisualization. if the output is not in the form of dictionary, or if the output contains anything other than a dictionary, go back to the same agent with an improvement message.

your output must be in format of dictionary with the agent name and the improvement message. there are only 2 agents:

generateDF

chooseVisualization

you must output in the format of dictionary:

```
{{

 "agentName": "<agentName>",
 "improvementMessage": "<improvementMessage>"

}}
```

```
}}
```

```
verify this agent output: {state["data"]}
```

```
<|eot_id|>
```

```
<|start_header_id|>assistant<|end_header_id|>
```

```
=====
```